



UNIVERSIDAD AUTÓNOMA METROPOLITANA

Unidad Azcapotzalco

División de Ciencias Básicas e Ingeniería
Licenciatura en Ingeniería en Computación



Sistema para la gestión de formularios sobre los cursos y autoevaluaciones docentes en la División de Ciencias y Artes para el Diseño (CyAD)

Proyecto Tecnológico

Reporte Final del Proyecto de Integración

Trimestre 2026-Primavera

Zuriel Godínez Ramos
2202003916
al2202003916@azc.uam.mx

M. en C. César Benavides Alvarez
Profesor Asociado
Departamento de Electrónica
cesarbenavidez@azc.uam.mx

M. en C. Josué Figueroa González
Profesor Asociado
Departamento de Sistemas
jfgo@azc.uam.mx

27 de septiembre de 2026

Declaratoria

Yo, César Benavides Álvarez, declaro que aprobé el contenido del presente Reporte de Proyecto de Integración y doy mi autorización para su publicación en la Biblioteca Digital, así como en el Repositorio Institucional de la UAM Azcapotzalco.

César Benavides Álvarez

Yo, Josué Figueroa González, declaro que aprobé el contenido del presente Reporte de Proyecto de Integración y doy mi autorización para su publicación en la Biblioteca Digital, así como en el Repositorio Institucional de la UAM Azcapotzalco.

Josué Figueroa González

Yo, Zuriel Godinez Ramos, doy mi autorización a la Coordinación de Servicios de Información de la Universidad Autónoma Metropolitana, Unidad Azcapotzalco, para publicar el presente documento en la Biblioteca Digital, así como en el Repositorio Institucional de la UAM Azcapotzalco.

Zuriel Godinez Ramos

Resumen

En la División de Ciencias y Artes para el Diseño (CyAD) de la Universidad Autónoma Metropolitana Unidad Azcapotzalco, los profesores entregan cada trimestre tres tipos de documentos: las *cartas temáticas* que describen el contenido de cada curso, los *requisitos de recuperación* para estudiantes en situación irregular y una *autoevaluación* única por trimestre de su propia labor docente. Hasta ahora este proceso se realizaba con formularios de Google y hojas de cálculo; esto implicaba tiempos muy largos para crear cada documento, dificultades para dar seguimiento y una alta probabilidad de errores, pues la mayoría de los datos se ingresaba manualmente.

Para atender esta necesidad, se desarrolló una aplicación web que permite a los profesores un manejo más sencillo de la información que deben capturar a lo largo del trimestre; como administrativo, se facilita el dar seguimiento a las actividades de los profesores y evaluar el cumplimiento por documento y por departamento.

La aplicación distingue dos roles autenticados, el administrador y el docente, y un acceso público sin sesión, cada uno con actividades bien delimitadas. El *administrador* inicia sesión con credenciales institucionales y se encarga de mantener actualizados los catálogos (departamentos, licenciaturas, posgrados, áreas y UEAs), dar de alta o de baja a los profesores, según se requiera, diseñar los formularios de autoevaluación (definiendo secciones, preguntas y rúbrica de puntajes), abrir y cerrar los periodos de captura para cada uno de los documentos, y consultar los reportes globales de cumplimiento que muestran qué profesores han entregado cada documento y cuáles siguen pendientes. El *docente* accede con sus credenciales para redactar, guardar como borrador y publicar sus propias cartas temáticas y requisitos de recuperación de cada UEA que imparte, responder la autoevaluación trimestral — cuyo nivel de desempeño se calcula automáticamente a partir de sus respuestas —, consultar el histórico de documentos que él mismo ha entregado en trimestres anteriores y despublicar un documento si necesita corregirlo.

El *público en general* (estudiantes, aspirantes, personal académico externo) no requiere iniciar sesión: consulta libremente los documentos publicados a través de la vista pública, filtrando por trimestre, programa académico y UEA para localizar la información del curso o grupo que le interesa.

La aplicación se organiza así alrededor de cinco funciones principales: acceso seguro con diferenciación de roles, captura y publicación de cartas temáticas, captura y publicación de requisitos de recuperación, llenado de autoevaluaciones con cálculo automático del nivel de desempeño, y generación de reportes de cumplimiento para la División.

El sistema se construyó con Django y Django REST Framework en el servidor, React en el navegador y PostgreSQL como base de datos. Se verificó con 197 pruebas automatizadas del *backend*, todas exitosas, y se entregaron todos los módulos comprometidos salvo la exportación de documentos a PDF y DOCX, que se sustituyó por la impresión desde el navegador y la exportación de reportes a XLSX; se añadieron, además, una vista pública de consulta, catálogos administrables con importación CSV y un constructor de formularios de autoevaluación. El sistema se validó en entorno local; no se realizó una prueba con usuarios ni se desplegó en el servidor institucional.

En este reporte se describe el proceso de desarrollo, los resultados obtenidos y las diferencias entre lo comprometido y lo entregado.

Índice

Resumen	1
Glosario y siglas	9
1. Introducción	12
2. Antecedentes	13
2.1. Trabajos relacionados	13
2.1.1. Uso de formularios digitales en la evaluación diagnóstica	13
2.1.2. La autoevaluación como estrategia de mejora docente	13
2.1.3. Configuración de formularios para evaluar atributos de egreso	13
2.1.4. Gestión escolar por trimestres en una unidad educativa	13
2.1.5. Gestión académica con el patrón Modelo–Vista–Controlador	14
2.1.6. Gestión administrativa en una escuela secundaria	14
3. Justificación	16
4. Objetivos	17
4.1. Objetivo General	17
4.2. Objetivos Específicos	17
5. Marco teórico	18
5.1. Documentos académicos gestionados dentro del sistema	18
5.2. Aplicación web y arquitectura cliente–servidor	20
5.3. Patrón Modelo–Vista–Controlador (MVC)	20
5.4. REST y APIs sobre HTTP	20
5.5. HTTP, <i>endpoints</i> , códigos de estado y CORS	20
5.6. Python y Django	21
5.7. Django REST Framework (DRF)	21
5.8. OpenAPI y <i>Swagger UI</i>	22
5.9. PostgreSQL	22
5.10. <i>Constraints</i> declarativos: unicidad compuesta, XOR y unicidad parcial	22
5.11. React	23
5.12. Vite	23
5.13. Tailwind CSS	24
5.14. Bibliotecas complementarias del frontend	24
5.15. Patrones frontend: SPA con <i>guards</i> , <i>interceptores</i> y estado de servidor	24
5.16. Componentes de interfaz: <i>form-builder</i> , cascada, acordeón, <i>dashboard</i> y <i>badge</i>	25
5.17. Autenticación con JSON Web Tokens (JWT)	25
5.18. Control de acceso basado en roles (RBAC)	26
5.19. Seguridad web y OWASP Top 10	27
5.20. Formatos de intercambio de datos	27
5.21. Máquina de estados y ciclo de vida de artefactos	28

5.22. Versionado inmutable y <i>snapshots</i> históricos	28
5.23. Escalas de medición: Likert y niveles de desempeño	29
5.24. Desarrollo incremental y control de versiones	29
5.25. Pruebas automatizadas	29
6. Desarrollo del proyecto	30
6.1. Arquitectura general	30
6.1.1. Estructura de directorios	31
6.2. Metodología y evolución del desarrollo	31
6.3. Modelo de datos	32
6.4. Módulo 1: autenticación y gestión de usuarios	37
6.4.1. Autenticación con JWT	37
6.4.2. Creación de un profesor con acceso al sistema	37
6.4.3. Roles y permisos	38
6.5. Módulo 2: catálogos institucionales	38
6.5.1. Precarga inicial y <i>fixtures</i>	39
6.5.2. Importación masiva de UEAs desde CSV	39
6.5.3. Restricciones de integridad de UEA	40
6.6. Módulo 3: habilitación de periodos por recurso	40
6.6.1. Invariante “a lo sumo uno activo por recurso”	41
6.6.2. Interfaz de usuario	41
6.6.3. Efectos sobre otros módulos	41
6.6.4. Eliminación protegida	41
6.7. Módulo 4: cartas temáticas y requisitos de recuperación	42
6.7.1. Creación y publicación desde el rol profesor	42
6.7.2. Reglas de propiedad y ediciones fuera de tiempo	42
6.7.3. Vista del administrador	43
6.8. Módulo 5: <i>form-builder</i> y autoevaluación docente	43
6.8.1. Flujo operativo del administrador (<i>form-builder</i>)	43
6.8.2. Máquina de estados del formulario	44
6.8.3. Sistema de puntuación (<i>scoring</i> ponderado)	45
6.8.4. Privacidad de la ponderación mientras se contesta	46
6.8.5. Flujo operativo del profesor	46
6.9. Módulo 6: reportes de cumplimiento y descargas XLSX	46
6.9.1. Endpoints de agregación	46
6.9.2. Generación de XLSX del lado del cliente	47
6.10. Módulo 7: vista pública de exploración	47
6.10.1. Cascada de tres pasos	48
6.10.2. Vista imprimible	48
6.11. Aspectos transversales	48
6.11.1. Documentación viva de la API	48
6.11.2. Formato de error unificado	49
6.11.3. Pruebas automatizadas	49
6.12. Seguridad	49

7. Resultados	51
7.1. Métricas del código y de la base de datos	51
7.2. Trazabilidad con los objetivos	52
7.3. Métricas de pruebas	52
7.4. Evidencia visual del sistema en operación	53
7.4.1. Autenticación	53
7.4.2. Vista Docente	53
7.4.3. Vista Administrador	53
7.4.4. Vista pública	53
7.5. Documentación de la API	54
8. Análisis y discusión de resultados	55
8.1. Comparación entregado vs. propuesta	55
8.2. Discusión de las brechas frente a la propuesta	56
8.2.1. Identidad por email en lugar de número económico	56
8.2.2. Ausencia de exportación a PDF/DOCX en backend	56
8.2.3. Sin restablecimiento de contraseña autoservicio	57
8.3. Elementos entregados que no estaban en la propuesta	57
8.4. Limitaciones del sistema actual	57
9. Conclusiones	59
9.1. Cumplimiento por objetivo	59
9.2. Aprendizajes técnicos	59
9.3. Trabajo futuro	60
10. Referencias	62
11. Apéndices	64
11.1. Código fuente	64
11.1.1. Estructura del repositorio	64
11.1.2. Backend — Modelo de Usuario con roles	65
11.1.3. Backend — Perfil de Profesor y Periodo	65
11.1.4. Backend — JWT y throttling	67
11.1.5. Backend — Documento académico base	67
11.1.6. Backend — Modelos del <i>form-builder</i>	68
11.1.7. Backend — Captura de respuestas y baremo	69
11.1.8. Backend — Cálculo ponderado del puntaje de autoevaluación	71
11.1.9. Backend — Permisos por rol	73
11.1.10. Frontend — Cliente HTTP con <i>refresh</i> automático de JWT	73
11.1.11. Frontend — Guard de rutas por rol	74
11.1.12. Frontend — Renderizador dinámico de preguntas	74
11.1.13. Backend — Comando de importación CSV para UEA	74
11.1.14. Backend — Constraint XOR y unicidad parcial de UEA	75
11.1.15. Backend — Ciclo de vida de Formulario	75
11.1.16. Backend — <i>Login</i> con distinción de cuenta desactivada	77

11.1.17.Frontend — Generación de XLSX por periodo	77
11.1.18.Nota sobre el listado completo	78
11.2. Entregables del plan de trabajo	78
11.3. Manual de instalación	80
11.3.1. Introducción	80
11.3.2. Prerrequisitos	80
11.3.3. Clonado del repositorio	82
11.3.4. Instalación mínima sin datos de demo	82
11.3.5. Datos de demostración (opcional)	85
11.3.6. Configuración del frontend	86
11.3.7. Verificación	86
11.3.8. Comandos administrativos útiles	87
11.3.9. Despliegue en servidor de producción	87
11.3.10.Solución de problemas	92
11.4. Manual de usuario	93
11.4.1. Introducción	93
11.4.2. Uso — Vista pública (sin iniciar sesión)	93
11.4.3. Acceso al sistema	94
11.4.4. Uso — rol Administrador	94
11.4.5. Uso — rol Profesor	97
11.4.6. Preguntas frecuentes	98

Índice de figuras

1.	Arquitectura de tres capas del sistema.	31
2.	Diagrama de casos de uso del sistema.	99
3.	Diagrama Entidad-Relación del sistema (agrupado por app Django).	100
4.	Modal “Nuevo Profesor” desde /admin/profesores.	100
5.	Importación masiva de UEAs desde CSV en /admin/catalogos/uea.	101
6.	/admin/catalogos/periodos: tabla con <i>chips</i> de banderas por recurso y modal “Nuevo periodo”.	101
7.	Máquina de estados del Formulario de autoevaluación. Las transiciones que preservan la versión se dibujan a la izquierda entre PUBLICADO y CERRADO; la revisión con incremento de versión (publicar_revision) se traza como arista externa.	102
8.	Panel “Niveles de Desempeño” del constructor de autoevaluaciones.	102
9.	Pestaña “Estadísticas” del constructor con las respuestas ya enviadas: promedio general, porcentaje por sección, distribución por nivel de desempeño y por pregunta.	103
10.	Pantalla de inicio de sesión.	103
11.	<i>Dashboard</i> del profesor: tarjetas de resumen y listas agrupadas por periodo.	104
12.	Formulario de creación/edición de carta temática.	104
13.	Formulario de requisitos de recuperación.	105
14.	Formulario de autoevaluación con renderizado dinámico de preguntas.	105
15.	<i>Dashboard</i> del administrador: KPIs globales y cumplimiento por departamento.	106
16.	Gestión de profesores: crear, editar, restablecer contraseña.	106
17.	Constructor de formularios de autoevaluación (<i>form-builder</i>) con secciones y preguntas ponderadas.	107
18.	Panel de reportes con descarga en XLSX.	107
19.	Portada pública: elección del tipo de documento a consultar.	108
20.	Vista pública <i>Explorar</i> : cascada periodo→programa→UEA con acordeón de documentos.	108
21.	Vista pública de una carta temática publicada, con formato imprimible.	109
22.	Portada pública: elección del tipo de documento a consultar.	109
23.	Vista pública “Explorar Carta Temática”, paso 1: selección de periodo.	110
24.	Consulta pública de Cartas Temáticas: periodo y licenciatura seleccionados, UEA con documento expandido.	110
25.	Consulta pública de Evaluaciones de Recuperación con UEA expandida.	111
26.	Vista pública imprimible de una carta temática publicada.	111
27.	Pantalla de inicio de sesión.	112
28.	Dashboard del administrador con KPIs globales y cumplimiento por departamento.	112
29.	Lista de profesores registrados.	113
30.	Modal “Nuevo Profesor” con datos de ejemplo listos para guardar.	113
31.	Modal “Editar Perfil del Profesor”.	114
32.	Modal “Restablecer contraseña” de un profesor.	114
33.	Nuevo departamento: clave y nombre.	115

34.	Nueva licenciatura: clave, nombre y departamento.	115
35.	Nuevo posgrado: clave, nombre y departamento.	116
36.	Nueva área: nombre y descripción (permite distinguir áreas homónimas). .	116
37.	Alta individual de una UEA.	117
38.	Importación masiva de UEA desde un archivo CSV.	117
39.	Gestión de periodos: tabla con los <i>chips</i> de banderas por recurso y modal de alta.	118
40.	Lista de formularios de autoevaluación con estado, periodo, preguntas y respuestas.	118
41.	Modal “Nuevo Formulario de Autoevaluación”.	119
42.	Constructor de formularios: pestaña “Preguntas” con secciones plegables y pesos.	119
43.	Constructor de formularios: pestaña “Niveles de Desempeño”.	120
44.	Constructor de formularios: pestaña “Estadísticas” con las respuestas ya enviadas.	120
45.	Panel de reportes con selección de tipo, periodo y departamento.	121
46.	Dashboard del profesor.	121
47.	Formulario de creación de carta temática.	122
48.	Formulario de creación de evaluación de recuperación.	122
49.	Formulario de autoevaluación renderizado dinámicamente para el profesor.	123

Índice de cuadros

1.	Comparación cualitativa de los trabajos relacionados con el sistema desarrollado.	14
2.	Métricas del código entregado.	51
3.	Trazabilidad entre objetivos específicos, funcionalidad y evidencia.	52
4.	Pruebas automatizadas y cobertura por módulo.	52
5.	Grado de cumplimiento por componente respecto a la propuesta.	55
6.	Entregables prometidos en la propuesta y su estado real.	78

Glosario y siglas

Este apartado reúne, como referencia rápida, la terminología institucional UAM-A/CyAD, las siglas técnicas y las bibliotecas mencionadas a lo largo del reporte.

Terminología institucional UAM-A / CyAD

UAM Universidad Autónoma Metropolitana

División Nivel organizativo intermedio entre la Unidad y los Departamentos. En este proyecto: División de Ciencias y Artes para el Diseño (CyAD).

Departamento Unidad organizativa académica que agrupa profesores por área disciplinar. CyAD tiene cuatro: *Evaluación del Diseño en el Tiempo, Investigación y Conocimiento para el Diseño, Procesos y Técnicas de Realización y Medio Ambiente*.

Licenciatura Programa de estudios de pregrado (ej. Arquitectura, Diseño de la Comunicación Gráfica, Diseño Industrial).

Posgrado Programa de estudios de nivel superior a la licenciatura.

Coordinación de Estudios Instancia académico-administrativa responsable de la operación de un programa docente.

Plan de estudios Documento oficial que fija las UEAs, su secuencia y sus créditos para una licenciatura o posgrado.

UEA *Unidad de Enseñanza-Aprendizaje*; corresponde a una asignatura en otros sistemas académicos. Se identifica por una clave numérica (ej. 1400027).

Área de conocimiento Agrupación temática de las UEAs dentro de un programa (por ejemplo “Optativas de interés profesional”, subdividida en áreas de concentración como Urbanismo o Diseño Ambiental). Distinta del Departamento.

Trimestre Unidad temporal académica de la UAM. Un plan de estudios se cursa nominalmente en 12 trimestres.

Periodo YY-{I,P,O} Nomenclatura del trimestre en el sistema: YY son los dos últimos dígitos del año y la letra codifica Invierno, Primavera u Otoño, respectivamente. Por ejemplo, 26-P designa el trimestre de Primavera de 2026.

Modalidad Forma en que se imparte una UEA: presencial, remoto o mixto.

Evaluación global Evaluación única prevista por la normatividad UAM para estudiantes que no aprobaron por vía ordinaria.

Número económico Identificador numérico único que la UAM asigna a cada trabajador académico.

Siglas técnicas

ACID *Atomicity, Consistency, Isolation, Durability* — propiedades que garantizan la corrección de las transacciones en un RDBMS.

API *Application Programming Interface* — contrato de operaciones que un componente expone a otros.

CI/CD *Continuous Integration / Continuous Delivery* — automatización de compilación, pruebas y despliegue tras cada cambio versionado.

CORS *Cross-Origin Resource Sharing* — protocolo del navegador que autoriza peticiones entre orígenes distintos (ver § 5.5).

CRUD *Create, Read, Update, Delete* — las cuatro operaciones básicas sobre un recurso persistente.

CSRF *Cross-Site Request Forgery* — ataque en el que un sitio malicioso induce al navegador de una víctima a enviar peticiones autenticadas a otro sitio.

DOM *Document Object Model* — representación en árbol de un documento HTML manipulable por JavaScript.

E2E *End-to-end* — prueba que ejercita el sistema completo desde la perspectiva del usuario.

FK *Foreign Key* — clave foránea; columna que referencia la clave primaria de otra tabla.

HMAC *Hash-based Message Authentication Code* — construcción que combina una función *hash* (SHA-256 en este proyecto) con una clave secreta para firmar mensajes.

HMR *Hot Module Replacement* — técnica de Vite que reemplaza módulos en el navegador sin recargar la página.

HTTP / HTTPS *HyperText Transfer Protocol* (y su variante cifrada con TLS) — protocolo de aplicación de la web.

JWT *JSON Web Token* — token firmado usado como credencial de sesión sin estado (RFC 7519).

KPI *Key Performance Indicator* — indicador clave de desempeño mostrado en los *dashboards*.

LDAP *Lightweight Directory Access Protocol* — protocolo estándar de directorios institucionales de usuarios.

ORM *Object-Relational Mapper* — capa que traduce entre objetos y filas de una base de datos relacional.

PBKDF2 *Password-Based Key Derivation Function 2* — algoritmo estándar para derivar una huella robusta a partir de una contraseña.

PDF *Portable Document Format* — formato de documento paginado listo para imprimir.

DOCX Formato de documento de Microsoft Word.

RBAC *Role-Based Access Control* — control de acceso basado en roles.

RDBMS *Relational Database Management System* — gestor de base de datos relacional (PostgreSQL en este proyecto).

REST *Representational State Transfer* — estilo arquitectónico para APIs sobre HTTP.

SMTP *Simple Mail Transfer Protocol* — protocolo estándar de envío de correo electrónico.

SPA *Single Page Application* — aplicación web cuya navegación ocurre sin recargar la página.

SSO *Single Sign-On* — mecanismo de inicio de sesión unificado entre varios sistemas.

SQL *Structured Query Language* — lenguaje estándar de consulta relacional.

TLS *Transport Layer Security* — protocolo que cifra el canal HTTP en HTTPS.

UI / UX *User Interface / User Experience* — interfaz de usuario / experiencia de usuario.

URI / URL *Uniform Resource Identifier / Locator* — identificador (y localizador) de un recurso en la web.

WSGI *Web Server Gateway Interface* — especificación de interfaz entre servidores web y aplicaciones Python.

XLSX Formato de libro de Microsoft Excel.

XSS *Cross-Site Scripting* — inyección de script malicioso en una página vista por otro usuario.

YAML *YAML Ain't Markup Language* — formato jerárquico textual usado por el esquema OpenAPI.

Bibliotecas y herramientas mencionadas

SheetJS (xlsx) Genera libros XLSX en el navegador; se usa para descargar reportes desde el rol administrador.

drf-spectacular Genera la especificación OpenAPI y la interfaz Swagger UI desde el código de DRF.

django-cors-headers Middleware que gestiona la lista blanca de orígenes CORS.

django-filter Añade filtros declarativos por *query string* a los *ViewSet*s de DRF.

python-decouple Lee la configuración desde archivos `.env`.

psycopg2 Adaptador PostgreSQL para Python que usa el ORM de Django.

djangoestframework-simplejwt Implementación de JWT (*access* + *refresh* con rotación) para DRF.

factory-boy Construcción de instancias de prueba con datos consistentes.

coverage.py* y *pytest-cov Miden la cobertura de código ejecutado por las pruebas.

pytest-django Integración de *pytest* con Django (fixtures de BD, cliente de pruebas, aislamiento).

WeasyPrint* y *ReportLab Generadores de PDF del lado del servidor considerados y descartados: el primero por sus dependencias nativas (GTK/Pango) y el segundo por requerir una maquetación programática manual.

Gunicorn* y *Nginx *Application server* WSGI y servidor HTTP inverso recomendados para el despliegue productivo (fuera del alcance actual).

Vitest*, *Jest*, *React Testing Library Herramientas típicas para probar componentes React; se mencionan como trabajo futuro (el proyecto aún no las incorpora).

Lighthouse* y *axe-core Herramientas de auditoría de rendimiento y accesibilidad, mencionadas como línea de trabajo pendiente.

Google Forms* / *Google Sheets Combinación que la Coordinación usaba antes del sistema (fuente del formulario ad hoc de autoevaluación y de la hoja de cálculo con los resultados); se referencia como motivación del proyecto.

ewpage

1. Introducción

En la Universidad Autónoma Metropolitana Unidad Azcapotzalco, particularmente en la División de Ciencias y Artes para el Diseño (CyAD), los profesores son responsables de llenar manualmente formularios en Google Forms que luego son capturados en hojas de cálculo para gestionar documentos clave como las cartas temáticas, las autoevaluaciones docentes y los requisitos de recuperación para alumnos. Este proceso, repetitivo y propenso a errores, dificulta la organización y el seguimiento adecuado de la información académica. Para resolver este problema, la propuesta de proyecto planteó el desarrollo de un sistema digital dirigido principalmente a los docentes, que automatice el llenado de datos académicos una vez que el usuario se ha identificado y selecciona el documento con el que va a trabajar. El sistema desarrollado permite a los profesores: (1) gestionar cartas temáticas, (2) realizar su autoevaluación docente al finalizar el trimestre, y (3) publicar requisitos de recuperación. Además, facilita a los administradores el acceso a las actividades realizadas por los docentes y la generación de reportes sobre el cumplimiento de estas actividades. El resto del reporte describe los antecedentes, el marco teórico, el desarrollo, los resultados y su contraste con lo comprometido en la propuesta.

2. Antecedentes

2.1. Trabajos relacionados

A continuación se revisan seis trabajos que abordan, desde ángulos distintos, la captura digital de información académica, la autoevaluación docente o la gestión escolar. Para cada uno se indica el problema que atiende, la solución adoptada y su relación con el sistema aquí desarrollado.

2.1.1. Uso de formularios digitales en la evaluación diagnóstica

Leyva, Pérez y Pérez [1] emplean Google Forms para aplicar evaluaciones diagnósticas a estudiantes de la Licenciatura en Turismo. Su interés es mostrar que una herramienta gratuita y sin requisitos técnicos basta para recolectar respuestas, concentrarlas de forma automática en una hoja de cálculo y analizarlas sin capturarlas de nuevo. Es, precisamente, el tipo de herramienta que la División de CyAD utilizaba antes de este proyecto, con la limitación de que no distingue perfiles de usuario ni valida los datos contra catálogos institucionales.

2.1.2. La autoevaluación como estrategia de mejora docente

Echeverría [2] estudia la autoevaluación desde la pedagogía: la plantea como un ejercicio de reflexión con el que el profesor reconoce sus fortalezas y sus áreas de oportunidad en el aula. No propone una herramienta informática, pero aporta el fundamento conceptual para que la autoevaluación trimestral del sistema no sea solo un trámite administrativo, sino un insumo de mejora.

2.1.3. Configuración de formularios para evaluar atributos de egreso

Cuautle [3] desarrolló, como proyecto terminal en la UAM Azcapotzalco, un sistema que permite configurar formularios y registrar los resultados de la evaluación de atributos de egreso por UEA. Se trata de un proceso que antes era manual y, por ello, propenso a errores de captura. Su solución precarga datos, valida al usuario y arma el formulario según la UEA y los atributos elegidos. Es el antecedente más cercano a este trabajo por su contexto institucional y por la configuración dinámica de formularios; difiere en que se orienta a la evaluación de atributos y no a los documentos que cada profesor entrega por trimestre.

2.1.4. Gestión escolar por trimestres en una unidad educativa

Callisaya y Aland [4] parten de un diagnóstico en una unidad educativa de El Alto, Bolivia, donde no existía un registro virtual de la información escolar. Construyen una aplicación que concentra los datos pedagógicos y permite consultarlos desde cualquier dispositivo, lo que mejora su disponibilidad y su trazabilidad. Su foco es el seguimiento de los estudiantes, no la documentación que produce el profesorado.

2.1.5. Gestión académica con el patrón Modelo–Vista–Controlador

Rojas et al. [5] construyen módulos para administrar planes de estudio, carga académica y disponibilidad docente, y organizan el desarrollo con el patrón MVC para separar datos, lógica e interfaz. De este trabajo se retoma la idea de estructurar el sistema por dominios; se diferencia en que su alcance es la planeación académica y no la captura de formularios.

2.1.6. Gestión administrativa en una escuela secundaria

González [6] atiende el crecimiento de la matrícula de una escuela secundaria, que había multiplicado los documentos manejados a mano y, con ellos, los errores de registro. Desarrolló un sistema que centraliza y automatiza los registros escolares, siguiendo la metodología de programación extrema (XP). Comparte con este proyecto la motivación de reducir la captura manual, aunque está dirigido al personal administrativo y no a los profesores.

Las similitudes y diferencias con el sistema desarrollado se presentan en la Tabla 1.

Tabla 1. Comparación cualitativa de los trabajos relacionados con el sistema desarrollado.

Ref.	Similitudes	Diferencias
[1]	■ Uso de formularios digitales para capturar información. ■ No requiere conocimientos técnicos.	■ Permite trabajar individualmente o de forma colaborativa a distancia. ■ No tiene un sistema de autenticación con roles específicos.
[2]	■ Relevancia de la autoevaluación docente como herramienta de mejora. ■ Permitir que el docente identifique sus fortalezas y áreas de mejora.	■ Aborda la autoevaluación desde una perspectiva pedagógica, no tecnológica. ■ No plantea un sistema automatizado.
[3]	■ Captura estructurada de datos mediante formularios. ■ Autocompletado de formularios a partir de la información en una base de datos.	■ Enfocado en evaluar atributos de egreso por UEA. ■ Módulo para indicar los atributos de egreso a evaluar. ■ Módulo para asignar a un profesor a la UEA a evaluar.
[4]	■ Registro y visualización de información académica.	■ Orientado al seguimiento pedagógico de estudiantes, no a documentos docentes.

Ref.	Similitudes	Diferencias
[5]	■ Módulos especializados para distintos procesos académicos. ■ Manejo de información de los planes de estudios.	■ Gestión enfocada en planes de estudio y disponibilidad docente, no en formularios.
[6]	■ Centralización de datos. ■ Automatización de tareas administrativas.	■ Diseñado principalmente para el personal administrativo, no para profesores.

En síntesis, los trabajos revisados cubren por separado la captura digital de datos, el sustento pedagógico de la autoevaluación y la gestión académica o escolar, pero ninguno reúne en una sola herramienta lo que la División de CyAD necesita: acceso diferenciado por rol, formularios de autoevaluación configurables con puntuación automática, publicación de documentos por trimestre y reportes de cumplimiento para la administración. Ese es el hueco que atiende el sistema descrito en este reporte.

3. Justificación

El principal beneficio esperado de este sistema es la optimización del tiempo de los profesores, al simplificar la captura de la información que deben entregar cada trimestre. Este beneficio se plantea a partir del proceso previo (formularios y hojas de cálculo) y no fue medido en este proyecto (véase la Sección efsec:análisis). Además, esta solución contribuye a la modernización de la administración académica, alineándose con la tendencia de digitalización en las instituciones educativas [7]. Un sistema bien estructurado no solo beneficia a los docentes, sino que también permite a la administración académica tener un mejor control y seguimiento de la información generada, promoviendo una gestión más eficiente y transparente.

4. Objetivos

4.1. Objetivo General

Desarrollar un sistema que permita a los profesores de la división de CyAD gestionar de manera eficiente sus datos académicos, la información sobre sus cursos impartidos, los requisitos de sus evaluaciones de recuperación, y su autoevaluación al final de cada trimestre.

4.2. Objetivos Específicos

1. Permitir el acceso al sistema mediante una autenticación que permita la identificación y uso del sistema según el rol del usuario.
2. Crear, modificar y consultar información sobre cartas temáticas y evaluaciones de recuperación por parte de los profesores.
3. Registrar información sobre la autoevaluación de los profesores al finalizar cada trimestre.
4. Crear reportes que faciliten al personal administrativo el seguimiento del cumplimiento de las actividades docentes.

5. Marco teórico

En esta sección se describen detalladamente los conceptos, tecnologías y herramientas necesarios para llevar a cabo la implementación del proyecto. Inicialmente se describen los documentos académicos gestionados dentro del sistema (la terminología institucional y las siglas se reúnen en el Glosario, previo a la Introducción); posteriormente, se definen conceptos relacionados con las tecnologías utilizadas para el desarrollo del sistema.

5.1. Documentos académicos gestionados dentro del sistema

El sistema gira alrededor de dos documentos institucionales que cada profesor elabora por cada UEA que imparte a lo largo de un trimestre: la *carta temática* y los *requisitos de recuperación*. Ambos se depositan en un espacio público consultable por alumnos y personal académico y comparten un conjunto de metadatos comunes: profesor autor, UEA, periodo trimestral, nombre e identificador de grupo, horario, modalidad (presencial, remoto o mixto) y estado del documento (*borrador* mientras se redacta, *publicado* una vez difundido).

Carta temática

Una *carta temática* es el programa detallado con el que un profesor comunica cómo va a impartir una UEA durante un trimestre. No sustituye al programa oficial de la licenciatura o del posgrado — que es fijo y aprobado por la División —, sino que lo particulariza: bajo qué enfoque se desarrollará el curso, con qué calendario, con qué materiales y bajo qué criterios de evaluación.

En el sistema, cada carta temática se compone de los siguientes campos de contenido, redactados como texto libre por el profesor:

Descripción de la UEA Presentación general de la asignatura y su ubicación dentro del plan de estudios.

Objetivo general Propósito global que se espera que el estudiante alcance al concluir el trimestre.

Objetivos particulares Metas específicas que, en conjunto, permiten alcanzar el objetivo general.

Contenido sintético Enumeración de los temas y subtemas que se abordarán.

Objetivos de aprendizaje Resultados observables (conocimientos, habilidades, actitudes) que el estudiante deberá demostrar.

Requerimientos Materiales, herramientas, equipo o *software* que el estudiante necesita para cursar la UEA.

Conocimientos previos Contenidos que el estudiante debe dominar antes de iniciar el curso.

Modalidad de evaluación Instrumentos y ponderaciones con los que se calificará al estudiante.

Revisiones y asesorías Momentos y modos en que el profesor atiende dudas o revisa avances.

Bibliografía Fuentes básicas y complementarias recomendadas.

Enlace Liga(s) al aula virtual, videoconferencia o repositorio, cuando la modalidad lo requiere.

Calendarización de actividades Secuencia semana a semana de temas, entregas y evaluaciones.

Requisitos de recuperación

Los *requisitos de recuperación* constituyen el documento con el que el profesor comunica en qué consiste la *evaluación global* que la normatividad de la UAM ofrece a los estudiantes que no aprueban la UEA por la vía ordinaria. Su publicación oportuna es indispensable porque el estudiante debe conocer, con antelación, cuáles son las condiciones exactas para presentarla: dónde, cuándo, con qué materiales y bajo qué reglas.

En el sistema, cada requisito de recuperación se compone de los siguientes campos:

Lugar Espacio físico o virtual donde se presentará la evaluación. Si es remoto, puede incluir liga y contraseña.

Duración aproximada Tiempo estimado que tomará la evaluación.

Fecha y hora Momento en que se aplicará la evaluación, como texto libre (por ejemplo, "Jueves 03 de septiembre, 10:00 h").

Recursos necesarios Materiales, herramientas o equipo que el estudiante debe traer o tener disponibles.

Requisitos Condiciones que el estudiante debe cumplir para poder presentar la evaluación (por ejemplo, entrega previa de investigación, maqueta o porcentaje mínimo de tareas del trimestre).

Notas Aclaraciones adicionales que el profesor considere pertinentes.

Los campos son de texto libre y no obligatorios individualmente, para acomodar la heterogeneidad de las UEA de la División (talleres, laboratorios, seminarios teóricos, cursos híbridos), pero se validan como conjunto al momento de publicar. Ambos documentos son únicos por la combinación (*profesor, periodo, UEA, grupo*): un mismo profesor no puede tener dos cartas temáticas ni dos requisitos vigentes para el mismo grupo en un mismo trimestre.

5.2. Aplicación web y arquitectura cliente–servidor

Una *aplicación web* es un programa cuya interfaz se ejecuta dentro de un navegador y cuya lógica principal reside en un servidor remoto. A diferencia de una aplicación de escritorio, no requiere instalación: basta con una URL. La arquitectura habitual es *cliente–servidor*, en la que dos programas independientes se comunican a través de una red: el *cliente* solicita datos u operaciones y el *servidor* las procesa y responde.

5.3. Patrón Modelo–Vista–Controlador (MVC)

El *Modelo–Vista–Controlador* es un patrón arquitectónico introducido por Krasner y Pope para Smalltalk-80 [8]. Divide una aplicación con interfaz gráfica en tres responsabilidades:

- **Modelo:** representa los datos y las reglas de negocio. Es independiente de cómo se muestran o se manipulan.
- **Vista:** presenta el estado del modelo al usuario. Puede haber varias vistas de un mismo modelo.
- **Controlador:** recibe las entradas del usuario, decide qué operación realizar sobre el modelo y actualiza la vista.

5.4. REST y APIs sobre HTTP

REST (Representational State Transfer) es un estilo arquitectónico definido por Roy Fielding en su tesis doctoral [9]. Establece un conjunto de restricciones para diseñar servicios distribuidos escalables sobre HTTP:

- Cada recurso se identifica por una URI única (por ejemplo, `/api/v1/cartas-tematicas/42/`).
- Las operaciones se expresan con los verbos HTTP estándar: GET (leer), POST (crear), PUT/PATCH (actualizar), DELETE (eliminar). Al conjunto de estas cuatro operaciones se le conoce como *CRUD*.
- Las respuestas usan una representación acordada (habitualmente JSON) y son *sin estado*: cada petición contiene toda la información necesaria para procesarla.

5.5. HTTP, endpoints, códigos de estado y CORS

HTTP (HyperText Transfer Protocol) es el protocolo de aplicación de la web sobre el que se transporta la API [10]. Su variante segura, *HTTPS*, cifra el canal con TLS y se recomienda en producción. Cada intercambio HTTP consta de una petición (verbo, URI, cabeceras, cuerpo opcional) y una respuesta (código de estado, cabeceras, cuerpo opcional).

Se llama *endpoint* a cada URI concreta que el servidor expone para un recurso u operación — por ejemplo, `POST /api/v1/auth/login/`. En este reporte los términos “endpoint” y “ruta de la API” se usan como sinónimos, siempre entendidos como el par (verbo, URI) que el cliente invoca.

Los *códigos de estado* más relevantes para el sistema son: 200 OK (éxito con contenido), 201 Created (recurso creado), 204 No Content (éxito sin cuerpo), 400 Bad Request (*payload* inválido o violación de unicidad), 401 Unauthorized (token ausente o expirado), 403 Forbidden (autenticado pero sin permiso), 404 Not Found y 429 Too Many Requests (*rate limit* excedido; devuelto por el *throttling* del *endpoint* de *login*).

CORS (Cross-Origin Resource Sharing) es el mecanismo con el que un navegador permite que un documento cargado desde un origen (esquema + dominio + puerto) haga peticiones a otro origen distinto [11]. Por defecto, la *política del mismo origen* lo prohíbe; el servidor debe declarar explícitamente qué orígenes acepta mediante cabeceras Access-Control-Allow-Origin y responder a las peticiones de *preflight* (OPTIONS). La configuración adoptada en el proyecto (lista blanca de orígenes con *django-cors-headers*) se describe en la Sección 6.12.

5.6. Python y Django

Python es un lenguaje de programación de alto nivel, interpretado y de tipado dinámico, ampliamente utilizado en desarrollo web [12].

Django [13] es un *framework* web escrito en Python que sigue el principio “*baterías incluidas*”: provee, listos para usar, los componentes que la mayoría de las aplicaciones web necesitan. Los más relevantes para este proyecto son:

- **ORM (Object-Relational Mapper):** permite definir el esquema de la base de datos como clases de Python (los modelos) y consultarla con métodos en lugar de SQL. Reduce errores de sintaxis, previene inyección SQL y hace portable el código entre motores de base de datos.
- **Sistema de migraciones:** cada cambio al modelo se traduce automáticamente en un archivo de migración versionable en git. Permite evolucionar la base de datos de forma reproducible entre entornos.
- **Sistema de autenticación:** usuarios, grupos, permisos y *hashing* de contraseñas ya vienen implementados; el proyecto extiende el modelo de usuario para agregar rol.
- **Panel de administración:** interfaz autogenerada para operar la base de datos, útil como respaldo del administrador.
- **Sistema de validación y formularios:** encapsula las reglas de negocio en el servidor.

5.7. Django REST Framework (DRF)

Django REST Framework [14] es una capa sobre Django que facilita construir APIs REST. Sus componentes principales son:

- **Serializers:** traducen entre objetos de Python (instancias de modelos) y representaciones serializables (JSON). Validan la entrada y controlan qué campos se exponen hacia afuera.

- **ViewSets y routers:** agrupan las operaciones CRUD sobre un recurso y las publican bajo URLs consistentes con las convenciones REST.
- **Permisos (*permission classes*):** clases reutilizables que declaran quién puede acceder a cada acción (por ejemplo, “solo el dueño del recurso puede modificarlo”). Se aplican a nivel de *ViewSet* y, mediante `has_object_permission`, también a nivel de instancia.
- **Throttling y rate limiting:** mecanismos equivalentes (el segundo es el término genérico) que limitan el número de peticiones que un cliente puede hacer en un lapso; útiles para mitigar ataques de fuerza bruta.
- **drf-spectacular:** complemento que analiza el código y genera automáticamente la especificación *OpenAPI* de la API, junto con un explorador *Swagger UI* interactivo.

En el proyecto DRF permite mantener la definición del contrato de la API cercana al modelo, con *serializers* explícitos que documentan qué campos son públicos y qué reglas de validación aplican.

5.8. OpenAPI y Swagger UI

OpenAPI es una especificación abierta para describir contratos de APIs REST en un formato máquina-legible (JSON o YAML) [15]. Un documento *OpenAPI* enumera los *endpoints*, los verbos que aceptan, los esquemas de los cuerpos de petición y respuesta, los códigos de estado posibles y los mecanismos de autenticación. Sirve como documentación viva, como insumo para generadores de clientes en distintos lenguajes y como fuente para pruebas de contrato.

Swagger UI es una interfaz web que consume un documento *OpenAPI* y produce un explorador navegable en el que se pueden probar las peticiones interactivamente. En el proyecto, *drf-spectacular* genera la especificación desde las anotaciones del código de DRF y la expone en dos rutas: `/api/schema/` (documento *OpenAPI* en YAML descargable) y `/api/docs/` (*Swagger UI*). Con esto se evita la desincronización clásica entre la documentación redactada a mano y el comportamiento real del servidor.

5.9. PostgreSQL

PostgreSQL [16] es un sistema de gestión de bases de datos relacional (*RDBMS*), libre y de código abierto, con más de 30 años de desarrollo. Se seleccionó principalmente por su compatibilidad con el ORM de Django.

5.10. Constraints declarativos: unicidad compuesta, XOR y unicidad parcial

Una *restricción declarativa* (*constraint*) se define a nivel del esquema de la base de datos y es el motor quien la hace cumplir. Complementa la validación aplicativa (en *serializers* y modelos) como *defensa en profundidad*: aunque una capa falle, la otra sigue previniendo estados inconsistentes. El proyecto usa tres variantes:

- **Unicidad compuesta:** `UniqueConstraint (fields=["profesor", "periodo", "uea", "id_grupo"])` garantiza que un profesor no tenga dos cartas temáticas para el mismo grupo en el mismo trimestre para la misma UEA. La validación se realiza en el motor con un índice único; el intento de duplicación devuelve un error de integridad que DRF traduce a 400 Bad Request.
- **Restricción XOR:** expresa “exactamente uno de un conjunto de campos debe estar presente”. En el proyecto se aplica a UEA, que debe apuntar a una licenciatura o a un posgrado, nunca a ambos ni a ninguno. Se implementa como `CheckConstraint` con predicados `Q(licenciatura__isnull=False) & Q(posgrado__isnull=True)` en disyunción con su opuesto.
- **Unicidad parcial:** `UniqueConstraint (condition=...)` restringe la unicidad a un subconjunto de filas. Se usa para forzar clave única por programa: la misma clave textual (por ejemplo “1400027”) puede coexistir en una licenciatura y en un posgrado, pero no repetirse dentro del mismo programa.

5.11. React

React [17] es una biblioteca de JavaScript, desarrollada originalmente por Facebook (hoy Meta), para construir interfaces de usuario a partir de *componentes* reutilizables.

Los conceptos centrales de React que se aprovechan son:

- **Componente:** función que recibe *props* (parámetros) y devuelve una descripción declarativa de la interfaz. Los componentes se componen entre sí formando árboles.
- **Estado (*state*):** datos internos de un componente que, al cambiar, disparan un nuevo renderizado.
- **Hooks:** funciones especiales (`useState`, `useEffect`, `useContext`, etc.) que permiten manejar estado y efectos secundarios en componentes funcionales.
- **Renderizado declarativo:** el programador describe cómo se ve la interfaz en función del estado; React se encarga de calcular y aplicar los cambios mínimos al DOM.

5.12. Vite

Vite [18] es una herramienta de *build* y servidor de desarrollo para aplicaciones frontend modernas que aprovecha los módulos nativos de JavaScript (*ES modules*) del navegador para arrancar el servidor en milisegundos, incluso en proyectos grandes.

Sus aportes en este proyecto son:

- **Servidor de desarrollo con *Hot Module Replacement* (HMR):** los cambios en el código se reflejan en el navegador sin recargar la página completa, preservando el estado.
- **Build optimizado para producción:** compila, minifica y divide el código en *chunks* para producir un paquete estático servible por cualquier servidor HTTP.
- **Configuración mínima y ecosistema de *plugins*.**

5.13. Tailwind CSS

Tailwind CSS [19] es un *framework* de estilos basado en *utility classes*: en lugar de escribir hojas de estilo (CSS) tradicionales, la apariencia se compone directamente en el marcado mediante clases atómicas (por ejemplo, `p-4`, `text-lg`, `bg-indigo-600`, `rounded-lg`).

Ventajas aprovechadas:

- Elimina la necesidad de nombrar clases semánticas cuando no aportan valor.
- El paso de *build* descarta las clases no utilizadas, produciendo hojas de estilo compactas.
- Aporta un sistema de diseño (espaciados, tipografía, paleta) coherente por defecto.
- Fomenta una interfaz responsiva mediante *breakpoint prefixes* (`sm:`, `md:`, `lg:`) y admite el prefijo `print:` para especializar la apariencia al momento de imprimir.

5.14. Bibliotecas complementarias del frontend

Alrededor de React se utilizan bibliotecas específicas para tareas concretas:

TanStack Query Gestor de estado de servidor [20]. Se encarga de solicitar, cachear, revalidar e invalidar los datos que llegan del backend. Aporta reintento automático ante fallos transitorios, *polling* declarativo y sincronización entre pestañas.

React Hook Form Biblioteca de manejo de formularios [21] que evita renderizados innecesarios y ofrece una API basada en *hooks*, ligera comparada con alternativas como *Formik*.

Zod Biblioteca de validación de esquemas con inferencia de tipos [22]. Se combina con React Hook Form para validar en el cliente antes de enviar al servidor.

React Router Manejo del enrutamiento del lado del cliente [23]. Permite construir SPAs con navegación sin recarga y *guards* de ruta según el rol.

axios Cliente HTTP [24] con soporte de *interceptors*, utilizado para adjuntar automáticamente el *access token* JWT y refrescarlo cuando expira.

SheetJS (xlsx) Genera archivos *.xlsx* en el navegador para la descarga de reportes [25].

5.15. Patrones frontend: SPA con *guards*, *interceptores* y estado de servidor

El frontend se apoya en tres patrones que estructuran el flujo de datos y de navegación:

Route guard (guardián de ruta) Componente contenedor que, antes de renderizar la ruta que envuelve, verifica una condición (autenticación, rol) y decide entre renderizar los hijos o emitir una redirección declarativa (el elemento `Navigate` de *React Router*). La redirección declarativa se prefiere sobre `navigate()` imperativo dentro del *render* para evitar efectos secundarios en el ciclo de vida de React. Su aplicación en el proyecto se describe en la Sección 6.

Interceptor (*axios*) Función que intercepta las peticiones o respuestas HTTP antes de que las reciba el código que las originó. Permite, por ejemplo, adjuntar automáticamente la cabecera `Authorization: Bearer` con el *access token* o reaccionar de forma centralizada a un `401 Unauthorized` renovando el token y reintentando la petición.

Estado de servidor vs. estado local La distinción, popularizada por TanStack Query, separa los datos que *pertenecen* al servidor (y que el cliente cachea) del estado propio de la interfaz (formularios en edición, apertura de *modals*). El servidor es la fuente de verdad; el cliente sincroniza con estrategias como *stale-while-revalidate* (mostrar lo cacheado y refrescar en paralelo) y *polling* (revalidar cada N segundos, útil para vistas colaborativas).

5.16. Componentes de interfaz: *form-builder*, cascada, acordeón, *dashboard* y *badge*

El sistema utiliza patrones de interacción específicos, mencionados a lo largo del reporte:

Form-builder Constructor visual de formularios. Permite al administrador definir la estructura (secciones, preguntas de distintos tipos, opciones, puntuaciones) desde una interfaz gráfica en lugar de programar. Se contrapone a los formularios estáticos codificados en el frontend. En este proyecto es la vía por la que la Coordinación diseña las autoevaluaciones docentes.

Cascada (*drill-down*) Selección progresiva en la que cada elección restringe el conjunto de opciones del siguiente paso. En la vista pública, el visitante elige primero un periodo, luego un programa (licenciatura o posgrado) y por último una UEA, filtrando los documentos a mostrar.

Acordeón (*collapsible sections*) Componente que agrupa contenido en paneles expandibles y colapsables. Se usa para navegar por UEAs agrupadas por trimestre en la vista pública y para las listas de documentos del profesor agrupadas por periodo.

Dashboard Tablero con indicadores clave de desempeño (*KPIs*) y resúmenes agregados, pensado para lectura rápida. El sistema entrega dos dashboards: uno para el profesor (sus documentos por periodo activo) y uno para el administrador (métricas globales de cumplimiento).

Badge y chip Etiquetas visuales pequeñas que comunican el estado de un objeto (por ejemplo, "BORRADOR", "PUBLICADO", "ACTIVO"). En este proyecto los *chips* clicables del listado de periodos también actúan como controles para activar o desactivar cada uno de los tres *flags* de recurso.

5.17. Autenticación con JSON Web Tokens (JWT)

Un *JSON Web Token* (JWT) [26] es un formato compacto y firmado criptográficamente para transmitir información entre partes. Se define en el estándar *RFC 7519* y consta

de tres partes codificadas en Base64URL y separadas por puntos: encabezado, *payload* (información del usuario y expiración) y firma (para verificar integridad).

En el proyecto la autenticación con JWT se implementa mediante *django-rest-framework-simplejwt* y sigue este flujo:

1. El usuario envía credenciales al endpoint `/api/v1/auth/login/`.
2. El servidor valida las credenciales y responde con dos tokens: uno de *acceso* (corta vida, típicamente 60 minutos) y uno de *refresco* (vida más larga, siete días).
3. El frontend envía el *access token* en el encabezado `Authorization: Bearer <token>` de cada petición posterior.
4. Cuando el *access token* expira, un *interceptor* del cliente HTTP solicita uno nuevo al endpoint `/api/v1/auth/refresh/` usando el *refresh token*.
5. Los *refresh tokens* rotan: cada renovación emite un nuevo *refresh token* y coloca el anterior en una lista negra (*blacklist*), de modo que no pueda reutilizarse [27].

Se prefirió JWT sobre el esquema tradicional de sesiones con *cookies* porque encaja mejor con APIs REST sin estado consumidas por un frontend desacoplado.

5.18. Control de acceso basado en roles (RBAC)

Role-Based Access Control (RBAC) es un modelo de autorización en el que los permisos no se asignan a usuarios individualmente sino a *roles*, y cada usuario recibe uno o varios roles [28]. El proyecto define dos roles autenticados (la consulta pública de documentos no requiere sesión ni rol):

- **ADMIN:** gestiona catálogos, profesores, formularios de autoevaluación y consulta reportes globales.
- **PROFESOR:** gestiona sus propias cartas temáticas, requisitos y autoevaluaciones.

Los permisos se aplican en dos capas:

- En el **backend** mediante clases *DRF permission* (por ejemplo, `IsOwnerProfesorOrAdminReadOnly`) que verifican rol y propiedad del recurso.
- En el **frontend** mediante *guards* de ruta que redirigen si el rol no coincide, y renderizado condicional que oculta acciones no autorizadas.

La verificación en ambos lados es *defensa en profundidad*: el backend es la fuente de verdad; el frontend evita mostrar interfaces inútiles. El principio de defensa en profundidad, tomado de la ingeniería de seguridad, consiste en superponer varias capas independientes de protección para que la falla de una no comprometa al sistema.

5.19. Seguridad web y OWASP Top 10

El OWASP Top 10 [29] es un documento consensuado por la *Open Web Application Security Project* que enumera las diez categorías de riesgo de seguridad más críticas en aplicaciones web. Sirve como *checklist* de referencia para desarrolladores. Las categorías atendidas explícitamente en este proyecto son:

- **A01 — Broken Access Control:** se mitiga con las verificaciones de rol y propiedad en el backend.
- **A02 — Cryptographic Failures:** contraseñas almacenadas como *hash* PBKDF2 (algoritmo por defecto de Django) y tokens firmados con HMAC-SHA256. El *hashing* es una función unidireccional que produce una huella de longitud fija a partir de la contraseña; PBKDF2 [30] la refuerza aplicando miles de iteraciones y un *salt* aleatorio, encareciendo los ataques de diccionario.
- **A03 — Injection:** prevenida por el ORM (consultas parametrizadas) y por la validación de entrada en *serializers*.
- **A05 — Security Misconfiguration:** en producción se establece `DEBUG=False` (la variable de Django que activa las trazas detalladas de error, útiles en desarrollo pero peligrosas en producción) y una lista blanca explícita de *hosts* y de orígenes CORS.
- **A07 — Identification and Authentication Failures:** se aplica *throttling* en el *login* (5 intentos por minuto, respuesta 429 Too Many Requests al superarse), rotación de *refresh tokens* y política de contraseñas fuerte.

5.20. Formatos de intercambio de datos

El sistema produce, consume o exporta datos en varios formatos, cada uno elegido por adecuarse a un caso de uso concreto:

JSON (*JavaScript Object Notation*) — formato textual ligero de pares clave–valor y arreglos anidados. Es el formato por defecto de la API REST (peticiones y respuestas) y de los *fixtures* de Django que precargan los catálogos institucionales.

JSONB — variante binaria e indexable de JSON que ofrece PostgreSQL; se usa dentro del modelo *Pregunta* para el campo `config` (por ejemplo, los límites de una escala lineal), evitando crear una tabla auxiliar por cada tipo de pregunta.

CSV (*Comma-Separated Values*) — formato tabular plano que separa campos por comas y filas por saltos de línea. Es el formato de intercambio con hojas de cálculo. El sistema lo usa para la importación masiva del catálogo de UEAs (`POST /uea/import-csv/` y el comando de gestión equivalente).

XLSX — formato binario de libros de Excel. Se genera del lado del cliente con *SheetJS*: el navegador arma un *workbook* con una hoja por periodo y lo descarga sin intervención del servidor. Esta elección reduce carga del backend y permite iterar rápidamente sobre el formato sin desplegar cambios.

YAML (*YAML Ain't Markup Language*) — formato textual jerárquico legible. En este proyecto solo aparece como formato de salida del esquema *OpenAPI* en `/api/schema/`.

PDF (*Portable Document Format*) — se obtiene desde el propio navegador vía `window.print()` sobre las vistas públicas del documento, que aplican reglas CSS con el prefijo `print:` de Tailwind para producir una salida limpia.

5.21. Máquina de estados y ciclo de vida de artefactos

Una *máquina de estados finitos* modela un artefacto como un conjunto de estados discretos y un conjunto de transiciones válidas entre ellos, disparadas por acciones. Aplicar este patrón hace explícitas las reglas de negocio (qué acciones son válidas en qué momento) y evita estados inconsistentes.

El sistema usa dos máquinas de estados:

- Los **documentos académicos** (cartas y requisitos) tienen dos estados: BORRADOR (editable por el dueño) y PUBLICADO (visible al público, solo despublicable por el dueño para volver a editar). La transición se dispara con la acción `cambiar-estado`.
- Los **formularios de autoevaluación** tienen tres estados: BORRADOR, PUBLICADO (aceptando respuestas) y CERRADO (histórico, sin recibir nuevas respuestas). Las acciones válidas son `publicar`, `cerrar`, `reabrir`, `despublicar` y `publicar_revision`. Esta última merece atención aparte por su efecto sobre el versionado (§ 5.22).

Modelar el ciclo como máquina de estados aporta idempotencia (aplicar `publicar` sobre un formulario ya publicado no cambia nada), inmutabilidad tras publicar (no se pueden alterar preguntas o pesos de un formulario publicado sin transitar por `despublicar` o `publicar_revision`) y trazabilidad de las transiciones.

5.22. Versionado inmutable y *snapshots* históricos

Dos técnicas complementarias garantizan que la información histórica sobreviva a los cambios administrativos posteriores:

Snapshot de identidad Al crear un documento, se copian el nombre y el correo del profesor en columnas dedicadas (`profesor_nombre`, `profesor_correo`). Si más adelante el usuario se elimina, la FK con `on_delete=SET_NULL` deja el documento sin dueño vivo, pero los *snapshots* preservan a quién pertenecía. Es una técnica de *denormalización controlada* que sacrifica una pequeña redundancia por trazabilidad institucional.

Versionado incremental de formulario Cada `Formulario` tiene un entero que indica la `version` del mismo (inicialmente 1). La acción `publicar_revision` incrementa esta versión al reabrir el formulario tras su cierre (y `publicar` también lo hace cuando el formulario se edita después de haber recibido respuestas); las respuestas ya enviadas quedan atadas a la versión con la que se contestaron mediante el campo

`version_formulario` de `Respuesta`. Además, cada `RespuestaSeccion` copia el peso de la sección al momento del envío, de modo que el desglose por sección permanece estable aunque el administrador ajuste pesos posteriormente. Este esquema hace posibles comparaciones históricas consistentes sin bloquear la evolución del instrumento.

5.23. Escalas de medición: Likert y niveles de desempeño

Una *escala de Likert* [31] es una escala psicométrica en la que el respondiente indica su grado de acuerdo con una afirmación mediante una opción entre varias equiespaciadas (típicamente 1 a 5: “Muy en desacuerdo” ... “Muy de acuerdo” o “Nunca” ... “Siempre”). En el sistema esta escala aparece como el tipo de pregunta `ESCALA_LINEAL` y también, en modo compacto de matriz filas×columnas, como el tipo `CUADRICULA`.

Un *nivel de desempeño* es un rango porcentual con etiqueta descriptiva, observación y color (por ejemplo, “0–59.99 En desarrollo”, “60–84.99 Satisfactorio”, “85–100 Sobresaliente”). Al enviar una autoevaluación, el sistema calcula el porcentaje global ponderado y busca el nivel cuyo rango lo contiene (si el valor cae en el límite de dos rangos, se asigna el de menor orden); adjunta el nivel y su observación al resultado. Este mecanismo permite convertir un puntaje continuo en una interpretación cualitativa comprensible para el profesor.

5.24. Desarrollo incremental y control de versiones

El *desarrollo incremental* es un enfoque en el que el sistema se construye en entregas sucesivas, cada una con una capacidad funcional verificable que se agrega a las anteriores, de modo que se puede revisar el avance y ajustar el rumbo antes de terminar [32]. Este enfoque encaja con un proyecto de una sola persona y requisitos que se refinan con la retroalimentación de la Coordinación.

El *control de versiones* con `git` registra cada cambio del código y permite recuperar cualquier estado previo. Un *monorepositorio* aloja en un único repositorio varios subproyectos relacionados (aquí, *backend* y *frontend*). La convención *Conventional Commits* [33] estandariza los mensajes de *commit* con prefijos (*feat*, *fix*, *docs*, etc.), lo que facilita leer el historial y automatizar futuros *changelogs*. La organización concreta del trabajo por hitos y del repositorio se describe en la Sección 6.2.

5.25. Pruebas automatizadas

Las pruebas automatizadas del proyecto se ejecutan con *pytest* [34] y *pytest-django*. Los conceptos utilizados son:

- **Prueba unitaria:** verifica una función o unidad aislada.
- **Prueba de integración:** verifica la interacción de varios componentes (por ejemplo, una petición HTTP que atraviesa *view*, *serializer*, ORM y base de datos).
- **Prueba end-to-end (E2E):** verifica el comportamiento del sistema completo desde el punto de vista del usuario, usualmente atravesando frontend y backend.

- **Factory:** patrón para construir instancias de prueba consistentes; se emplea la biblioteca *factory-boy*.
- **Cobertura de código:** porcentaje del código ejecutado por las pruebas. Se mide con *coverage.py* a través del complemento *pytest-cov*.
- **Fixture (dos acepciones):** en el contexto de *pytest*, es una función que provee datos preparados a una prueba; en el contexto de Django, es un archivo (JSON o YAML) con datos precargables al inicializar el sistema. Ambas acepciones se usan en el proyecto y se distinguen en el texto por el contexto.

6. Desarrollo del proyecto

Esta sección narra el desarrollo del sistema *paso a paso*: de la arquitectura general al flujo operativo de cada módulo, cubriendo la construcción del modelo de datos, la gestión de profesores y catálogos, la habilitación de periodos por recurso, la elaboración y publicación de documentos, la construcción de formularios de autoevaluación con su sistema de puntuación, la descarga de reportes y la vista pública de exploración. El código se organiza como un único monorepositorio con dos subproyectos: el `backend/` (Django) y el `frontend/` (React + Vite), versionados con *git* bajo el directorio raíz `proyecto_terminal_cyad/`. Los fragmentos de código que ilustran cada mecanismo se recopilan en el Apéndice 11.1 y se citan por su *label* desde esta sección.

6.1. Arquitectura general

El sistema sigue una arquitectura cliente–servidor de tres capas:

1. **Capa de presentación (frontend):** aplicación SPA en React 19 servida por Vite 8; renderizada en el navegador; consume la *API REST* mediante *axios*; almacena el *JWT* en `localStorage` y gestiona el ciclo de vida de las sesiones (*refresh* automático transparente, expulsión al expirar el *refresh token*).
2. **Capa de aplicación (backend):** servidor Django 5 que expone la *API REST* bajo el prefijo `/api/v1/`; orquesta la lógica de negocio en *ViewSet* de DRF; valida y serializa entradas con *Serializers*; documenta la API con *drf-spectacular* (`/api/docs/` y `/api/schema/`).
3. **Capa de persistencia:** base de datos PostgreSQL 17 accedida vía el ORM de Django, con migraciones versionadas por app y *constraints* declarativos que actúan como defensa en profundidad.

La Figura 1 muestra el diagrama de arquitectura y la Figura 2 el diagrama de casos de uso, con los actores *Administrador*, *Profesor* y *Público* y sus interacciones con los módulos funcionales.

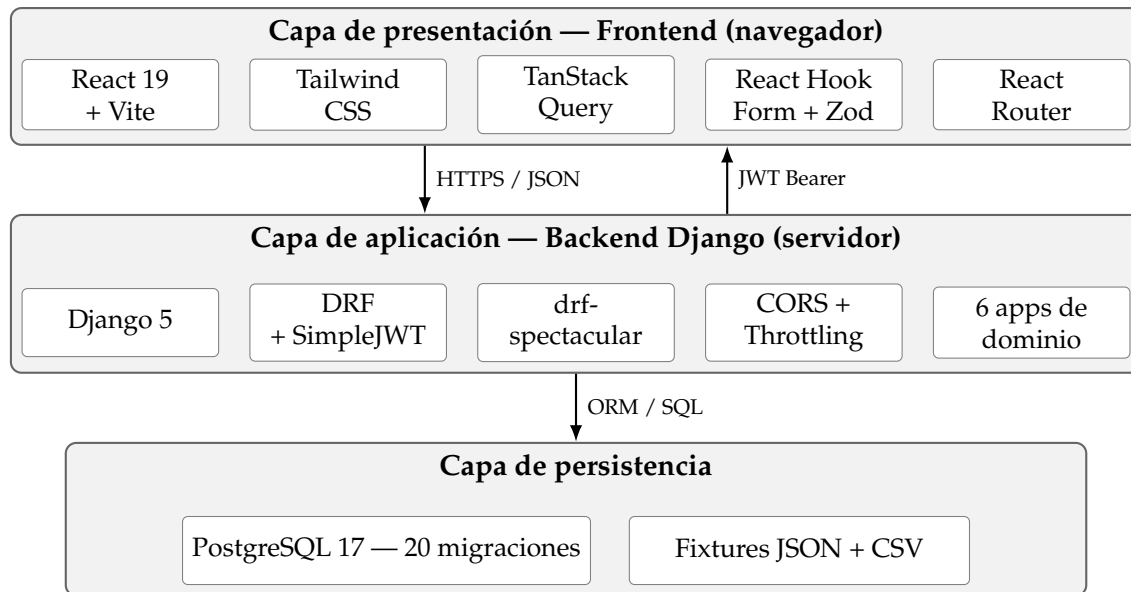


Figura 1. Arquitectura de tres capas del sistema.

6.1.1. Estructura de directorios

- **backend/:** proyecto Django. Contiene la configuración global (`config/`), las seis apps (`core`, `accounts`, `catalogos`, `documentos`, `autoevaluacion`, `reportes`), *scripts* operativos (`scripts/`), suite de pruebas (`tests/`), `confest.py` con *fixtures* globales de *pytest* y `requirements.txt`.
- **frontend/:** aplicación React + Vite. Contiene `src/pages/` (por rol: `admin/`, `profesor/`, `publico/`), `src/components/ui/` (componentes reutilizables), `src/api/` (clientes HTTP), `src/auth/` (contexto de sesión y *guards*), `src/layouts/` y `src/utils/`.
- **docs/:** documentación complementaria; contiene este reporte (`docs/reporte/`) con los manuales de instalación y de usuario integrados como subsecciones nativas del Apéndice 11.

6.2. Metodología y evolución del desarrollo

El desarrollo se organizó por *hitos* incrementales (M0–M9), cada uno con un objetivo verificable. M0 estableció el andamiaje del monorepo (Django + React inicializados, PostgreSQL conectado, JWT operativo). Los hitos M1 a M8 construyeron los módulos funcionales sucesivos y M9 corresponde al pulido, la integración con la documentación y la captura de evidencia visual. La organización por hitos sigue un modelo de desarrollo incremental [32]: cada hito entrega una capacidad funcional verificable sobre la anterior.

Dos episodios reorientaron el trabajo y quedan reflejados en el historial de *git*:

1. **Rediseño de junio de 2026 (documentos aplanados):** la propuesta original modelaba la carta temática como un árbol de subtablas (Tema, Subtema, Bibliografía,

`CriterioEvaluacion`). Tras validar con la Coordinación que la mayoría de los profesores redacta cada rubro como texto continuo, se decidió aplanar el modelo a doce `TextField` libres. La transición se materializó en la migración `documentos/0002_rediseño_documentos`, que redujo la complejidad del *form-builder* de cartas y aceleró el desarrollo del resto del módulo.

2. **Nueve incrementos “PR1–PR9” en autoevaluación:** el módulo más complejo se entregó en nueve *pull requests* sucesivos etiquetados en los mensajes de *commit*: PR1 (*form-builder* básico y tipos simples), PR2 (opciones puntuables), PR3 (tipo `ESCALA_LINEAL`), PR4 (tipo `CUADRICULA`), PR5 (secciones con peso y ponderación 100 %), PR6 (`NivelDesempeno`), PR7 (`versionado` y `publicar_revision`), PR8 (endpoints de estadísticas) y PR9 (duplicación entre periodos y pulido final).

La configuración por entorno se distribuye en un archivo `.env` local, leído por *python-decouple*; contiene `SECRET_KEY`, credenciales de PostgreSQL, `DEBUG` y `CORS_ALLOWED_ORIGINS`. El archivo nunca se versiona; en su lugar, un `.env.example` documenta las variables necesarias. Los mensajes de *commit* siguen la convención *Conventional Commits*, y el módulo de autoevaluación se dividió en los incrementos PR1–PR9 descritos arriba.

6.3. Modelo de datos

El modelo de datos se distribuye por app siguiendo la separación de dominios. Un diagrama entidad-relación completo se muestra en la Figura 3. En total, el sistema cuenta con 20 migraciones propias distribuidas entre las apps (`accounts`: 3, `autoevaluacion`: 5, `catalogos`: 7 y `documentos`: 5; `core` y `reportes` no definen tablas y por ello no tienen migraciones), además de las que aportan bibliotecas de terceros (por ejemplo, la lista negra de *simplejwt*).

A continuación se describe cada entidad por app: campos que la conforman, restricciones declarativas y cardinalidad de sus relaciones. La notación empleada es la de *crow's foot* en formato “1–N” (uno a muchos), “1–1” (uno a uno), “0..1–N” (opcional a muchos) y “N–M” (muchos a muchos).

App `core` — infraestructura compartida

No define tablas físicas. Aporta dos *mixins* abstractos que la mayoría de los modelos concretos heredan:

- `TimeStampedModel`: agrega `created_at` (`auto_now_add`) y `updated_at` (`auto_now`).
- `EstadoActivoModel`: agrega el campo booleano `estado` para borrado lógico (*soft delete*).

Además provee la paginación por defecto (`DefaultPagination`), el *exception handler* unificado (`api_exception_handler`) y las clases de permisos DRF (`IsAdmin`, `IsProfesor`, `IsAdminOrReadOnly`, `IsOwnerProfesorOrAdminReadOnly`) usadas transversalmente.

App accounts — usuarios y perfiles de profesor

Fragmentos del código de estos modelos se recogen en el Apéndice 11.1 (“Backend — Perfil de Profesor y Periodo”).

Usuario Extiende `AbstractBaseUser + PermissionsMixin`. Campos: `email` (único, `USERNAME_FIELD`), `nombre`, `rol` $\in \{ADMIN, PROFESOR\}$, `is_active`, `is_staff`, `password` (hash PBKDF2 vía `set_password`), `created_at`, `updated_at`. Se acompaña de `UsuarioManager` con `create_user` y `create_superuser`.

Profesor Perfil extendido del usuario con `rol=PROFESOR`. Hereda de `TimeStampedModel` y `EstadoActivoModel`. Campos propios: `numero_economico` (único, opcional), `nombre_completo`, `correo_institucional` (único) y FK a `Departamento`. Un `save()` sobrescrito sincroniza `Usuario.is_active` con `Profesor.estado`, de modo que desactivar el perfil bloquea inmediatamente el inicio de sesión.

Relaciones:

- Usuario 1–1 Profesor (OneToOne con `on_delete=CASCADE`; borrar el Usuario borra el Profesor).
- Departamento 1–N Profesor (FK con `on_delete=SET_NULL`, opcional).

App catalogos — catálogos institucionales de CyAD

Departamento Departamento académico. Campos: `clave` (única), `nombre`, más los heredados de `TimeStampedModel` y `EstadoActivoModel`.

Licenciatura Programa de licenciatura. Campos: `clave` (única), `nombre`, `orden` (para ordenamiento en UI) y FK a `Departamento` (`SET_NULL`).

Posgrado Programa de posgrado (maestría, especialidad o doctorado). Mismos campos que `Licenciatura`.

Area Área curricular a la que pertenece una UEA (por ejemplo, “Licenciatura” u “Optativas de interés profesional”, esta última subdividida por su descripción en áreas de concentración). Campos: `nombre`, `descripcion`, con `unique_together=(nombre, descripcion)`.

UEA Unidad de Enseñanza-Aprendizaje. Campos: `clave` (normalizada a mayúsculas en `save()`), `nombre`, `trimestre` (entero 1–12 o cadena romana “VII–XII” para optativas), `tipo` $\in \{OBL, OPT, OTRO\}$, `creditos`, `liga` (URL oficial de la UEA). FKs con `on_delete=PROTECT`: a `Licenciatura`, `Posgrado` y `Area`, todas opcionales. Tres *constraints* declarativos: `CheckConstraint` `uea_xor_licenciatura_posgrado` (exactamente uno de los dos programas está poblado); y dos `UniqueConstraint` parciales sobre (`clave`, `licenciatura`) y (`clave`, `posgrado`).

Periodo Periodo trimestral (por ejemplo, “26-P”). Campos: `clave` (única), `fecha_inicio`, `fecha_fin`, tres booleanos independientes de habilitación por recurso (`activo_cartas`, `activo_requisitos`, `activo_autoevaluacion`), un booleano derivado `activo` (OR de los tres, sincronizado en `save()`) y estado (habilitación general). El `save()` sobrescrito garantiza la invariante “a lo sumo un periodo activo por recurso” apagando el *flag* correspondiente en los demás periodos durante la misma llamada a `save()`.

Relaciones:

- Departamento 1-N Licenciatura y Departamento 1-N Posgrado (SET_NULL, opcional).
- Licenciatura 1-N UEA y Posgrado 1-N UEA (PROTECT, mutuamente excluyentes por el XOR).
- Area 1-N UEA (PROTECT, opcional).
- Periodo es referenciado desde documentos y autoevaluacion (véase abajo).

App documentos — cartas temáticas y requisitos de recuperación

Toda la app comparte la clase abstracta `DocumentoAcademicoBase` con los campos comunes: FK opcional a Profesor (SET_NULL, para preservar el documento como historial si el profesor es eliminado), *snapshots* de identidad `profesor_nombre` y `profesor_correo` (rellenados automáticamente al crear el documento), FK a UEA y Periodo (ambas PROTECT), `nombre_grupo`, `id_grupo`, `horario`, `modalidad` ∈ {PRESENCIAL, REMOTO, MIXTO}, `estado` ∈ {BORRADOR, PUBLICADO} y las marcas de tiempo de `TimeStampedModel`. Los métodos `puede_editar()` y `puede_eliminar()` exigen `estado=BORRADOR`.

CartaTematica Doce campos `TextField` libres para el contenido pedagógico: `descripcion_uea`, `objetivo_general`, `objetivos_particulares`, `contenido_sintetico`, `objetivos_aprendizaje`, `requerimientos`, `conocimientos_previos`, `modalidad_evaluacion`, `revisiones_a_sesorias`, `bibliografia`, `enlace` y `calendarizacion_actividades`. `UniqueConstraint` `unique_carta_profesor_periodo_uea_grupo` sobre (`profesor`, `periodo`, `uea`, `id_grupo`).

RequisitoRecuperacion Seis campos textuales: `lugar`, `duracion_aprox`, `fecha_hora`, `recursos_necesarios`, `requisitos` y `notas`. Misma `UniqueConstraint` compuesta que `CartaTematica`.

Relaciones (idénticas para ambos documentos):

- Profesor 0..1-N CartaTematica / RequisitoRecuperacion (SET_NULL).
- UEA 1-N CartaTematica / RequisitoRecuperacion (PROTECT).
- Periodo 1-N CartaTematica / RequisitoRecuperacion (PROTECT).

App autoevaluacion — *form-builder*, respuestas y scoring

Es la app con el modelo más rico. Se agrupa en tres bloques: definición del formulario, definición del baremo y captura de respuestas. Fragmentos representativos de estos modelos se listan en el Apéndice 11.1 (“Backend — Modelos del *form-builder*” y “Backend — Captura de respuestas y baremo”).

Definición del formulario.

Formulario Encabezado del instrumento. Campos: titulo, descripcion, FK a Periodo (PROTECT), estado $\in \{BORRADOR, PUBLICADO, CERRADO\}$, version (entero incrementable), una_respuesta_por_profesor (booleano), FK opcional a Usuario en created_by (SET_NULL), y timestamps de ciclo de vida published_at y closed_at. Los métodos publicar, cerrar, reabrir, despublicar y publicar_revision materializan la máquina de estados descrita en la Sección 6.8.2.

Seccion Bloque temático. Campos: FK a Formulario (CASCADE), titulo, descripcion, orden, peso decimal (max_digits=5, decimal_places=2) cuya suma sobre todas las secciones debe ser 100 % ($\pm 0,01$) para poder publicar.

Pregunta Ítem individual. Campos: FK a Formulario (CASCADE) y FK opcional a Seccion (SET_NULL), tipo (ocho valores del TextChoices), texto, ayuda, obligatoria, orden y config (JSON dependiente del tipo: por ejemplo, {min, max, label_min, label_max, puntos_factor} para ESCALA_LINEAL o {puntos_si, puntos_no} para SI_NO). El conjunto TIPOS_NO_PUNTABLES = {TEXTO_CORTO, TEXTO_LARGO} marca las preguntas que se ignoran en el cálculo del puntaje.

OpcionPregunta Opción seleccionable. Campos: FK a Pregunta (CASCADE), texto, valor (identificador estable), puntos decimal y orden.

FilaCuadrícula Fila de una pregunta CUADRICULA. Campos: FK a Pregunta (CASCADE), texto y orden. Las columnas de la cuadrícula reutilizan OpcionPregunta.

Definición del baremo.

NivelDesempeno Rango porcentual con observación cualitativa. Campos: FK a Formulario (CASCADE), nombre, porcentaje_min y porcentaje_max decimales, observacion, color (clave para la UI: green|blue|yellow|red|gray) y orden.

Captura de respuestas.

Respuesta Envío por profesor. Campos: FK a Formulario (PROTECT), FK a Profesor (PROTECT), version_formulario (copia de Formulario.version al iniciar),

estado $\in \{BORRADOR, ENVIADO\}$, enviado_at y los agregados calculados al enviar: puntaje_obtenido, puntaje_maximo, porcentaje y FK a NivelDesempeno (SET_NULL). UniqueConstraint sobre (formulario, profesor, version_formulario) que permite responder de nuevo tras una publicar_revision.

RespuestaPregunta Ítem de respuesta. Campos: FK a Respuesta (CASCADE), FK a Pregunta (PROTECT), valor_texto (para tipos abiertos) y opciones_seleccionadas (ManyToManyField a OpcionPregunta). unique_together=(respuesta, pregunta).

RespuestaCelda Elección columna-por-fila en una cuadrícula. Campos: FK a RespuestaPregunta (CASCADE), FK a FilaCuadricula (PROTECT) y FK a OpcionPregunta (PROTECT). unique_together=(respuesta_pregunta, fila).

RespuestaSeccion *Snapshot* del puntaje por sección al enviar. Campos: FK a Respuesta (CASCADE), FK a Seccion (PROTECT), peso (copia del peso vigente al enviar), puntaje_obtenido, puntaje_maximo y porcentaje. El campo peso preservado permite reconstruir el desglose aunque el administrador re-ajuste los pesos y publique una revisión.

Relaciones (agrupadas por dueño):

- Periodo 1- N Formulario y Usuario 0,1- N Formulario (*creador*).
- Formulario 1- N {Seccion, Pregunta, NivelDesempeno, Respuesta}.
- Seccion 0,1- N Pregunta (una pregunta puede quedar huérfana si su sección se elimina, aunque la validación de publicación lo prohíbe).
- Pregunta 1- N OpcionPregunta y Pregunta 1- N FilaCuadricula.
- Profesor 1- N Respuesta y NivelDesempeno 0,1- N Respuesta.
- Respuesta 1- N RespuestaPregunta y Respuesta 1- N RespuestaSeccion.
- Pregunta 1- N RespuestaPregunta; Seccion 1- N RespuestaSeccion.
- RespuestaPregunta $N-M$ OpcionPregunta (opciones_seleccionadas, con tabla intermedia autogenerada).
- RespuestaPregunta 1- N RespuestaCelda; FilaCuadricula 1- N RespuestaCelda; OpcionPregunta 1- N RespuestaCelda.

App reportes — vistas de agregación

No define modelos propios. Contribuye únicamente con *views* (véase el Módulo 6, Sección 6.9) que ejecutan agregaciones sobre las entidades de documentos, autoevaluacion y catalogos usando aggregate, annotate, Count y Avg del ORM.

6.4. Módulo 1: autenticación y gestión de usuarios

Este módulo, implementado en la app `accounts`, controla el acceso al sistema y la gestión del ciclo de vida de los usuarios.

6.4.1. Autenticación con JWT

El `endpoint` `POST /api/v1/auth/login/` extiende `TokenObtainPairView` de `simplejwt` con un `serializer` personalizado (Apéndice 11.1, “Backend — Login con distinción de cuenta desactivada”) que:

- Diferencia *credenciales inválidas de cuenta desactivada* en los mensajes de error mediante un código `account_disabled`, para dar diagnóstico útil sin filtrar existencia de cuentas.
- Emite dos tokens: un *access token* con vigencia de 60 minutos y un *refresh token* con vigencia de 7 días. Los *refresh tokens* se rotan en cada uso (`ROTATE_REFRESH_TOKENS=True`) y el anterior queda invalidado en la lista negra de `simplejwt` (`BLACKLIST_AFTER_ROTATION=True`), por lo que reutilizarlo devuelve 401.
- Aplica `throttle_scope="login"` para limitar a cinco intentos por minuto por dirección IP (véase § 6.12).
- Devuelve, junto con los tokens, un objeto `user` con `id`, `email`, `nombre`, `rol`, útil para poblar el contexto de sesión del frontend en la misma respuesta.

El `endpoint` `POST /api/v1/auth/refresh/` intercambia un *refresh token* válido por un nuevo par de tokens. El cliente frontend intercepta las respuestas HTTP 401 y dispara automáticamente el *refresh* antes de reintentar la petición original: mientras el *refresh* está en curso, las peticiones concurrentes que también fallan con 401 se encolan y se despachan de una sola vez con el nuevo *access token* (evita la “tormenta de *refresh*”). El `endpoint` `GET /api/v1/auth/me/` devuelve el perfil completo del usuario autenticado, incluyendo el *perfil_profesor* embebido cuando aplica.

6.4.2. Creación de un profesor con acceso al sistema

La creación de un profesor sigue un flujo operativo unificado desde el rol administrador:

1. El administrador entra a `/admin/profesores`, oprime *Nuevo profesor* y llena el formulario en la ventana modal: *nombre completo*, *correo institucional* (que será el `USERNAME_FIELD`), *número económico*, *departamento* y *contraseña inicial*.
2. El frontend envía `POST /api/v1/profesores/crear-con-usuario/`. En una única transacción atómica el backend:
 - a) verifica si el correo ya corresponde a un usuario existente sin perfil de profesor asociado (“usuario huérfano”); si es así, lo reutiliza actualizando su rol y contraseña;

- b) de lo contrario crea el `Usuario` con `rol=PROFESOR` y almacena la contraseña como *hash* PBKDF2;
 - c) crea el `Profesor` enlazado por FK al usuario y al departamento.
3. Devuelve 201 `Created` con el profesor completo. Este puede iniciar sesión inmediatamente con el correo y la contraseña recibida.

La misma tabla ofrece acciones adicionales por fila, además de *Editar* (que incluye la casilla *Cuenta activa*):

- **Restablecer contraseña:** dispara una ventana modal secundaria; `POST /api/v1/usuarios/{id}/set-password/` valida la nueva contraseña con la política estándar de Django (longitud mínima, no común, no numérica, no similar al perfil) y la almacena como *hash*.
- **Activar / desactivar:** la casilla *Cuenta activa* de la ventana modal de edición actualiza `Profesor.estado`, que sincroniza `Usuario.is_active`; la API expone además `POST /api/v1/usuarios/{id}/toggle-activo/`, que conmuta `is_active`. Al desactivar, el usuario queda con sesión expirada al vencer su *access token* actual (máximo 60 minutos) y no podrá refrescar ni iniciar sesión de nuevo.
- **Eliminar:** borra el perfil de profesor y su cuenta de usuario; los documentos ya creados se conservan gracias a los *snapshots* de identidad (§ 5.22).

La Figura 4 muestra la ventana modal “Nuevo Profesor” con los campos que consume el formulario descrito arriba.

6.4.3. Roles y permisos

Los dos roles *ADMIN* y *PROFESOR* se materializan como valores de un `TextChoices` (mecanismo de Django que combina una enumeración con etiquetas legibles) en el modelo `Usuario`. La comprobación de rol se realiza:

- En **backend**, mediante *permission classes* de DRF: `IsAdmin` para operaciones exclusivas de administrador, `IsAdminOrReadOnly` para catálogos, `IsProfesor` para acciones del profesor y `IsOwnerProfesorOrAdminReadOnly` para restringir la escritura al dueño del recurso y permitir solo lectura al administrador (Apéndice 11.1, “Backend — Permisos por rol”). El chequeo tiene dos capas: `has_permission` (a nivel de *ViewSet*) y `has_object_permission` (a nivel de instancia).
- En **frontend**, mediante los *route guards* `AdminRoute` y `ProfesorRoute` (§ 5.15) que componen `ProtectedRoute + RoleRoute` y redirigen si el rol no coincide.

6.5. Módulo 2: catálogos institucionales

La app `catalogos` expone *ViewSets* de CRUD completo para `Departamento`, `Licenciatura`, `Posgrado`, `Area`, `UEA` y `Periodo`, todos regidos por `IsAdminOrReadOnly`: cualquier usuario autenticado los lee (necesario para poblar selectores en el frontend), pero solo el administrador los modifica.

6.5.1. Precarga inicial y *fixtures*

Cuatro *fixtures* JSON (en `backend/catalogos/fixtures/`) precargan los catálogos institucionales estables: los cuatro departamentos de CyAD, las licenciaturas y posgrados activos, y las áreas de conocimiento. Se aplican con el comando `loaddata` de Django. Es necesario situarse primero en el directorio del proyecto Django (`backend/`, donde vive `manage.py`), activar el entorno virtual que aísla las dependencias (para que `python` y `manage.py` usen la versión de Django instalada por `pip install -r requirements.txt`, no el intérprete del sistema) y respetar el orden que dictan las claves foráneas (departamentos antes que licenciaturas y posgrados):

```
cd backend
.\.venv\Scripts\Activate.ps1    # Windows (PowerShell)
# source .venv/bin/activate      # Linux / macOS
python manage.py loaddata \
    catalogos/fixtures/departamentos.json \
    catalogos/fixtures/licenciaturas.json \
    catalogos/fixtures/posgrados.json \
    catalogos/fixtures/areas.json
```

Las rutas indicadas son relativas a `backend/`. El comando `loaddata` es idempotente (usa la clave primaria de cada registro), por lo que se puede re-ejecutar sin duplicar datos: si un registro ya existe, lo actualiza. Esto reduce el tiempo de puesta en marcha del sistema a minutos, sin capturar manualmente los mismos datos.

Los Periodos académicos *no* se precargan como *fixture*: son datos temporales (una clave y dos fechas por trimestre) que envejecen mal y llevarían el JSON a estar desactualizado unos meses después de cada corte. El primer administrador captura el periodo vigente desde la interfaz web (`/admin/catalogos/periodos`), activando los tres *flags* por recurso descritos en la Subsubsección “Módulo 3: habilitación de periodos por recurso”. La creación del entorno virtual (`python -m venv .venv`), la instalación de dependencias y la configuración del archivo `.env` se documentan en el manual de instalación (Apéndice 11.3, “Manual de instalación”).

6.5.2. Importación masiva de UEAs desde CSV

Las UEAs son numerosas (una por asignatura por programa: 430 en las cuatro licenciaturas de CyAD y 81 en los posgrados vigentes, 511 en total), por lo que la carga se hace en bloque desde CSV. El proyecto ya trae los dos CSV oficiales que mantiene coordinación académica (`ueas_licenciatura.csv` y `x` dentro de `backend/catalogos/fixtures/`, y ofrece dos vías de carga idempotente que consumen el mismo formato:

- **Desde la línea de comandos:** el comando de gestión `python manage.py cargar_catalogos_csv--ueas-csv<ruta>` (Apéndice 11.1, “Backend — Comando de importación CSV para UEA”). Es la vía canónica para el arranque institucional y para actualizaciones incrementales cuando coordinación envía un CSV nuevo.
- **Desde la interfaz:** `POST /api/v1/uea/import-csv/` con `multipart/form-data`. El backend lee las cabeceras `clave`, `nombre`, `programa_clave`, `trimestre`,

`tipo`, `creditos`, `area_nombre`, `area_descripcion` y `url`, hace *upsert* usando cachés en memoria para evitar *queries* redundantes y detecta colisiones cuando la clave existe en licenciatura y posgrado simultáneamente.

Ambas rutas resuelven el área de cada UEA contra los pares (`nombre`, `descripcion`) de la tabla `Area` (precargada por `areas.json`); las áreas que no se encuentren se reportan como *warning* y la UEA queda con `area = NULL`, nunca se crean áreas nuevas de forma silenciosa.

6.5.3. Restricciones de integridad de UEA

El modelo UEA refuerza dos reglas del dominio como *constraints* declarativos (Apéndice 11.1, “Backend — Constraint XOR y unicidad parcial de UEA”):

- `uea_xor_licenciatura_posgrado`: una UEA pertenece a una licenciatura o a un posgrado, nunca a ambos ni a ninguno (§ 5.10).
- Dos `UniqueConstraint` parciales que hacen la clave única por programa: la clave “1400027” puede coexistir en una licenciatura y en un posgrado, pero no repetirse dentro del mismo programa.

Estas reglas se validan además en el *serializer* para producir mensajes de error legibles.

La Figura 5 ilustra la ventana modal de importación de UEAs desde CSV en `/admin/catalogos/uea`, con el formato de columnas aceptado.

6.6. Módulo 3: habilitación de periodos por recurso

Cada `Periodo` tiene tres *flags* de activación independientes:

activo_cartas Habilita la captura y publicación de cartas temáticas para ese periodo.

activo_requisitos Habilita la captura y publicación de requisitos de recuperación.

activo_autoevaluacion Habilita la creación de formularios de autoevaluación y la recepción de respuestas.

La separación en tres *flags* refleja el ritmo real de la Coordinación de Estudios: la ventana de captura de cartas se abre al inicio del trimestre, la de requisitos hacia la mitad, y la de autoevaluación al final. Los tres pueden estar activos en periodos distintos simultáneamente. Hacerlo de esta manera evita que los profesores se confundan y publiquen documentos en periodos que no corresponden a la fase de captura vigente.

6.6.1. Invariante “a lo sumo uno activo por recurso”

El sistema garantiza que solo un periodo esté activo por recurso a la vez. Esto se implementa con un `save()` sobrescrito en `Periodo`: al persistir un periodo con `activo_cartas=True`, la misma operación de guardado hace `activo_cartas=False` en todos los demás periodos. Lo mismo aplica a los otros dos *flags*. Un método de clase `Periodo.get_activo(recurso)` devuelve el periodo actualmente activo para un recurso dado, consumido por dashboards y por el asignador automático de periodo en los formularios de creación.

6.6.2. Interfaz de usuario

En `/admin/catalogos/periodos` cada fila del listado muestra tres *chips* (etiquetas) clicables (verde=activo, gris=inactivo). Al hacer clic sobre uno, el frontend dispara `PATCH /api/v1/periodos/{id}/` con el nuevo valor booleano; el resultado se refleja en toda la tabla porque los demás *chips* del mismo recurso pasan a gris automáticamente.

6.6.3. Efectos sobre otros módulos

La habilitación se propaga:

- El *serializer* de `CartaTematica` rechaza altas si el periodo destino no tiene `activo_cartas=True` (equivalente para requisitos y autoevaluación).
- El *ViewSet* de documentos calcula un *flag* derivado `puede_editar_ahora` que combina `estado=BORRADOR` con `periodo.activo_cartas=True`. El frontend lo usa para deshabilitar botones de edición cuando el periodo se cierra.
- El *endpoint* `GET /api/v1/periodos/activos/` devuelve un objeto `{cartas: {...}, requisitos: {...}, autoevaluacion: {...}}` consumido por los dashboards y por los formularios de creación de documento (para asignar automáticamente el periodo activo).

6.6.4. Eliminación protegida

Como `Periodo` es referenciado por múltiples entidades con FK `PROTECT`, eliminar un periodo requiere una operación cuidadosa:

1. El administrador consulta `GET /api/v1/periodos/{id}/preview-eliminacion/`, que devuelve el conteo de documentos y respuestas que se borrarían en cascada.
2. El `DELETE` está bloqueado si el periodo está activo para algún recurso.
3. Si el usuario confirma, `perform_destroy` realiza una cascada manual dentro de una transacción atómica en el orden `Respuesta → Formulario → CartaTematica → RequisitoRecuperacion → Periodo`.

La Figura 6 muestra la tabla de periodos con los tres *chips* por fila (Cartas, Requisitos, Autoevaluación) que reflejan el estado activo/inactivo, junto con la ventana modal de alta.

6.7. Módulo 4: cartas temáticas y requisitos de recuperación

Este módulo, implementado en la app `documentos`, unifica el CRUD de los dos documentos porque comparten patrón: misma clase base abstracta, mismo ciclo de vida BORRADOR $\leftrightarrow$ PUBLICADO, misma unicidad compuesta y mismas reglas de propiedad. Difieren en los campos específicos y en el *flag* de periodo que los habilita.

6.7.1. Creación y publicación desde el rol profesor

El flujo operativo del profesor es el siguiente:

1. Entra a `/profesor/cartas` (o `/profesor/requisitos`). Ve la lista de sus documentos agrupada por periodo con el componente `CollapsiblePeriodoCard` (§ 5.16, patrón acordeón). La tarjeta del periodo activo se abre automáticamente al terminar de cargar los periodos activos.
2. Oprime *Nueva carta*. El formulario (`CartaFormPage`) asigna automáticamente el periodo activo consultando `GET /periodos/activos/`; ese campo no es editable, evitando errores de captura.
3. Selecciona la UEA con un buscador libre que sugiere las diez mejores coincidencias por clave o nombre.
4. Llena *nombre de grupo*, *id de grupo*, *horario*, *modalidad* (presencial/remoto/mixto) y los doce campos de contenido de la carta (o los seis del requisito).
5. Elige *Guardar borrador* (`POST /cartas-tematicas/` o `PATCH /cartas-tematicas/{id}/`) o *Guardar y publicar*. Esta segunda opción encadena la actualización con `POST /cartas-tematicas/{id}/cambiar-estado/` para transicionar el documento a PUBLICADO.

Al crearse el documento, el `ViewSet` llena automáticamente los campos `profesor_nombre` y `profesor_correo` como *snapshots* de identidad (§ 5.22), extraídos del perfil del profesor autenticado. Estos *snapshots* sobreviven a la eventual desactivación o eliminación del usuario.

Las evidencias visuales del dashboard del profesor y de los formularios de creación de documentos se muestran en las Figuras 11, 12 y 13 de la Sección 7.

6.7.2. Reglas de propiedad y ediciones fuera de tiempo

El `ViewSet` de cartas usa el permiso `IsOwnerProfesorOrAdminReadOnly`: el profesor solo ve y edita sus propios documentos; el administrador tiene acceso de solo lectura a todos. Adicionalmente, un método privado `_profesor_puede_modificar(obj)` bloquea cualquier modificación — incluida la transición de estado — si el periodo del documento ya no tiene el *flag* correspondiente activo. Con esto, un profesor no puede alterar retroactivamente cartas de trimestres cerrados.

La `UniqueConstraint` compuesta (*profesor*, *periodo*, *uea*, *id_grupo*) impide, a nivel de motor de base de datos, que un profesor tenga dos documentos para el mismo grupo en el

mismo trimestre. En el evento de una violación, DRF devuelve 400 `Bad Request` con el siguiente mensaje: “Ya existe un documento para este profesor, periodo, UEA y grupo”.

6.7.3. Vista del administrador

El administrador dispone de las páginas `/admin/documentos/cartas` y `/admin/documentos/requisitos` con la vista global de los documentos *publicados*, filtrable por periodo, con enlaces a la vista pública imprimible de cada uno. La interfaz es de solo lectura: la publicación es responsabilidad del dueño; el administrador supervisa y descarga.

6.8. Módulo 5: *form-builder* y autoevaluación docente

Este es el módulo más elaborado del sistema. Se construyó en nueve incrementos PR1–PR9 (Sección 6.2) y culmina con el envío por parte del profesor de una respuesta puntuada y evaluada contra rangos de desempeño.

6.8.1. Flujo operativo del administrador (*form-builder*)

1. El administrador entra a `/admin/autoevaluacion` y ve la lista de formularios (con *polling* cada 15 s para reflejar cambios entre pestañas). Crea un formulario nuevo asociado a un `Periodo` que tenga `activo_autoevaluacion=True`.
2. Entra al constructor (`FormularioBuilderPage`). La interfaz se organiza en tres pestañas: *Preguntas*, *Niveles* y *Estadísticas*.
3. En *Preguntas* añade **secciones** con *peso decimal*. Un indicador visual muestra en tiempo real “ Σ pesos = X %” con marca de verificación cuando la suma es exactamente 100 (tolerancia $\pm 0,01$) o advertencia en caso contrario.
4. Añade preguntas dentro de cada sección. El sistema soporta ocho tipos:
 - a) Texto corto (una línea).
 - b) Texto largo (párrafo).
 - c) Opción única (*radio*).
 - d) Casillas (*checkboxes* múltiples).
 - e) Sí/No.
 - f) Escala lineal (Likert 1–*N* con etiquetas configurables; § 5.23).
 - g) Lista desplegable.
 - h) Cuadrícula (matriz filas $\times$ columnas para preguntas Likert compactas).

Para los tipos con opciones, define *puntos* por opción; para `ESCALA_LINEAL` configura `min`, `max`, `puntos_factor`; para `CUADRICULA` define filas y opciones (columnas).

5. En *Niveles* define los *niveles de desempeño*: rangos porcentuales con etiqueta, observación textual y color (por ejemplo, “0–59.99 En desarrollo”, “60–84.99 Satisfactorio”, “85–100 Sobresaliente”). Si un porcentaje cae en el límite de dos rangos, se asigna el nivel de menor orden.
6. Cuando la estructura está completa, oprime *Publicar*. El backend valida la estructura (véase la transición `publicar` en la Sección 6.8.2) y transiciona el formulario a `PUBLICADO`.

6.8.2. Máquina de estados del formulario

Un Formulario vive en una máquina de estados con tres estados (`BORRADOR`, `PUBLICADO` y `CERRADO`) y cinco transiciones etiquetadas, además de una acción transversal `duplicar`. La Figura 7 resume el diagrama; la implementación de las transiciones se encuentra en el Apéndice 11.1 (“Backend — Ciclo de vida de Formulario”).

La estructura del formulario (secciones, preguntas, opciones, pesos y niveles) solo es editable en estado `BORRADOR`; cualquier cambio a un formulario publicado o cerrado pasa por `despublicar`.

Transiciones que preservan la versión.

- `BORRADOR` → `PUBLICADO`: acción `publicar`. Valida que haya al menos una sección, que la suma de pesos sea $100 \pm 0,01$ %, que no existan preguntas huérfanas (sin sección) y que cada sección con peso > 0 tenga al menos una pregunta puntuable. Registra `published_at`. Si ya existían respuestas de la versión vigente (es decir, el formulario se editó después de haber sido contestado), la publicación incrementa automáticamente la versión para no mezclar respuestas de instrumentos distintos.
- `PUBLICADO` → `CERRADO`: acción `cerrar`. Deja de recibir respuestas. Registra `closed_at`.
- `CERRADO` → `PUBLICADO`: acción `reabrir`. Se emplea cuando el administrador cerró el formulario anticipadamente o por error, y desea volver a aceptar respuestas del **mismo instrumento** (mismos ítems, mismos pesos, misma *version*). Las respuestas ya recibidas se conservan y siguen contabilizando; los profesores que no habían respondido pueden hacerlo sin fricción.
- `PUBLICADO/CERRADO` → `BORRADOR`: acción `despublicar`. Es la única vía para editar preguntas, pesos y niveles; no incrementa la versión ni borra respuestas.

Transición con nueva versión.

- `CERRADO` → `PUBLICADO` (versión $n + 1$): acción `publicar_revision`. Se utiliza cuando el administrador quiere abrir una nueva ronda de respuestas sobre un formulario ya cerrado. A diferencia de `reabrir`, **incrementa *version***: las respuestas anteriores quedan asociadas a su versión histórica y no cuentan para el conteo de cumplimiento de la nueva; cada profesor debe volver a responder la revisión.

Acción transversal.

- **duplicar:** clona la estructura completa (secciones, preguntas, opciones, filas y niveles) a otro periodo, dentro de una transacción atómica, sin arrastrar respuestas. Puede ejecutarse desde cualquier estado y facilita reutilizar un formulario ya afinado en trimestres sucesivos.

6.8.3. Sistema de puntuación (*scoring* ponderado)

Cuando el profesor envía una respuesta, la acción `POST /respuestas/{id}/enviar/` desencadena el cálculo del puntaje. El algoritmo, resumido en el Apéndice 11.1 (“Backend — Cálculo ponderado del puntaje de autoevaluación”), procede así:

1. Para cada `Seccion`, recorre sus `Preguntas` puntables e ignora las de tipo `TEXTO_CORTO/TEXTO_LARGO` (colección `TIPOS_NO_PUNTABLES`).
2. Suma los puntos obtenidos y el máximo posible según el tipo:
 - **CUADRICULA:** $\text{máximo} = (\text{mayor } \textit{puntos} \text{ entre las opciones}) \times (\text{número de filas})$; $\text{obtenido} = \text{suma de puntos de cada celda respondida}$.
 - **ESCALA_LINEAL:** $\text{máximo} = \text{max} \times \text{puntos_factor}$; $\text{obtenido} = \text{valor respondido} \times \text{puntos_factor}$.
 - **SI_NO:** puntos configurados para la opción elegida (`puntos_si` o `puntos_no`).
 - **CASILLAS:** suma de puntos de todas las opciones marcadas.
 - **OPCION_UNICA / LISTA_DESPLEGABLE:** puntos de la opción seleccionada; $\text{máximo} = \text{mayor } \textit{puntos} \text{ entre las opciones}$.
3. Calcula el porcentaje por sección: $\text{porcentaje}_s = (\text{obtenido}_s / \text{máximo}_s) \times 100$.
4. Calcula el porcentaje ponderado global sumando, sobre todas las secciones, $\text{porcentaje}_s \times (\text{peso}_s / 100)$.
5. Persiste un `RespuestaSeccion` por sección con `puntaje`, `puntaje_maximo`, `porcentaje` y una **copia del peso** de la sección. Este *snapshot* garantiza que el desglose histórico permanezca estable aun si el administrador ajusta los pesos y publica una revisión.
6. Determina el `NivelDesempeno` correspondiente buscando el rango `[porcentaje_min, porcentaje_max]` que contiene al porcentaje global; adjunta el nivel y su observación al resultado.

6.8.4. Privacidad de la ponderación mientras se contesta

El sistema separa dos *serializers* de `OpcionPregunta`: `OpcionPreguntaSerializer` (para el administrador) expone el campo `puntos`; `OpcionPreguntaPublicSerializer` (para el profesor mientras contesta) lo omite. Así el respondiente no puede optimizar su elección conociendo la ponderación de antemano. Al enviar, el profesor recibe el resultado completo con desglose.

6.8.5. Flujo operativo del profesor

1. El profesor entra a `/profesor/autoevaluacion`. Ve la lista de formularios disponibles con un *badge* por estado (Pendiente, En progreso, Respondido, Periodo cerrado) y *polling* cada 15 s.
2. Abre un formulario. El `QuestionRenderer` (Apéndice 11.1, “Frontend — Renderizador dinámico de preguntas”) despacha por `pregunta.tipo` y produce el control apropiado (*radio*, *checkbox*, *select*, escala 1–*N*, cuadrícula o *textarea*).
3. Puede *Guardar borrador* tantas veces como quiera (POST `/respuestas/` inicialmente, luego PATCH `/respuestas/{id}/`).
4. Cuando termina, oprime *Enviar* (POST `/respuestas/{id}/enviar/`). Si faltan respuestas obligatorias, el backend responde con `preguntas_faltantes` y el frontend resalta cada pregunta faltante en rojo y hace *scroll* suave a la primera.
5. Al éxito, se muestra un *ResultCard* con el puntaje obtenido, el porcentaje global, el nivel de desempeño asignado, la observación textual del nivel y el **desglose por sección** (porcentaje de la sección y aporte al total).

Además del constructor completo (Figura 17) y del formulario contestado por el profesor (Figura 14) que se muestran en la Sección 7, la Figura 8 presenta el panel de *Niveles de Desempeño* con los rangos configurables por el administrador, y las estadísticas agregadas por formulario se ilustran en la Figura 9 más adelante en esta sección.

6.9. Módulo 6: reportes de cumplimiento y descargas XLSX

6.9.1. Endpoints de agregación

La app `reportes` expone cinco endpoints de solo lectura, todos restringidos al rol *Administrador*:

GET `/api/v1/reportes/dashboard/?periodo={id}` Métricas globales: total de profesores activos, conteo de cartas y requisitos por estado (BORRADOR/PUBLICADO), y desglose de formularios y respuestas de autoevaluación.

GET `/api/v1/reportes/cumplimiento/?periodo=&departamento=` Detalle de cumplimiento por departamento: para cada uno, cuenta profesores activos y cuántos publicaron carta y requisito en el periodo, con porcentajes.

GET /api/v1/reportes/cumplimiento-licenciatura/ Ídem, agrupado por licenciatura, con conteos de UEA cubiertas.

GET /api/v1/reportes/autoevaluacion-profesores/ Lista de profesores con su porcentaje del último formulario, indicando quién ya envió y quién no.

GET /api/v1/reportes/autoevaluacion/ Estadísticas descriptivas por formulario: respuestas enviadas, total de profesores, tasa de respuesta, promedio general de porcentaje, distribución por nivel de desempeño y promedio por sección — este último calculado con `RespuestaSeccion.objects.filter(...).aggregate(Avg("porcentaje"))`.

Todos los agregados se calculan con la API de agregación del ORM (`aggregate`, `annotate`, `Count`, `Avg`), sin ejecutar SQL manual, aprovechando la parametrización automática frente a inyección.

6.9.2. Generación de XLSX del lado del cliente

Para las descargas tabulares el frontend usa *SheetJS* (biblioteca `xlsx`), evitando cargar al backend con generación de binarios (Apéndice 11.1, “Frontend — Generación de XLSX por periodo”). El patrón adoptado es **una hoja por periodo**:

1. El administrador entra a `/admin/reportes` y elige tipo de reporte (cartas, requisitos o autoevaluación), periodo (o *Todos*) y opcionalmente departamento.
2. El frontend pide los datos ya publicados, con los filtros aplicados y sin paginación: `GET /api/v1/publico/cartas/?estado=PUBLICADO&periodo={id}&profesor__departamento={id}` (o su equivalente).
3. Agrupa las filas en un `Map<periodo_clave, rows[]>`.
4. Crea el libro con `XLSX.utils.book_new()` y añade una hoja por grupo con `XLSX.utils.json_to_sheet(rows.map(rowToExcel))` y `XLSX.utils.book_append_sheet(wb, ws, sheetName(clave))`.
5. Dispara la descarga en el navegador con `XLSX.writeFile(wb, filename)`.

El compromiso es que el navegador procesa la conversión — aceptable para los volúmenes de una división (cientos de documentos por trimestre), aunque degradaría con volúmenes mucho mayores. La ventaja es que se puede iterar sobre el formato de las hojas sin desplegar cambios en el servidor.

La evidencia visual del panel de reportes está en la Figura 18.

6.10. Módulo 7: vista pública de exploración

Este módulo permite a cualquier persona consultar cartas y requisitos publicados sin autenticarse. Se apoya en un conjunto de *endpoints* bajo el prefijo `/api/v1/publico/` con permiso `AllowAny` y `authentication_classes=[]`, sin paginación y filtrados a documentos en estado `PUBLICADO`.

6.10.1. Cascada de tres pasos

La página `ExplorarPage (/publico/explorar/:tipo con tipo ∈ {cartas, requisitos})` implementa una cascada (§ 5.16):

1. **Periodo:** se listan los periodos que tengan documentos publicados del tipo elegido mediante `GET /publico/periodos/?tipo=carta`. Esto evita mostrar periodos vacíos.
2. **Programa:** el visitante elige entre licenciaturas y posgrados vía `GET /publico/licenciaturas/` y `GET /publico/posgrados/`.
3. **UEA:** se cargan las UEA con documentos disponibles vía `GET /publico/uea/?licenciatura={id}&periodo={id}&con_documentos_de=carta`. Para licenciaturas, la interfaz las agrupa por *trimestre* (patrón acordeón) o por nombre del área optativa; para posgrados, se muestran en un bloque único.

Los documentos publicados por cada UEA se cargan bajo demanda (*lazy load*) al expandir la sección correspondiente, usando la propiedad `enabled: expanded` de *TanStack Query*. Esto evita descargar cientos de documentos en la carga inicial.

6.10.2. Vista imprimible

Al hacer clic sobre un documento, la aplicación abre `/publico/cartas/{id}` o `/publico/requisitos/{id}`. Estas páginas usan `PublicDocumentLayout`, un envoltorio que aplica clases Tailwind con el prefijo `print:` (por ejemplo, `print:hidden` en la barra de acciones, `print:bg-white` en el fondo) para producir un PDF limpio desde el diálogo de impresión del navegador (`window.print()`). No se usan bibliotecas de generación de PDF del lado del servidor.

Un cliente axios separado (`publicClient` en `src/api/publico.js`) **sin** interceptores de JWT sirve estos endpoints, para evitar enviar `Authorization` cuando el usuario está autenticado en otra pestaña. El backend igualmente ignoraría el token porque los endpoints públicos usan `AllowAny`, pero el aislamiento simplifica el razonamiento.

La evidencia visual está en la Figura 20 de la Sección 7, y la vista de detalle imprimible se muestra en la Figura 21.

6.11. Aspectos transversales

6.11.1. Documentación viva de la API

El complemento *drf-spectacular* analiza las anotaciones de los *ViewSets* y *serializers* y genera un documento OpenAPI (§ 5.8) expuesto en `/api/schema/` (YAML descargable) y `/api/docs/` (Swagger UI interactivo). Esto elimina la clase de errores que produce mantener documentación manual desincronizada.

6.11.2. Formato de error unificado

Un *exception handler* personalizado (`core.exceptions.api_exception_handler`) envuelve todas las respuestas de error de DRF como `{success: false, status_code: N, errors: {...}}`. El frontend consume este contrato con una utilidad `parseApiError()` que prioriza campos conocidos (`preguntas_faltantes`, `detail`, `non_field_errors`) para producir mensajes legibles.

6.11.3. Pruebas automatizadas

El *backend* cuenta con una suite de pruebas automatizadas construida con *pytest* 8, *pytest-django* 4 y *factory-boy* 3. La distribución de pruebas por módulo es:

- `tests/test_auth.py`: 25 pruebas.
- `tests/test_autoevaluacion.py`: 81 pruebas.
- `tests/test_catalogos.py`: 34 pruebas.
- `tests/test_documentos.py`: 32 pruebas.
- `tests/test_reportes.py`: 25 pruebas.
- **Total**: 197 pruebas.

El archivo `backend/conftest.py` incluye una *fixture autouse* que limpia el *cache* de Django antes de cada prueba, evitando que el *throttling* acumulado del *login* contamine tests posteriores. La ejecución de la suite completa toma varios minutos (alrededor de siete en el equipo de desarrollo) y no requiere infraestructura adicional (usa una base de datos de test aislada). Los detalles del reporte de cobertura se presentan en la Sección 7.

6.12. Seguridad

Las medidas de seguridad implementadas siguen las recomendaciones de *OWASP Top 10* [29]:

- **Modo DEBUG seguro por defecto**: `DEBUG=False` salvo que `.env` lo active explícitamente (`config/settings.py`), evitando la exposición accidental de trazas y variables en producción.
- **Rate limiting del login**: cinco intentos por minuto vía `ScopedRateThrottle` de DRF con `throttle_scope="login"`. Después del quinto intento fallido, el endpoint responde 429 Too Many Requests.
- **Validación de contraseñas**: se aplican los cuatro validadores estándar de Django (similitud con datos de usuario, longitud mínima, contraseñas comunes, contraseñas numéricas).

- **Hashing de contraseñas:** PBKDF2 con SHA-256 por defecto (algoritmo estándar de Django para el cálculo del *hash* de las contraseñas [30]), reforzado con *salt* aleatorio.
- **CORS lista blanca:** los orígenes permitidos se configuran vía `CORS_ALLOWED_ORIGINS` y se envían credenciales solo con los que estén en la lista.
- **Autenticación por JWT rotativo:** el *refresh* rota en cada uso, minimizando el impacto de una filtración; los tokens se firman con HMAC-SHA256.
- **Diferenciación de rol y permisos por ViewSet:** no hay endpoints sin permiso explícito; el *default* global es `IsAuthenticated`. La escritura sobre recursos propios se restringe adicionalmente con `IsOwnerProfessorOrAdminReadOnly`.
- **Endpoints públicos aislados:** bajo el prefijo `/api/v1/publico/`, expuestos con permiso `AllowAny` pero limitados a lectura de documentos en estado *PUBLICADO*, sin exponer campos internos ni contadores.
- **Prevención de inyección SQL:** garantizada por el uso exclusivo del ORM de Django; no se ejecuta SQL crudo con interpolación de cadenas en ninguna vista.

7. Resultados

Esta sección presenta los resultados obtenidos con evidencia cuantitativa (métricas del código, tests, cobertura y endpoints) y evidencia cualitativa (capturas de pantalla del sistema en operación con datos de *seed*).

7.1. Métricas del código y de la base de datos

La Tabla 2 resume el volumen del sistema entregado.

Tabla 2. Métricas del código entregado.

Componente	Cantidad	Notas
Apps Django	6	core, accounts, catalogos, documentos, autoevaluacion, reportes
Migraciones propias	20	Distribuidas entre las apps de dominio
Endpoints REST de la API v1	~45	Documentados automáticamente vía OpenAPI en /api/docs/
Modelos concretos	20	Sin contar las clases abstractas TimeStampedModel, EstadoActivoModel y DocumentoAcademicoBase
Dependencias Python	13	8 de ejecución, 4 de pruebas y 1 operativa; ver backend/requirements.txt
Dependencias JavaScript (prod)	9	Ver frontend/package.json
Tipos de pregunta soportados	8	TEXTO_CORTO, TEXTO_LARGO, OPCION_UNICA, CASILLAS, SI_NO, ESCALA_LINEAL, LISTA_DESPLEGABLE, CUADRICULA
Componentes UI reutilizables	10+	Button, Alert, Badge, Table, Modal, FormField, Loading, EmptyState, CollapsiblePeriodoCard, entre otros
Fixtures y CSV precargados	6	Cuatro <i>fixtures</i> JSON (departamentos, licenciaturas, posgrados, áreas) y dos CSV de UEA (catalogos/fixtures/)

7.2. Trazabilidad con los objetivos

La Tabla 3 relaciona cada objetivo específico (Sección 4) con la funcionalidad que lo atiende y la evidencia que lo respalda.

Tabla 3. Trazabilidad entre objetivos específicos, funcionalidad y evidencia.

Obj.	Objetivo	Funcionalidad	Evidencia
1	Acceso con autenticación y diferenciación de roles	Login con JWT, roles ADMIN/PROFESOR, permisos por rol y limitación de intentos	Figura 10; tests/test_auth.py (25 pruebas)
2	Crear, modificar y consultar cartas y requisitos	CRUD con ciclo Borrador/Publicado y vista pública	Figuras 12, 13 y 21; tests/test_documentos.py (32 pruebas)
3	Registrar la autoevaluación al final del trimestre	Constructor de formularios, captura de respuestas y puntaje ponderado	Figuras 17 y 14; tests/test_autoevaluacion.py (81 pruebas)
4	Reportes de cumplimiento para la administración	Endpoints de agregación, tablero y exportación XLSX	Figuras 15 y 18; tests/test_reportes.py (25 pruebas)

7.3. Métricas de pruebas

La Tabla 4 muestra el número de pruebas automatizadas del *backend* por archivo y la cobertura de línea del código de cada *app*, medida con `coverage.py` a través de `pytest-cov` (comando `pytest -cov=. -cov-report=term`). La cobertura se calcula sobre el código de la aplicación, sin contar las propias pruebas ni las migraciones.

Tabla 4. Pruebas automatizadas y cobertura por módulo.

Archivo de pruebas	Pruebas	App cubierta	Cobertura
tests/test_auth.py	25	accounts	90 %
tests/test_autoevaluacion.py	81	autoevaluacion	86 %
tests/test_catalogos.py	34	catalogos	89 %
tests/test_documentos.py	32	documentos	96 %
tests/test_reportes.py	25	reportes	98 %

Archivo de pruebas	Pruebas	App cubierta	Cobertura
Total	197	Código de la aplicación	≈89 %

La ejecución completa de la *suite* arroja **197 pruebas exitosas de 197** (100 %) y una **cobertura de línea de aproximadamente 89 %** sobre el código de la aplicación. La cifra global que reporta la herramienta (94 %) incluye los archivos de prueba, que se ejecutan por completo, por lo que no se toma como medida del código de producción. Las *apps* `core` y `config` suman otro 90 % y 80 % respectivamente. El módulo con menor cobertura es `autoevaluacion/views.py` (79 %).

7.4. Evidencia visual del sistema en operación

Las siguientes capturas se tomaron con el sistema ejecutándose localmente contra la base de datos de *seed* (`python scripts/seed_demo.py`). Las cuentas utilizadas son las de demostración descritas en el manual de instalación (Apéndice 11.3.5) y no deben emplearse en producción.

7.4.1. Autenticación

La Figura 10 muestra la pantalla de acceso, punto de entrada de los roles administrador y docente (objetivo específico 1). Tras validar el correo y la contraseña, el sistema redirige al tablero que corresponde al rol de la cuenta; ante un intento fallido o una cuenta desactivada muestra un mensaje distinto sin revelar si el correo existe.

7.4.2. Vista Docente

Las Figuras 11 a 14 cubren el trabajo del profesor (objetivos específicos 2 y 3). El tablero (Figura 11) resume, para el periodo activo, los documentos pendientes y publicados; los formularios de carta temática y de requisitos de recuperación (Figuras 12 y 13) asignan el periodo activo automáticamente y permiten guardar como borrador o publicar; y el formulario de autoevaluación (Figura 14) se construye dinámicamente a partir de la definición hecha por el administrador.

7.4.3. Vista Administrador

Las Figuras 15 a 18 muestran las herramientas del administrador (objetivos específicos 1 y 4). El tablero concentra los indicadores globales y el cumplimiento por departamento (Figura 15); la gestión de profesores cubre el alta, la edición y el restablecimiento de contraseña (Figura 16); el constructor de formularios permite definir secciones ponderadas y preguntas (Figura 17); y el panel de reportes genera las descargas en XLSX (Figura 18).

7.4.4. Vista pública

Las Figuras 19 a 21 ilustran la consulta sin iniciar sesión. La portada ofrece los dos tipos de documento; la cascada periodo → programa → UEA (Figura 20) evita mostrar opciones

vacías, y la vista de detalle (Figura 21) está preparada para imprimirse a PDF desde el navegador, que es el sustituto adoptado para la exportación de documentos (Sección 8).

7.5. Documentación de la API

La API queda documentada automáticamente vía *drf-spectacular*. En `/api/docs/` se accede a la interfaz *Swagger UI* interactiva, y en `/api/schema/` al esquema OpenAPI en YAML descargable.

8. Análisis y discusión de resultados

Esta sección compara lo entregado con lo comprometido en la propuesta, discute las decisiones de diseño que se apartaron de aquella y examina las limitaciones del sistema actual.

8.1. Comparación entregado vs. propuesta

La Tabla 5 sintetiza el grado de cumplimiento de cada componente del sistema respecto de lo que se prometió en la propuesta.

Tabla 5. Grado de cumplimiento por componente respecto a la propuesta.

Componente	Cumplimiento	Observaciones
Módulo de autenticación	Completo con matices	Identidad por email en lugar de número económico; <i>rate limiting</i> sí; restablecimiento autoservicio no.
Cartas temáticas (CRUD)	Completo	Ciclo Borrador/Publicado, unicidad, permisos, snapshots de identidad.
Cartas temáticas (contenido estructurado)	Reemplazado	Los submodelos Tema/Subtema/Bibliografía/CriterioEvaluacion se reemplazaron por texto libre tras el rediseño de junio 2026.
Requisitos de recuperación (CRUD)	Completo	Mismo patrón que cartas.
Requisitos de recuperación (RequisitoItem)	Reemplazado	También aplanado en el rediseño.
Exportación a PDF/-DOCX	No entregado	Se implementó exportación a XLSX en el cliente para reportes; documentos usan <code>window.print()</code> del navegador.
Autoevaluación docente (formulario)	Ampliado	Se entregó un <i>form-builder</i> configurable con ocho tipos de pregunta y nueve incrementos (PR1–PR9).
Autoevaluación (scoring y observaciones)	Completo	<i>Scoring</i> ponderado por sección con niveles de desempeño automáticos.
Autoevaluación (PDF descargable por el docente)	No entregado	El resultado se muestra en pantalla; no hay descarga en PDF.
Autoevaluación (privacidad admin)	Ajustado	El admin ve porcentajes agregados y quién respondió; el detalle cualitativo queda para el propio profesor.

Componente	Cumplimiento	Observaciones
Reportes de cumplimiento	Completo	Dashboards por departamento y por licenciatura; exportación a XLSX.
Reportes en PDF	No entregado	Ver observación de exportación.
Base de datos PostgreSQL	Completo	20 migraciones, <i>constraints</i> declarativos, fixtures y CSV para catálogos.
Autenticación JWT + validación de contraseñas + CORS lista blanca	Completo	Ver Sección 6.12.
Manual de usuario y de instalación	Entregados	Integrados como subsecciones nativas del Apéndice 11 (véanse Subsecciones 11.3 y 11.4).
Vista pública de exploración	Extra (no comprometido)	Cascada periodo→programa→UEA.
Catálogos administrables completos	Extra (no comprometido)	Departamentos, licenciaturas, posgrados, áreas, UEA (con CSV) y periodos.

8.2. Discusión de las brechas frente a la propuesta

8.2.1. Identidad por email en lugar de número económico

La propuesta indicaba autenticación con número económico + contraseña. En el sistema entregado, el `USERNAME_FIELD` es `email` y el número económico existe como campo opcional del perfil `Profesor`. La decisión se sostuvo por tres razones: (a) el email institucional es único, garantizado y conocido por todo el personal; (b) el número económico no siempre está disponible en el momento del alta; y (c) el email facilita la comunicación (envío de correos institucionales, futuras notificaciones y restablecimiento de contraseña). El impacto para el usuario es mínimo: en la práctica ingresa su email institucional en lugar de su número.

8.2.2. Ausencia de exportación a PDF/DOCX en backend

Ésta es la brecha de mayor visibilidad. La propuesta prometía generación de PDFs y DOCXs para cartas, requisitos, resultados de autoevaluación y reportes. En el sistema entregado, la única forma de *generar un PDF* de una carta o requisito es abrir la vista pública e imprimir a PDF desde el navegador (`window.print()`); la única exportación estructurada es la descarga a XLSX de los reportes, generada en el cliente con *SheetJS*. Las razones prácticas fueron: (a) evitar la dependencia de librerías pesadas (*WeasyPrint* requiere GTK; *ReportLab* requiere maquetación manual); (b) priorizar el flujo de operación diaria (captura, publicación, reportes en línea) sobre entregables descargables; y (c) concentrar el esfuerzo en el módulo de autoevaluación, técnicamente más complejo. En la Sección 9 se marca la generación nativa de PDFs como trabajo futuro prioritario.

8.2.3. Sin restablecimiento de contraseña autoservicio

Solo el administrador puede restablecer la contraseña de un profesor. Un flujo de recuperación de contraseña con token por correo se documenta como línea de trabajo futuro pero no se entregó porque el envío de correos requiere infraestructura adicional (SMTP institucional configurado) que no estuvo disponible en el trimestre de desarrollo.

8.3. Elementos entregados que no estaban en la propuesta

Estos componentes se incorporaron durante el desarrollo sin estar comprometidos en la propuesta, y elevan el valor del sistema:

- **Vista pública *Explorar*:** cascada periodo→programa→UEA para consulta de cartas y requisitos publicados. Beneficia a estudiantes y personal externo.
- **Form-builder de autoevaluación:** en lugar de un formulario fijo, el administrador diseña formularios con ocho tipos de pregunta, secciones ponderadas y versionado.
- **Catálogos administrables completos** con importación CSV.
- **Snapshots de identidad del profesor** en los documentos, para preservar autoría aunque el registro sea desactivado.
- **Suite de 197 pruebas automatizadas** en el backend.

8.4. Limitaciones del sistema actual

- **Sin pruebas en el frontend:** no hay *Vitest*, *Jest* ni *React Testing Library* configurados. Los flujos críticos se validan solo por inspección manual y por los tests de la API que consumen.
- **Sin CI/CD:** no hay pipeline configurado; las pruebas y *lints* se ejecutan manualmente antes de cada *commit*.
- **Sin backups automáticos configurados:** la política de respaldo dependerá de la infraestructura de despliegue.
- **Sin métricas de uso:** no se registra actividad de usuarios para análisis posterior (no hay tablero de uso ni registros estructurados).
- **Sin validación con usuarios ni medición de tiempos:** no se realizó una prueba piloto con profesores ni con personal de la Coordinación, por lo que el ahorro de tiempo y la reducción de errores que motivan el proyecto (Sección 3) son beneficios esperados, no medidos. Tampoco se aplicó un instrumento de usabilidad como la escala SUS.
- **Sin despliegue:** el sistema se ejecutó solo en entorno local; no se ha instalado en el servidor institucional.

- **Tokens en `localStorage`:** el cliente guarda los *JWT* en el almacenamiento local del navegador, lo que los expone a un ataque XSS [35]. La mitigación recomendada es emitirlos en *cookies* `HttpOnly`, lo que requeriría ajustar el *backend* y la política CORS.
- **Sin paginación en la vista pública ni en los reportes:** los *endpoints* públicos devuelven todas las filas; con volúmenes mucho mayores a los de una división sería necesario paginar.
- **Sin accesibilidad medida:** aunque *Tailwind* y los componentes reutilizables favorecen la accesibilidad, no se realizó una auditoría formal con *Lighthouse* o *axe-core*.

Estas limitaciones no impiden la operación del sistema pero delimitan expectativas y sugieren prioridades para iteraciones futuras.

9. Conclusiones

El proyecto cumplió con el objetivo general planteado en la propuesta: se desarrolló y verificó, en entorno local, un sistema web funcional que permite a los profesores de la División de Ciencias y Artes para el Diseño (CyAD) de la UAM-A gestionar cartas temáticas, requisitos de recuperación y responder autoevaluaciones, con acceso diferenciado por rol y reportes de cumplimiento para el personal administrativo. Los cuatro objetivos específicos se cumplieron sustancialmente, con matices que se detallan en la Sección 8 y que se sintetizan aquí.

9.1. Cumplimiento por objetivo

1. **Autenticación con roles:** cumplido. El sistema autentica mediante *JWT*, diferencia dos roles y aplica *throttling* sobre el *login*. El único matiz es que la identidad primaria del usuario es el *email* y no el número económico, decisión de diseño explicada en la Sección 8.
2. **CRUD de cartas y requisitos:** cumplido en cuanto a las operaciones y al ciclo Borrador/Publicado. La exportación a PDF y a DOCX —prometida en la propuesta— se sustituyó por la impresión nativa del navegador (`window.print()`) en la vista pública, sin generación de archivos binarios estructurados en el backend.
3. **Autoevaluación docente:** cumplido y ampliado. El sistema entregado va más allá de un formulario fijo: incorpora un *form-builder* configurable por el administrador, ocho tipos de pregunta, *scoring* ponderado por sección, niveles de desempeño automáticos y versionado de formularios que preserva la integridad histórica.
4. **Reportes de cumplimiento:** cumplido en cuanto a los indicadores. La exportación se implementó en formato *XLSX* en el cliente en lugar de *PDF* en el servidor.

9.2. Aprendizajes técnicos

Los principales aprendizajes derivados del desarrollo se enuncian a continuación.

- **Modelar antes de codificar.** El rediseño de junio de 2026 que aplanó los documentos (eliminación de los submodelos Tema, Subtema, Bibliografía, CriterioEvaluacion) enseñó la importancia de contar con la plantilla oficial *antes* de estructurar el esquema. Ante la ausencia de plantilla normativa, un contenido textual libre resultó más flexible y evitó imponer una estructura que la División no había validado.
- **Separar dominio en apps.** Django facilita la separación de dominios en apps independientes con sus propios modelos, vistas, urls y migraciones. Esta disciplina probó ser útil para acotar el impacto de los cambios y para orientar el sistema de pruebas.
- **JWT con *refresh* rotativo.** La combinación *access* corto + *refresh* rotativo mitiga bien el riesgo de filtración de tokens; el precio es que el cliente debe implementar la lógica de *refresh* transparente, lo cual se resolvió con un *interceptor* de *axios*.

- **Testing como red de seguridad.** Las 197 pruebas automatizadas del backend permitieron reestructurar con confianza el módulo de autoevaluación en nueve incrementos sucesivos (PR1–PR9), incluyendo cambios profundos como el paso a *scoring* ponderado y el versionado de formularios.
- **Estado del servidor con *TanStack Query*.** Usar una biblioteca dedicada al estado del servidor eliminó una capa entera de código de *fetching* manual y aportó funcionalidades no triviales: *cache*, revalidación, *polling* y *stale-while-revalidate*.

9.3. Trabajo futuro

El sistema entregado es funcional y suficiente para arrancar la operación real de la División, pero se identifican las siguientes líneas de trabajo futuro:

1. **Exportación nativa a PDF.** Incorporar *WeasyPrint* o *ReportLab* en el backend para generar PDFs firmables de cartas temáticas, requisitos de recuperación y resultados de autoevaluación. Esta funcionalidad se prometió en la propuesta y quedó pendiente.
2. **Reset de contraseña autoservicio.** Añadir un flujo de recuperación de contraseña con envío de token único al correo institucional, evitando que el administrador sea el único capaz de restablecer contraseñas.
3. **Integración con directorio institucional.** Si la División cuenta con un LDAP o SSO institucional, integrarlo evitaría mantener contraseñas independientes.
4. **Cobertura de pruebas del frontend.** El frontend actualmente carece de suite de pruebas; añadir Vitest con *React Testing Library* para cubrir al menos los flujos críticos (*login*, *guards* de rol, envío de autoevaluación) elevaría la confianza para futuros cambios.
5. **Estructuración opcional del contenido de cartas.** Si en el futuro la Coordinación de CyAD define una plantilla oficial de carta temática con submodelos (temario por unidad, bibliografía por tipo, criterios de evaluación tabulados), es posible reintroducir la jerarquía sin afectar los documentos existentes mediante una migración adicional.
6. **Notificaciones.** Enviar recordatorios automáticos por correo a profesores que no han publicado cartas o respondido su autoevaluación cerca del cierre del periodo.
7. **Prueba piloto con usuarios.** Medir con un grupo de profesores y personal de la Coordinación el tiempo de captura frente al proceso anterior y aplicar una escala de usabilidad (SUS), para sustentar con datos los beneficios que hoy solo se plantean.
8. **Almacenamiento seguro de tokens.** Migrar los *JWT* de *localStorage* a *cookies* *HttpOnly*.
9. **Despliegue en el servidor institucional `cyad.azc.uam.mx`.**

En conjunto, el sistema desarrollado sienta una base sólida para la modernización de la administración académica de la División de CyAD y demuestra que un equipo pequeño puede sustituir un flujo manual con *Google Forms* por una solución propia, mantenible y con espacio de crecimiento.

10. Referencias

- [1] H. P. Leyva López, M. G. Pérez Vera, and S. M. Pérez Vera, "Google forms en la evaluación diagnóstica como apoyo en las actividades docentes. caso con estudiantes de la licenciatura en turismo," *RIDE. Revista Iberoamericana para la Investigación y el Desarrollo Educativo*, vol. 9, no. 17, pp. 84–111, 2018.
- [2] J. M. A. Echeverría, "La autoevaluación docente de aula: un camino para mejorar la práctica educativa," *Revista Electrónica Diálogos Educativos. REDE*, vol. 11, no. 22, pp. 183–196, 2011.
- [3] A. Cuautle Vasquez, "Sistema para la configuración de formularios y captura de los resultados de la evaluación de atributos de egreso en un proceso de mejora continua," Proyecto terminal, División de Ciencias Básicas e Ingeniería, Universidad Autónoma Metropolitana Azcapotzalco, México, 2022.
- [4] E. A. Callisaya Gutierrez, "Sistema académico de gestión escolar trimestralizado," Tesis, Universidad Pública de El Alto, Bolivia, 2024. [Online]. Available: <https://repositorio.upea.bo/jspui/handle/123456789/1062>
- [5] E. M. Rojas, M. R. Ramírez, H. B. R. Moreno, M. d. C. S. Soto, N. d. C. O. Millán, and L. M. C. Suárez, "Sistema de gestión académica a través del desarrollo de modelo-vista-controlador," *RISTI – Revista Ibérica de Sistemas e Tecnologías de Informação*, 2019. [Online]. Available: <https://reunir.unir.net/handle/123456789/8290>
- [6] J. C. González *et al.*, "Sistema de gestión administrativa para la escuela secundaria n.º 44 "jorge luis borges"," Tesis de licenciatura, Facultad de Tecnología y Ciencias Aplicadas, Departamento de Informática, 2017.
- [7] M. G. P. Martínez, "La digitalización de las instituciones de educación superior en el salvador: Retos y oportunidades," *Perspectiva Científica*, vol. 1, no. 2, pp. 27–32, 2024.
- [8] G. E. Krasner and S. T. Pope, "A cookbook for using the model-view-controller user interface paradigm in Smalltalk-80," *Journal of Object-Oriented Programming*, vol. 1, no. 3, pp. 26–49, 1988.
- [9] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000. [Online]. Available: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [10] R. Fielding, M. Nottingham, and J. Reschke, "HTTP semantics," RFC 9110, Internet Engineering Task Force (IETF), 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9110>
- [11] WHATWG, "Fetch standard: CORS protocol," 2024. [Online]. Available: <https://fetch.spec.whatwg.org/>
- [12] Python Software Foundation, "Python 3 documentation," 2024. [Online]. Available: <https://docs.python.org/3/>

- [13] Django Software Foundation, "Django documentation," 2024. [Online]. Available: <https://docs.djangoproject.com/en/5.1/>
- [14] T. Christie and Django REST Framework contributors, "Django REST Framework documentation," 2024. [Online]. Available: <https://www.django-rest-framework.org/>
- [15] OpenAPI Initiative, "OpenAPI specification v3.1.0," 2021. [Online]. Available: <https://spec.openapis.org/oas/v3.1.0>
- [16] PostgreSQL Global Development Group, "PostgreSQL 17 documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/17/>
- [17] Meta Open Source, "React documentation," 2024. [Online]. Available: <https://react.dev/>
- [18] Vite contributors, "Vite: Next generation frontend tooling," 2024. [Online]. Available: <https://vitejs.dev/>
- [19] Tailwind Labs, "Tailwind CSS: A utility-first CSS framework," 2024. [Online]. Available: <https://tailwindcss.com/docs>
- [20] TanStack, "TanStack Query documentation," 2024. [Online]. Available: <https://tanstack.com/query/latest>
- [21] React Hook Form contributors, "React Hook Form documentation," 2024. [Online]. Available: <https://react-hook-form.com/>
- [22] Zod contributors, "Zod: TypeScript-first schema validation," 2024. [Online]. Available: <https://zod.dev/>
- [23] Remix Software, "React Router documentation," 2024. [Online]. Available: <https://reactrouter.com/>
- [24] Axios contributors, "Axios documentation," 2024. [Online]. Available: <https://axios-http.com/docs/intro>
- [25] SheetJS, "SheetJS community edition documentation," 2024. [Online]. Available: <https://docs.sheetjs.com/>
- [26] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [27] Django REST Framework Simple JWT contributors, "Simple JWT documentation: Blacklist app," 2024. [Online]. Available: https://django-rest-framework-simplejwt.readthedocs.io/en/latest/blacklist_app.html
- [28] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

- [29] OWASP Foundation, “OWASP Top 10: The ten most critical web application security risks,” 2021. [Online]. Available: <https://owasp.org/Top10/>
- [30] K. Moriarty, B. Kaliski, and A. Rusch, “PKCS #5: Password-based cryptography specification version 2.1,” RFC 8018, Internet Engineering Task Force (IETF), 2017. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8018>
- [31] R. Likert, “A technique for the measurement of attitudes,” *Archives of Psychology*, vol. 22, no. 140, pp. 1–55, 1932.
- [32] I. Sommerville, *Software Engineering*, 9th ed. Boston: Pearson, 2011.
- [33] Conventional Commits, “Conventional commits 1.0.0,” 2020. [Online]. Available: <https://www.conventionalcommits.org/en/v1.0.0/>
- [34] H. Krekel and pytest-dev team, “pytest documentation,” 2024. [Online]. Available: <https://docs.pytest.org/>
- [35] OWASP Foundation, “HTML5 security cheat sheet: Web storage,” 2024. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html

11. Apéndices

11.1. Código fuente

Este apéndice presenta *fragmentos curados* del código fuente que ilustran las decisiones de diseño más importantes. El código fuente completo (backend, frontend, tests, scripts y fixtures) se entrega en el archivo `codigo_fuente.zip` de la carpeta de entrega, generado con `git archive HEAD`.

11.1.1. Estructura del repositorio

```
proyecto_terminal_cyad/
|-- backend/
|   |-- config/           # settings, urls, wsgi
|   |-- core/             # utilidades: pagination, exceptions
|   |-- accounts/         # Usuario, Profesor, JWT
|   |-- catalogos/        # Departamento, Licenciatura, UEA, Periodo
|   |-- documentos/       # CartaTematica, RequisitoRecuperacion
|   |-- autoevaluacion/   # Formulario, Pregunta, Respuesta, Nivel
|   |-- reportes/         # Dashboards y cumplimiento
|   |-- scripts/          # seed_users, seed_demo, create_db
|   |-- tests/            # 197 pruebas con pytest
|   |-- manage.py
|   |-- requirements.txt
|-- frontend/
|   |-- src/
|   |   |-- api/          # clientes HTTP (auth, documentos, ...)
```

```

| | | |-- auth/           # AuthContext, ProtectedRoute, RoleRoute
| | | |-- components/ui/  # Button, Alert, Badge, Table, Modal
| | | |-- layouts/       # AdminLayout, ProfesorLayout
| | | |-- pages/
| | | | |-- admin/       # dashboards y CRUD por catalogo
| | | | |-- profesor/    # cartas, requisitos, autoevaluacion
| | | | `-- publico/     # vista Explorar y documentos publicados
| | | |-- App.jsx        # rutas + guards
| | | `-- main.jsx       # bootstrap React + TanStack Query
| |-- index.html
| |-- package.json
| `-- vite.config.js
|-- docs/
| `-- reporte/           # este documento (incluye los manuales)
`-- README.md

```

11.1.2. Backend — Modelo de Usuario con roles

Fragmento de backend/accounts/models.py: modelo de usuario personalizado con dos roles.

```

1 class Usuario(AbstractBaseUser, PermissionsMixin):
2     class Rol(models.TextChoices):
3         ADMIN      = "ADMIN",      "Administrador"
4         PROFESOR   = "PROFESOR",   "Profesor"
5
6     email          = models.EmailField(unique=True)
7     nombre         = models.CharField(max_length=150)
8     rol            = models.CharField(max_length=10, choices=Rol.choices)
9     is_active      = models.BooleanField(default=True)
10    is_staff        = models.BooleanField(default=False)
11
12    USERNAME_FIELD = "email"
13    REQUIRED_FIELDS = ["nombre"]
14
15    objects = UserManager()

```

11.1.3. Backend — Perfil de Profesor y Periodo

Fragmento de backend/accounts/models.py y backend/catalogos/models.py: el perfil Profesor sincroniza Usuario.is_active con Profesor.estado para que desactivar el perfil bloquee el inicio de sesión, y el modelo Periodo garantiza en save() la invariante “a lo sumo un periodo activo por recurso”.

```

1 # backend/accounts/models.py
2 class Profesor(TimeStampedModel, EstadoActivoModel):
3     usuario = models.OneToOneField(
4         Usuario, on_delete=models.CASCADE, related_name="perfil_profesor",
5         limit_choices_to={"rol": Usuario.Rol.PROFESOR},
6     )
7     numero_economico = models.CharField(max_length=20, unique=True,
8                                         null=True, blank=True)

```

```

9     nombre_completo      = models.CharField(max_length=255)
10    correo_institucional = models.EmailField(unique=True)
11    departamento         = models.ForeignKey(
12        "catalogos.Departamento", on_delete=models.SET_NULL,
13        null=True, blank=True, related_name="profesores",
14    )
15
16    def save(self, *args, **kwargs):
17        # Sincroniza el login del Usuario con el estado del perfil.
18        super().save(*args, **kwargs)
19        if self.usuario_id and self.usuario.is_active != bool(self.estado)
    ↪ :
20            self.usuario.is_active = bool(self.estado)
21            self.usuario.save(update_fields=["is_active"])
22
23
24    # backend/catalogos/models.py
25    class Periodo(TimeStampedModel):
26        class Recurso(models.TextChoices):
27            CARTAS          = "CARTAS",          "Cartas Temáticas"
28            REQUISITOS      = "REQUISITOS",      "Requisitos de Recuperación"
29            AUTOEVALUACION = "AUTOEVALUACION", "Autoevaluación"
30
31        clave              = models.CharField(max_length=10, unique=True)
32        fecha_inicio       = models.DateField()
33        fecha_fin          = models.DateField()
34        activo_cartas      = models.BooleanField(default=False)
35        activo_requisitos  = models.BooleanField(default=False)
36        activo_autoevaluacion = models.BooleanField(default=False)
37        activo = models.BooleanField(default=False, editable=False) # OR de
    ↪ los tres
38        estado = models.BooleanField(default=True)
39
40        @classmethod
41        def get_activo(cls, recurso, hoy=None):
42            field = {"CARTAS": "activo_cartas",
43                    "REQUISITOS": "activo_requisitos",
44                    "AUTOEVALUACION": "activo_autoevaluacion"}.get(recurso)
45            return cls.objects.filter(**{field: True, "estado": True}).first()
    ↪ if field else None
46
47        def save(self, *args, **kwargs):
48            # Invariante: a lo sumo un Periodo con cada flag activo.
49            per_resource = ("activo_cartas", "activo_requisitos", "
    ↪ activo_autoevaluacion")
50            for field in per_resource:
51                if getattr(self, field):
52                    Periodo.objects.exclude(pk=self.pk) \
53                        .filter(**{field: True}).update(**{field: False})
54            self.activo = any(getattr(self, f) for f in per_resource)
55            super().save(*args, **kwargs)

```

11.1.4. Backend — JWT y throttling

Fragmento de backend/config/settings.py.

```

1 REST_FRAMEWORK = {
2     "DEFAULT_AUTHENTICATION_CLASSES": (
3         "rest_framework_simplejwt.authentication.JWTAuthentication",
4     ),
5     "DEFAULT_PERMISSION_CLASSES": (
6         "rest_framework.permissions.IsAuthenticated",
7     ),
8     "DEFAULT_THROTTLE_CLASSES": (
9         "rest_framework.throttling.ScopedRateThrottle",
10    ),
11    "DEFAULT_THROTTLE_RATES": {
12        "login": "5/min",
13    },
14    "PAGE_SIZE": 20,
15 }
16
17 SIMPLE_JWT = {
18     "ACCESS_TOKEN_LIFETIME": timedelta(minutes=60),
19     "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
20     "ROTATE_REFRESH_TOKENS": True,
21     "BLACKLIST_AFTER_ROTATION": True, # requiere la app token_blacklist
22     "AUTH_HEADER_TYPES": ("Bearer",),
23 }

```

11.1.5. Backend — Documento académico base

Fragmento de backend/documentos/models.py. Clase abstracta con campos comunes y *snapshot* de identidad del profesor.

```

1 class DocumentoAcademicoBase(TimeStampedModel):
2     class Estado(models.TextChoices):
3         BORRADOR = "BORRADOR", "Borrador"
4         PUBLICADO = "PUBLICADO", "Publicado"
5
6     profesor = models.ForeignKey(
7         Profesor, on_delete=models.SET_NULL, null=True, blank=True
8     )
9     profesor_nombre = models.CharField(max_length=200, blank=True)
10    profesor_correo = models.EmailField(blank=True)
11
12    uea = models.ForeignKey(UEA, on_delete=models.PROTECT)
13    periodo = models.ForeignKey(Periodo, on_delete=models.PROTECT)
14    nombre_grupo = models.CharField(max_length=50)
15    id_grupo = models.CharField(max_length=50)
16    horario = models.CharField(max_length=200, blank=True)
17    modalidad = models.CharField(
18        max_length=15,
19        choices=[("PRESENCIAL", "Presencial"), ("REMOTO", "Remoto"), ("MIXTO",
20        ↪ "Mixto")],
21    )

```

```

21     estado      = models.CharField(
22         max_length=15,
23         choices=Estado.choices,
24         default=Estado.BORRADOR,
25     )
26
27     class Meta:
28         abstract = True

```

11.1.6. Backend — Modelos del *form-builder*

Fragmento de backend/autoevaluacion/models.py: cinco modelos que definen la estructura editable de un formulario. El ciclo de vida de Formulario (publicar/cerrar/reabrir/despublicar/publicar_revision) se detalla en el listado “Backend — Ciclo de vida de Formulario”.

```

1  class Formulario(TimeStampedModel):
2      class Estado(models.TextChoices):
3          BORRADOR = "BORRADOR", "Borrador"
4          PUBLICADO = "PUBLICADO", "Publicado"
5          CERRADO = "CERRADO", "Cerrado"
6
7      titulo      = models.CharField(max_length=255)
8      descripcion = models.TextField(blank=True)
9      periodo     = models.ForeignKey("catalogos.Periodo",
10                                     on_delete=models.PROTECT,
11                                     related_name="formularios")
12      estado      = models.CharField(max_length=10, choices=Estado.choices,
13                                     default=Estado.BORRADOR)
14      version     = models.PositiveIntegerField(default=1) # +1 en
15      ↪ publicar_revision
16      una_respuesta_por_profesor = models.BooleanField(default=True)
17      created_by  = models.ForeignKey("accounts.Usuario",
18                                     on_delete=models.SET_NULL,
19                                     null=True, blank=True, related_name="
20                                     ↪ +")
21      published_at = models.DateTimeField(null=True, blank=True)
22      closed_at    = models.DateTimeField(null=True, blank=True)
23
24      class Seccion(models.Model):
25          formulario = models.ForeignKey(Formulario, on_delete=models.CASCADE,
26                                       related_name="secciones")
27          titulo     = models.CharField(max_length=255)
28          descripcion = models.TextField(blank=True)
29          orden      = models.PositiveSmallIntegerField(default=0)
30          # La suma de pesos de las secciones de un formulario debe ser 100 % (
31          ↪ ±0.01)
32          # para poder publicarlo.
33          peso       = models.DecimalField(max_digits=5, decimal_places=2,
34          ↪ default=0)

```

```

34 class Pregunta(models.Model):
35     class Tipo(models.TextChoices):
36         TEXTO_CORTO = "TEXTO_CORTO", "Texto corto"
37         TEXTO_LARGO = "TEXTO_LARGO", "Texto largo"
38         OPCION_UNICA = "OPCION_UNICA", "Opción única"
39         CASILLAS = "CASILLAS", "Casillas de verificación"
40     ↪ "
41     SI_NO = "SI_NO", "Sí / No"
42     ESCALA_LINEAL = "ESCALA_LINEAL", "Escala lineal"
43     LISTA_DESPLEGABLE = "LISTA_DESPLEGABLE", "Lista desplegable"
44     CUADRICULA = "CUADRICULA", "Cuadrícula de opciones"
45
46     # Estas se ignoran en el scoring (respuesta cualitativa abierta).
47     TIPOS_NO_PUNTABLES = {"TEXTO_CORTO", "TEXTO_LARGO"}
48
49     formulario = models.ForeignKey(Formulario, on_delete=models.CASCADE,
50                                   related_name="preguntas")
51     seccion = models.ForeignKey(Seccion, on_delete=models.SET_NULL,
52                                null=True, blank=True, related_name="
53                                ↪ preguntas")
54     tipo = models.CharField(max_length=20, choices=Tipo.choices)
55     texto = models.TextField()
56     ayuda = models.CharField(max_length=500, blank=True)
57     obligatoria = models.BooleanField(default=True)
58     orden = models.PositiveSmallIntegerField(default=0)
59     # JSON dependiente del tipo:
60     #     ESCALA_LINEAL -> {min, max, label_min, label_max, puntos_factor}
61     #     SI_NO -> {puntos_si, puntos_no}
62     config = models.JSONField(default=dict, blank=True)
63
64 class OpcionPregunta(models.Model):
65     pregunta = models.ForeignKey(Pregunta, on_delete=models.CASCADE,
66                                 related_name="opciones")
67     texto = models.CharField(max_length=255)
68     valor = models.CharField(max_length=100, blank=True)
69     puntos = models.DecimalField(max_digits=5, decimal_places=2, default
70                                ↪ =0)
71     orden = models.PositiveSmallIntegerField(default=0)
72
73 class FilaCuadricula(models.Model):
74     # Las filas de una CUADRICULA. Sus columnas reutilizan OpcionPregunta.
75     pregunta = models.ForeignKey(Pregunta, on_delete=models.CASCADE,
76                                 related_name="filas")
77     texto = models.CharField(max_length=255)
78     orden = models.PositiveSmallIntegerField(default=0)

```

11.1.7. Backend — Captura de respuestas y baremo

Fragmento de backend/autoevaluacion/models.py: modelos que persisten cada envío del profesor y el baremo (NivelDesempeno) contra el que se evalúa. El al-

goritmo que rellena `puntaje_obtenido`, `porcentaje`, `nivel_desempeno` y crea un `RespuestaSeccion` por sección se muestra en el listado “Backend — Scoring ponderado de autoevaluación”. El campo `RespuestaSeccion.peso` es un *snapshot* del peso vigente al enviar, precisamente para que el desglose histórico permanezca estable si el administrador reajusta pesos y publica una revisión.

```

1 class NivelDesempeno(models.Model):
2     formulario      = models.ForeignKey(Formulario, on_delete=models.
    ↳ CASCADE,
3                                     related_name="niveles")
4     nombre          = models.CharField(max_length=100)
5     porcentaje_min  = models.DecimalField(max_digits=5, decimal_places=2)
6     porcentaje_max  = models.DecimalField(max_digits=5, decimal_places=2)
7     observacion     = models.TextField()
8     # Clave para la UI: green | blue | yellow | red | gray
9     color           = models.CharField(max_length=20, default="gray")
10    orden            = models.PositiveSmallIntegerField(default=0)
11
12
13 class Respuesta(TimeStampedModel):
14     class Estado(models.TextChoices):
15         BORRADOR = "BORRADOR", "Borrador"
16         ENVIADO  = "ENVIADO",  "Enviado"
17
18     formulario      = models.ForeignKey(Formulario, on_delete=models.
    ↳ PROTECT,
19                                     related_name="respuestas")
20     profesor        = models.ForeignKey("accounts.Profesor",
21                                     on_delete=models.PROTECT,
22                                     related_name="
    ↳ respuestas_formulario")
23     # Copia de Formulario.version al iniciar; permite volver a responder
    ↳ tras
24     # publicar_revision sin violar la unicidad compuesta de abajo.
25     version_formulario = models.PositiveIntegerField(default=1)
26     estado           = models.CharField(max_length=10, choices=Estado.
    ↳ choices,
27                                     default=Estado.BORRADOR)
28     enviado_at       = models.DateTimeField(null=True, blank=True)
29     puntaje_obtenido  = models.DecimalField(max_digits=7, decimal_places
    ↳ =2,
30                                     null=True, blank=True)
31     puntaje_maximo    = models.DecimalField(max_digits=7, decimal_places
    ↳ =2,
32                                     null=True, blank=True)
33     porcentaje        = models.DecimalField(max_digits=5, decimal_places
    ↳ =2,
34                                     null=True, blank=True)
35     nivel_desempeno  = models.ForeignKey(NivelDesempeno,
36                                     on_delete=models.SET_NULL,
37                                     null=True, blank=True,
38                                     related_name="respuestas")
39

```

```

40     class Meta:
41         constraints = [
42             models.UniqueConstraint(
43                 fields=["formulario", "profesor", "version_formulario"],
44                 name="unique_respuesta_formulario_profesor_version",
45             )
46         ]
47
48
49 class RespuestaPregunta(models.Model):
50     respuesta = models.ForeignKey(Respuesta,
51                                   on_delete=models.CASCADE,
52                                   related_name="items")
53     pregunta = models.ForeignKey(Pregunta,
54                                   on_delete=models.PROTECT,
55                                   related_name="
↳ respuestas_recibidas")
56     valor_texto = models.TextField(blank=True)
57     opciones_seleccionadas = models.ManyToManyField(OpcionPregunta, blank=
↳ True,
58                                                       related_name="
↳ selecciones")
59
60     class Meta:
61         unique_together = [["respuesta", "pregunta"]]
62
63
64 class RespuestaSeccion(models.Model):
65     # Snapshot por sección al enviar: preserva el peso vigente para el
↳ desglose.
66     respuesta = models.ForeignKey(Respuesta, on_delete=models.
↳ CASCADE,
67                                   related_name="secciones_resultado
↳ ")
68     seccion = models.ForeignKey(Seccion, on_delete=models.PROTECT
↳ ,
69                                   related_name="resultados")
70     peso = models.DecimalField(max_digits=5, decimal_places=2)
71     puntaje_obtenido = models.DecimalField(max_digits=7, decimal_places=2)
72     puntaje_maximo = models.DecimalField(max_digits=7, decimal_places=2)
73     porcentaje = models.DecimalField(max_digits=5, decimal_places=2,
74                                       default=0)
75
76     class Meta:
77         unique_together = [["respuesta", "seccion"]]

```

11.1.8. Backend — Cálculo ponderado del puntaje de autoevaluación

Fragmento simplificado de backend/autoevaluacion/views.py: cálculo del puntaje ponderado por sección, con manejo por tipo de pregunta y *snapshot* del peso en RespuestaSeccion.

```

1 def _calcular_puntaje(respuesta):

```

```

2 resultado_secciones = []
3 item_by_pregunta = {r.pregunta_id: r for r in respuesta.items.all()}
4 for seccion in respuesta.formulario.secciones.all():
5     sec_obt, sec_max = Decimal("0"), Decimal("0")
6     for pregunta in seccion.preguntas.all():
7         if pregunta.tipo in Pregunta.TIPOS_NO_PUNTABLES:
8             continue
9         item = item_by_pregunta.get(pregunta.id)
10        if pregunta.tipo == Pregunta.Tipo.CUADRICULA:
11            opts, filas = list(pregunta.opciones.all()), list(pregunta
↪ .filas.all())
12            if opts and filas:
13                sec_max += max(o.puntos for o in opts) * len(filas)
14            if item:
15                sec_obt += sum(c.opcion.puntos for c in item.celdas.
↪ all())
16        elif pregunta.tipo == Pregunta.Tipo.ESCALA_LINEAL:
17            cfg = pregunta.config or {}
18            factor = Decimal(str(cfg.get("puntos_factor", 1)))
19            sec_max += Decimal(str(cfg.get("max", 5))) * factor
20            if item and item.valor_texto:
21                sec_obt += Decimal(item.valor_texto) * factor
22            # ... SI_NO, CASILLAS, OPCION_UNICA / LISTA_DESPLEGABLE ...
23            porcentaje = (sec_obt / sec_max * 100).quantize(Decimal("0.01")) \
24                if sec_max > 0 else Decimal("0")
25            resultado_secciones.append({
26                "seccion": seccion, "obtenido": sec_obt, "maximo": sec_max,
27                "porcentaje": porcentaje, "peso": seccion.peso,
28            })
29
30            porcentaje_global = sum(
31                s["porcentaje"] * s["peso"] / Decimal("100") for s in
↪ resultado_secciones
32            ).quantize(Decimal("0.01"))
33
34            with transaction.atomic():
35                respuesta.porcentaje = porcentaje_global
36                respuesta.nivel_desempeno = NivelDesempeno.objects.filter(
37                    formulario=respuesta.formulario,
38                    porcentaje_min__lte=porcentaje_global,
39                    porcentaje_max__gte=porcentaje_global,
40                ).first()
41                respuesta.save()
42                for s in resultado_secciones:
43                    RespuestaSeccion.objects.update_or_create(
44                        respuesta=respuesta, seccion=s["seccion"],
45                        defaults={
46                            "puntaje_obtenido": s["obtenido"],
47                            "puntaje_maximo": s["maximo"],
48                            "porcentaje": s["porcentaje"],
49                            "peso": s["peso"], # snapshot para
↪ desglose historico
50                        },
51                    )

```

11.1.9. Backend — Permisos por rol

Fragmento del sistema de permisos que garantiza que cada profesor solo edite sus documentos.

```
1 class IsOwnerProfesorOrAdminReadOnly(permissions.BasePermission):
2     def has_permission(self, request, view):
3         return request.user.is_authenticated
4
5     def has_object_permission(self, request, view, obj):
6         if request.user.rol == Usuario.Rol.ADMIN:
7             return request.method in permissions.SAFE_METHODS
8         return obj.profesor and obj.profesor.usuario_id == request.user.id
```

11.1.10. Frontend — Cliente HTTP con *refresh* automático de JWT

Fragmento de frontend/src/api/client.js.

```
1 const client = axios.create({
2   baseURL: import.meta.env.VITE_API_URL,
3 });
4
5 client.interceptors.request.use((config) => {
6   const token = sessionStorage.getAccessToken();
7   if (token) config.headers.Authorization = `Bearer ${token}`;
8   return config;
9 });
10
11 let refreshing = null;
12
13 client.interceptors.response.use(
14   (r) => r,
15   async (error) => {
16     const { config, response } = error;
17     if (response?.status === 401 && !config.__retried) {
18       config.__retried = true;
19       refreshing = refreshing ?? refreshAccessToken();
20       const newToken = await refreshing.finally(() => (refreshing = null))
21       ↪ ;
22       if (newToken) {
23         config.headers.Authorization = `Bearer ${newToken}`;
24         return client(config);
25       }
26       return Promise.reject(error);
27     }
28   });
```

11.1.11. Frontend — Guard de rutas por rol

Fragmento de frontend/src/auth/RoleRoute.jsx.

```

1 export function RoleRoute({ role, children }) {
2   const { user, isLoading } = useAuth();
3   if (isLoading) return <Loading />;
4   if (!user) return <Navigate to="/login" replace />;
5   if (user.rol !== role) return <Navigate to="/" replace />;
6   return children;
7 }
8
9 export const AdminRoute = ({ children }) =>
10  <RoleRoute role="ADMIN" >{children}</RoleRoute>;
11 export const ProfesorRoute = ({ children }) =>
12  <RoleRoute role="PROFESOR" >{children}</RoleRoute>;

```

11.1.12. Frontend — Renderizador dinámico de preguntas

Fragmento de frontend/src/pages/profesor/autoevaluacion/AutoevaluacionFormPage.jsx que despacha por tipo de pregunta.

```

1 function PreguntaRenderer({ pregunta, value, onChange }) {
2   switch (pregunta.tipo) {
3     case "TEXTO_CORTO":      return <TextInput  {...{value, onChange}}
4     ↪ />;
5     case "TEXTO_LARGO":     return <TextArea   {...{value, onChange}}
6     ↪ />;
7     case "OPCION_UNICA":    return <RadioGroup opciones={pregunta.
8     ↪ opciones} {...{value, onChange}} />;
9     case "CASILLAS":        return <CheckGroup opciones={pregunta.
10    ↪ opciones} {...{value, onChange}} />;
11    case "SI_NO":            return <YesNo      {...{value, onChange}}
12    ↪ />;
13    case "ESCALA_LINEAL":    return <LinearScale config={pregunta.config}
14    ↪ {...{value, onChange}} />;
15    case "LISTA_DESPLEGABLE": return <Select    opciones={pregunta.
16    ↪ opciones} {...{value, onChange}} />;
17    case "CUADRICULA":       return <Grid       config={pregunta.config}
18    ↪ filas={pregunta.filas} {...{value, onChange}} />;
19    default:                 return <UnknownType tipo={pregunta.tipo} />;
20  }
21 }

```

11.1.13. Backend — Comando de importación CSV para UEA

Comando de gestión python manage.py cargar_catalogos_csv para carga masiva idempotente. Fragmento simplificado.

```

1 class Command(BaseCommand):
2     help = "Carga catalogos (Areas, Licenciaturas, Posgrados, UEAs) desde
3     ↪ CSV"
4
5

```

```

4     def add_arguments(self, parser):
5         parser.add_argument("--ueas", type=str, help="Path al CSV de UEAs"
↪ )
6
7     def handle(self, *args, **opts):
8         ruta = opts["ueas"]
9         with open(ruta, newline="", encoding="utf-8") as fh:
10            for fila in csv.DictReader(fh):
11                UEA.objects.update_or_create(
12                    clave=fila["clave"],
13                    licenciatura=Licenciatura.objects.get(clave=fila["
↪ licenciatura"]),
14                    defaults={
15                        "nombre": fila["nombre"],
16                        "trimestre": int(fila["trimestre"]),
17                        "creditos": int(fila["creditos"]),
18                        "tipo": fila["tipo"],
19                    },
20                )

```

11.1.14. Backend — Constraint XOR y unicidad parcial de UEA

Fragmento de backend/catalogos/models.py: reglas de integridad declarativas que fuerzan que una UEA pertenezca a licenciatura o a posgrado (nunca a ambos ni a ninguno), y que la clave sea única dentro de cada programa.

```

1 class Meta:
2     constraints = [
3         CheckConstraint(
4             condition=(
5                 (Q(licenciatura__isnull=False) & Q(posgrado__isnull=True))
6                 | (Q(licenciatura__isnull=True) & Q(posgrado__isnull=
↪ False))
7             ),
8             name="uea_xor_licenciatura_posgrado",
9         ),
10        UniqueConstraint(
11            fields=["clave", "licenciatura"],
12            condition=Q(licenciatura__isnull=False),
13            name="uea_clave_unica_por_licenciatura",
14        ),
15        UniqueConstraint(
16            fields=["clave", "posgrado"],
17            condition=Q(posgrado__isnull=False),
18            name="uea_clave_unica_por_posgrado",
19        ),
20    ]

```

11.1.15. Backend — Ciclo de vida de Formulario

Fragmento de backend/autoevaluacion/models.py: validación de estructura y transiciones de la máquina de estados BORRADOR ↔ PUBLICADO ↔ CERRADO.

```
1 def _validar_estructura_para_publicar(self):
2     secciones = list(self.secciones.all())
3     if not secciones:
4         raise ValueError("Debe definir al menos una seccion antes de
↪ publicar.")
5     suma = sum(s.peso for s in secciones)
6     if abs(suma - Decimal("100")) > Decimal("0.01"):
7         raise ValueError(f"La suma de pesos debe ser 100% (actual: {suma
↪ }%).")
8     if self.preguntas.filter(seccion__isnull=True).exists():
9         raise ValueError("Hay preguntas sin seccion asignada.")
10
11 def publicar(self):
12     # BORRADOR -> PUBLICADO
13     if self.estado != self.Estado.BORRADOR:
14         raise ValueError("Solo se puede publicar un formulario en estado
↪ BORRADOR.")
15     self._validar_estructura_para_publicar()
16     # Si ya hay respuestas de la version vigente (se edito tras
↪ contestarse),
17     # la nueva publicacion es una version nueva.
18     if self.respuestas.filter(version_formulario=self.version).exists():
19         self.version += 1
20     self.estado = self.Estado.PUBLICADO
21     self.published_at = timezone.now()
22     self.save()
23
24 def cerrar(self):
25     # PUBLICADO -> CERRADO
26     if self.estado != self.Estado.PUBLICADO:
27         raise ValueError("Solo se puede cerrar un formulario PUBLICADO.")
28     self.estado = self.Estado.CERRADO
29     self.closed_at = timezone.now()
30     self.save()
31
32 def reabrir(self):
33     # CERRADO -> PUBLICADO (misma
↪ version)
34     if self.estado != self.Estado.CERRADO:
35         raise ValueError("Solo se puede reabrir un formulario CERRADO.")
36     self.estado = self.Estado.PUBLICADO
37     self.closed_at = None
38     self.save()
39
40 def despublicar(self):
41     # PUBLICADO/CERRADO -> BORRADOR
42     if self.estado not in (self.Estado.PUBLICADO, self.Estado.CERRADO):
43         raise ValueError("Solo se puede despublicar un formulario
↪ PUBLICADO o CERRADO.")
44     self.estado = self.Estado.BORRADOR # sin cambiar version ni borrar
↪ respuestas
45     self.save()
46
47 def publicar_revision(self):
48     # CERRADO -> PUBLICADO (version
↪ + 1)
49     if self.estado != self.Estado.CERRADO:
50         raise ValueError("Solo se puede publicar una revision de un
```

```

    ↪ formulario CERRADO.")
46     self._validar_estructura_para_publicar()
47     self.version += 1
48     self.estado = self.Estado.PUBLICADO
49     self.published_at = timezone.now()
50     self.save()

```

11.1.16. Backend — Login con distinción de cuenta desactivada

Fragmento de backend/accounts/serializers.py. El *serializer* personalizado distingue *credenciales inválidas* de *cuenta desactivada* devolviendo un código `account_disabled` sin filtrar la existencia del correo.

```

1  class CustomTokenObtainPairSerializer(TokenObtainPairSerializer):
2      def validate(self, attrs):
3          email = (attrs.get(self.username_field) or "").strip().lower()
4          password = attrs.get("password") or ""
5          usuario = Usuario.objects.filter(email__iexact=email).first()
6          if usuario and usuario.check_password(password) and not usuario.
    ↪ is_active:
7              raise AuthenticationFailed(
8                  {"code": "account_disabled",
9                   "detail": "Tu cuenta se encuentra desactivada. "
10                      "Contacta al administrador para reactivarla."},
11                  code="account_disabled",
12              )
13          data = super().validate(attrs) # credenciales OK, usuario activo
14          u = self.user
15          data["user"] = {"id": u.id, "email": u.email,
16                          "nombre": u.nombre, "rol": u.rol}
17          return data

```

11.1.17. Frontend — Generación de XLSX por periodo

Fragmento simplificado de frontend/src/pages/admin/reportes/Reportes Page.jsx: descarga en el navegador con *SheetJS*, agrupando las filas por periodo en hojas separadas del mismo libro.

```

1  async function descargarDocumento(publicPath, fetcher) {
2      const params = { estado: "PUBLICADO", page_size: 500 }
3      if (periodoId) params.periodo = periodoId
4      if (deptoId) params.profesor__departamento = deptoId
5
6      const res = await fetcher(params)
7      const rows = res.data?.results ?? res.data ?? []
8
9      const porPeriodo = new Map()
10     for (const row of rows) {
11         const clave = row.periodo_clave ?? "Sin periodo"
12         if (!porPeriodo.has(clave)) porPeriodo.set(clave, [])
13         porPeriodo.get(clave).push(row)
14     }

```

```

15
16  const wb = XLSX.utils.book_new()
17  if (porPeriodo.size === 0) {
18    XLSX.utils.book_append_sheet(wb, XLSX.utils.json_to_sheet([]), "Sin
    ↪ datos")
19  } else {
20    for (const [clave, groupRows] of porPeriodo) {
21      const ws = XLSX.utils.json_to_sheet(
22        groupRows.map((r) => rowToExcel(r, publicPath))
23      )
24      XLSX.utils.book_append_sheet(wb, ws, sheetName(clave))
25    }
26  }
27  XLSX.writeFile(wb, `${publicPath}-${Date.now()}.xlsx`)
28 }

```

11.1.18. Nota sobre el listado completo

Los archivos aquí incluidos son una selección representativa de los fragmentos más ilustrativos. El código fuente completo (backend, frontend, *tests*, scripts y fixtures) se entrega en el archivo `codigo_fuente.zip`. Para regenerarlo desde el repositorio se ejecuta desde la raíz del proyecto:

```
git archive HEAD -o docs/reporte/entregables/codigo_fuente.zip
```

Adicionalmente, la documentación viva de la API se genera automáticamente vía *drf-spectacular* y se sirve en `/api/docs/` (interfaz Swagger UI) y `/api/schema/` (esquema OpenAPI en YAML).

11.2. Entregables del plan de trabajo

La Tabla 6 replica el calendario de actividades comprometido en la propuesta (198 horas en total para el Trimestre 2026-Primavera) con el estado real de cada entregable al cierre del proyecto.

Tabla 6. Entregables prometidos en la propuesta y su estado real.

No.	Actividad	Hrs.	Estado y evidencia
1	Diseño e implementación de la base de datos	20	Completo. 20 migraciones, seis apps, PostgreSQL 17. Diagrama ER en Figura 3.
2	Diseño de las interfaces por módulo	10	Completo. UI implementada en React 19 + Tailwind, con componentes reutilizables (Button, Alert, Badge, Table, Modal, FormField, etc). Capturas en Sección 7.

No.	Actividad	Hrs.	Estado y evidencia
3	Desarrollo del módulo de autenticación	25	Completo. <code>accounts/</code> + 25 pruebas (<code>tests/test_auth.py</code>). Login JWT con <i>throttling</i> .
4	Desarrollo del módulo de cartas temáticas	25	Completo. <code>documentos/</code> (parte <code>CartaTematica</code>) + tests en <code>tests/test_documentos.py</code> . Sin exportación PDF/DOCX (véase Sección 8).
5	Desarrollo del módulo de requisitos de recuperación	25	Completo. <code>documentos/</code> (parte <code>RequisitoRecuperacion</code>). Mismo alcance/limitaciones que cartas.
6	Desarrollo del módulo de autoevaluación docente	25	Ampliado. <code>autoevaluacion/</code> con <i>form-builder</i> , 8 tipos de pregunta, <i>scoring</i> ponderado y versionado (9 incrementos PR1–PR9). 81 pruebas.
7	Desarrollo del módulo de generación de reportes	25	Completo. <code>reportes/</code> + 25 pruebas. Export XLSX en cliente (no PDF).
8	Integración del sistema con la base de datos	10	Completo. ORM operativo, <i>fixtures</i> , comando <code>cargar_catalogos_csv</code> y <i>scripts</i> <code>seed_users</code> / <code>seed_demo</code> .
9	Prueba y corrección de errores	8	Completo. 197 pruebas automatizadas + <i>smoke test</i> de extremo a extremo. Historial git con <i>fixes</i> rastreables (por ejemplo <i>fix(seguridad)</i> , <i>fix(autoevaluacion)</i> , <i>fix(catalogos)</i>).
10	Documentación para usuarios	10	Completo. Contenido integrado en las Subsecciones 11.3 (<i>Manual de instalación</i>) y 11.4 (<i>Manual de usuario</i>) de este mismo apéndice.
11	Redacción del reporte final del proyecto	15	Completo. Este documento (<code>docs/reporte/main.pdf</code>) con once secciones numeradas, un apéndice con cuatro subsecciones, índices y bibliografía.
<i>Entregables adicionales no contemplados originalmente:</i>			
E1	Vista pública de exploración (cascada periodo→programa→UEA)	—	Entregado (extra). Beneficia a estudiantes al elegir carga.

No.	Actividad	Hrs.	Estado y evidencia
E2	Catálogos administrables completos + importación CSV	—	Entregado (extra). Departamentos, licenciaturas, posgrados, áreas, UEA, periodos.
E3	Documentación OpenAPI/Swagger auto-generada	—	Entregado (extra). Vía <i>drf-spectacular</i> en <code>/api/schema/</code> y <code>/api/docs/</code> .

Los manuales de instalación y de usuario forman parte de este mismo apéndice como subsecciones hermanas: la Subsección 11.3 contiene el *Manual de instalación* y la Subsección 11.4 el *Manual de usuario*.

11.3. Manual de instalación

11.3.1. Introducción

Este manual describe cómo instalar y ejecutar el *Sistema para la gestión de formularios sobre los cursos y autoevaluaciones docentes* de la División de Ciencias y Artes para el Diseño (CyAD) en un entorno de desarrollo local. La instalación en producción sigue los mismos pasos, cambiando únicamente el archivo `.env` y la forma de servir la aplicación (*unicorn* + *Nginx*, ver Subsubsección 11.3.9).

11.3.2. Prerrequisitos

Software	Versión mínima	Uso
Python	3.11+ (probado con 3.13)	Ejecución del backend Django.
Node.js	20+ (probado con 22 LTS)	Ejecución del frontend Vite + React.
PostgreSQL	14+ (probado con 17)	Base de datos relacional.
Git	2.40+	Clonado del repositorio.
<i>npm</i> (incluido con Node)	10+	Gestor de paquetes JS.

En Windows se recomienda utilizar *Git Bash* o *PowerShell*. En Linux/macOS los comandos son idénticos, cambiando la ruta del entorno virtual (`bin/` en lugar de `Scripts/`).

Nota importante para Windows: el instalador oficial de PostgreSQL no agrega `psql` al `PATH` del sistema; esto no impide la instalación, que en esta guía usa el *script* `scripts/create_db.py` en lugar de `psql` (ver la Solución de problemas al final de este manual si necesita invocar `psql` directamente).

Requisitos mínimos de hardware

Las siguientes cifras se derivan de mediciones sobre un entorno con `seed_demo` cargado, ejecutando `backend/scripts/medir_hardware.py` (descrito al final de este apartado): el proceso de Django consume aproximadamente 120 MB de RAM en reposo, PostgreSQL alrededor de 80 MB con la carga de demostración, y el *build* del frontend (`npm run build`) alcanza un pico de ~250 MB durante la compilación (no en tiempo de ejecución).

Escenario	RAM mínima	RAM recomendada	Disco
Desarrollo local (una sola persona)	4 GB	8 GB	5 GB libres
Servidor pequeño (< 50 profesores, un solo periodo activo)	2 GB	4 GB	10 GB libres
Servidor CyAD completo (> 300 profesores, histórico multi-periodo)	4 GB	8 GB	20 GB libres

Sobre CPU: cualquier procesador x86-64 de doble núcleo publicado en la última década es suficiente para el runtime; el cuello de botella es la memoria durante `npm run build`, que solo ocurre al desplegar una nueva versión, no en operación diaria. En un servidor dedicado bastan *dos vCPU*; para desarrollo local la máquina que edita el código suele tener recursos de sobra.

Ancho de banda: al ser una aplicación de tipo formulario, el consumo por sesión es del orden de decenas de KB por interacción. Cualquier enlace institucional (> 10 Mbps) resulta holgado.

Reproducibilidad de las cifras

Las tres cifras anteriores (Django en reposo, PostgreSQL con demo, pico de `npm run build`) se pueden reproducir sobre cualquier host ejecutando el *script* versionado `backend/scripts/medir_hardware.py`. El *script*:

- `arranca manage.py runserver --noreload` en un subprocesso, sirve una petición a `/api/schema/` para forzar la inicialización de la app y muestrea el RSS del proceso con `psutil`;
- enumera todos los procesos `postgres` visibles al usuario y suma su RSS (refleja la carga vigente de la base; ideal medirlo después de correr `python scripts/seed_demo.py`);
- ejecuta `npm run build` en `frontend/` y muestrea el RSS del árbol de procesos cada 200 ms hasta que termina, guardando el máximo.

Al finalizar imprime una tabla legible en la consola y deja el detalle en `backend/scripts/medir_hardware.out.json`, con la marca de tiempo y el hardware del host (`platform.uname()`, RAM total, vCPU). Requiere `psutil` (incluido en `backend/requirements.txt`) y que `frontend/node_modules/` exista (correr `npm install` antes).

11.3.3. Clonado del repositorio

```
git clone --depth 1 https://github.com/Aldebaran-GR/proyecto_terminal_cyad.git
↪ .
```

11.3.4. Instalación mínima sin datos de demo

Esta es la ruta recomendada para un despliegue institucional real. Deja el sistema con los catálogos oficiales de CyAD (departamentos, licenciaturas, posgrados, áreas y UEAs) y una única cuenta de administrador; no crea profesores de prueba, formularios ficticios ni respuestas simuladas. Es la única ruta apropiada para producción; el flujo con datos de demostración se documenta después (Subsubsección 11.3.5) y no debe ejecutarse en un ambiente productivo.

1. Entorno virtual y dependencias del backend

```
cd backend
python -m venv .venv
.venv\Scripts\Activate.ps1      # Windows (PowerShell)
# source .venv/bin/activate     # Linux / macOS (en lugar de la línea anterior)
pip install -r requirements.txt
```

Windows bloquea por defecto la ejecución de *scripts* `.ps1`. Si al activar aparece un error de *execution policy*, o el símbolo de la terminal no cambia a `(.venv)`, ejecute lo siguiente una sola vez en esa terminal (no persiste; solo aplica a la sesión actual) y repita la activación:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.venv\Scripts\Activate.ps1
```

Si `pip install -r requirements.txt` falla, confirme que el `.venv` esté activo y que la versión de Python sea 3.11 o superior.

Para descartar dudas sobre qué intérprete quedó activo (opcional, y obligatorio si aparece el incidente descrito abajo), ejecute:

```
python -c "import sys; print(sys.executable)"
```

La salida debe apuntar a `...\proyecto_terminal_cyad\backend\.venv\Scripts\python.exe` en Windows (o `.../backend/.venv/bin/python` en Linux/macOS). Si apunta a otro Python (por ejemplo `C:\Python313\` o `...\AppData\Local\Programs\Python\...`), el `.venv` no está activo de verdad.

Incidente frecuente: `ModuleNotFoundError: No module named 'decouple'` al ejecutar `scripts/create_db.py` o `manage.py`, pese a que `pip install -r requirements.txt` no reportó errores. Esto ocurre cuando esa instalación se realizó contra el Python global en lugar del Python del `.venv` —típicamente porque la activación falló silenciosamente (símbolo de la terminal sin el prefijo `(.venv)`, o una terminal nueva donde no se volvió a activar) y el comando `pip` del `PATH` apuntaba a otro intérprete. El diagnóstico es la misma verificación de arriba (`python -c import sys; print(sys.executable)`); si confirma que el intérprete activo no es el del `.venv`, la forma de corregirlo —inmune al estado de activación y al orden del `PATH`— es invocar el ejecutable del `.venv` por su ruta completa, tanto para instalar como para ejecutar cualquier *script* posterior:

```
.venv\Scripts\python.exe -m pip install -r requirements.txt      # Windows
.venv/bin/python -m pip install -r requirements.txt             # Linux/macOS
```

Verifique con `.venv\Scripts\python.exe -c import decouple; print('OK')` (o su equivalente en `bin/python`) antes de continuar con el resto de la instalación.

2. Archivo `.env` del backend

Ubicación: el archivo `.env` que Django lee debe estar en la carpeta `backend/` (donde vive `manage.py`), *no* en la raíz del repositorio. El repositorio incluye `backend/.env.example` como plantilla; cópielo y ajuste los valores:

```
cp .env.example .env      # desde backend/
```

Contenido esperado (valores por defecto de la plantilla; ajuste `DB_PASSWORD` y `SECRET_KEY` a su entorno):

```
1 SECRET_KEY=cambia-esta-clave-por-una-aleatoria-larga
2 DEBUG=True
3 ALLOWED_HOSTS=localhost,127.0.0.1
4 DB_NAME=proyecto_terminal_cyad
5 DB_USER=postgres
6 DB_PASSWORD=tu_password_de_postgres
7 DB_HOST=localhost
8 DB_PORT=5432
9 JWT_ACCESS_MINUTES=60
10 JWT_REFRESH_DAYS=7
11 CORS_ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173,http://
    ↪ localhost:5174
```

Para producción se debe establecer `DEBUG=False`, definir `ALLOWED_HOSTS` con el dominio real y generar una `SECRET_KEY` nueva (comando en la Subsubsección 11.3.9).

3. Crear la base de datos vacía

El proyecto utiliza por defecto el usuario `postgres` y una base de datos llamada `proyecto_terminal_cyad` (ver `backend/.env.example`). La forma recomendada es dejar que el script auxiliar cree la base automáticamente:

```
python scripts/create_db.py
```

4. Aplicar migraciones

```
python manage.py migrate
```

5. Crear la primera cuenta de administrador

Con el comando interactivo de Django:

```
python manage.py createsuperuser
```

Se pedirán correo, nombre y contraseña. Este usuario tendrá rol ADMIN y podrá crear el resto de cuentas desde la interfaz web.

Alternativa no interactiva (útil para automatizar el aprovisionamiento), definiendo antes las variables de entorno que `createsuperuser` espera:

```
$env:DJANGO_SUPERUSER_EMAIL="admin@azc.uam.mx"           # PowerShell
$env:DJANGO_SUPERUSER_NOMBRE="Administrador CyAD"
$env:DJANGO_SUPERUSER_PASSWORD="<password-fuerte>"
python manage.py createsuperuser --noinput
```

En *Git Bash*, Linux o macOS use `export VAR=valor` en vez de la sintaxis de PowerShell.

6. Cargar los catálogos base (fixtures JSON versionados)

Departamentos, licenciaturas, posgrados y áreas están precargados en fixtures JSON dentro del repositorio, y se aplican con el comando `loaddata` de Django. Ejecutar en el orden que dictan las claves foráneas:

```
python manage.py loaddata catalogos/fixtures/departamentos.json
python manage.py loaddata catalogos/fixtures/licenciaturas.json
python manage.py loaddata catalogos/fixtures/posgrados.json
python manage.py loaddata catalogos/fixtures/areas.json
```

Los fixtures son idempotentes (usan clave primaria explícita): se pueden re-ejecutar sin duplicar registros.

7. Cargar las UEAs oficiales

Los CSV oficiales de coordinación académica (`ueas_licenciatura.csv` con las UEAs de DCG, ARQ, DI y DiPS; `ueas_posgrado.csv` con las UEAs de PPCDA, PDB y PDEU) viven versionados en `backend/catalogos/fixtures/`. Se cargan con el comando de gestión `cargar_catalogos_csv`:

```
python manage.py cargar_catalogos_csv --ueas-csv catalogos/fixtures/
↳ ueas_licenciatura.csv
python manage.py cargar_catalogos_csv --ueas-csv catalogos/fixtures/
↳ ueas_posgrado.csv
```

El comando valida claves duplicadas, reporta filas rechazadas y resuelve el área de cada UEA contra los pares (nombre, descripción) de `areas.json`; las áreas que no encuentre las lista como *warning* y la UEA queda con `area = NULL` (nunca se crea

un área nueva de forma silenciosa). La carga es idempotente por el `update_or_create` interno.

Si coordinación entrega en el futuro un CSV incremental (por ejemplo cuando cambie un plan de estudios), se carga con el mismo comando apuntando `--ueas-csv` a la nueva ruta; el efecto es un *upsert* por (*clave*, *programa*).

8. Verificación con pytest (antes de levantar servidores)

```
pytest # desde backend/
```

9. Arrancar el servidor de desarrollo del backend

```
python manage.py runserver 8000
```

El backend queda disponible en `http://localhost:8000`. La documentación interactiva de la API está en `http://localhost:8000/api/docs/`.

10. Crear el formulario de autoevaluación institucional

Con el backend corriendo (paso 9) y el frontend levantado (Subsubsección 11.3.6), iniciar sesión con el superuser del paso 5 y, desde Autoevaluación → Nuevo formulario, construir el formulario del periodo vigente (secciones, preguntas, opciones puntuables y niveles de desempeño). Publicarlo cuando la suma de pesos de las secciones alcance 100 %.

Los usuarios de profesor se crean posteriormente desde Profesores → Nuevo profesor en la interfaz de administrador; en cada alta el administrador captura el correo y una contraseña inicial, que entrega personalmente al docente.

11.3.5. Datos de demostración (opcional)

Advertencia: este bloque es exclusivo del *desarrollo local* y de la evaluación en escritorio; *no debe ejecutarse en un ambiente productivo*. Crea un profesor de prueba, un formulario ya publicado y respuestas ficticias que después será tedioso eliminar.

Bootstrap unificado (Windows PowerShell)

Como atajo, el repositorio incluye `scripts/bootstrap.ps1` que orquesta *creación de BD, migraciones y siembra de datos de demo* en un único comando. Debe ejecutarse desde la raíz del repositorio, con el `.venv` ya activo y `backend/.env` configurado:

```
.\scripts\bootstrap.ps1 # Desde proyecto_terminal_cyad/
```

El script imprime tres pasos numerados ([1/3] crear BD, [2/3] migrar, [3/3] seed) y al finalizar recuerda cómo arrancar el backend y el frontend en dos terminales distintas. Falla temprano si falta `backend/.env` o si el árbol del repositorio no está completo.

Seed manual (Linux, macOS, o alternativa a Bootstrap)

Con la base de datos ya creada y las migraciones aplicadas (pasos 3 y 4 de la Subsubsección 11.3.4), ejecute:

```
python scripts/seed_demo.py
```

El *seed* es idempotente y crea, en un solo comando, catálogos y UEAs *de muestra* (4 departamentos, 4 licenciaturas, 1 posgrado, 2 áreas y 5 UEAs — 4 de licenciatura y 1 de posgrado, para validar la relación excluyente licenciatura/posgrado de UEA — sin invocar `loaddata` ni `cargar_catalogos_csv`, por lo que no deja cargado el catálogo oficial de 511 UEAs de la Subsubsección 11.3.4), usuarios de prueba, un periodo activo (26-I) y el formulario de autoevaluación. Al finalizar, existirán las siguientes cuentas:

Rol	Correo	Contraseña
Administrador	admin@cyad.uam.mx	Admin1234!
Profesor	profesor@cyad.uam.mx	Profesor1234!
Profesor (segundo)	profesor2@cyad.uam.mx	Profesor1234!

Estas contraseñas deben cambiarse antes del despliegue a producción (y, de hecho, la base entera debería recrearse limpia siguiendo la Subsubsección 11.3.4).

11.3.6. Configuración del frontend

En una terminal aparte:

```
cd frontend
cp .env.example .env
npm install
npm run dev
```

El repositorio incluye `frontend/.env.example` ya con `VITE_API_URL=http://localhost:8000/api/v1`, que coincide con el valor por defecto de `frontend/src/api/client.js`; para desarrollo local con el backend en el puerto 8000 basta copiarlo tal cual.

El frontend queda disponible en `http://localhost:5173`. Si el puerto está ocupado, Vite usa automáticamente el siguiente libre e imprime la URL real en consola. Vite realiza *hot reload* automático al editar archivos fuente.

Si el backend no está en `localhost:8000`, ajuste `VITE_API_URL` en `frontend/.env` (o cree `frontend/.env.local`, que Vite prioriza) con:

```
VITE_API_URL=http://tu-servidor:8000/api/v1
```

11.3.7. Verificación

1. Abra `http://localhost:5173/` — debe mostrarse la portada pública, con accesos a “Explorar” por tipo de documento.

2. Inicie sesión con el superuser creado en el paso 5 de la Subsubsección 11.3.4 (o, si sembró la demo, con `admin@cyad.uam.mx / Admin1234!`). Debe redirigir al *dashboard* de administrador.
3. Si sembró la demo, cierre sesión e inicie con `profesor@cyad.uam.mx / Profesor1234!`. Debe redirigir al *dashboard* de profesor.
4. *Opcional, solo si sembró la demo de la Subsubsección 11.3.5:* con el backend corriendo, ejecute `python scripts/smoke_test.py` desde `backend/`. Verifica de extremo a extremo, vía HTTP, el *login* de `admin` y `profesor`, `/auth/me`, catálogos, formularios disponibles, creación de respuesta, creación de carta temática y los reportes de administrador; no aplica a la instalación limpia porque depende de las cuentas de prueba del seed.

11.3.8. Comandos administrativos útiles

Comando	Uso
<code>python manage.py cargar_catalogos_csv --ueas-csv <archivo></code>	Importar UEAs adicionales desde CSV.
<code>python manage.py createsuperuser</code>	Crear cuenta administradora adicional.
<code>python manage.py dumpdata <app></code>	Exportar datos a JSON.
<code>python scripts/medir_hardware.py</code>	Medir RAM de Django, PostgreSQL y pico de <code>npm run build</code> .
<code>pytest --cov=. --cov-report=html</code>	Ejecutar pruebas con reporte de cobertura (<code>pytest-cov</code> , incluido en <code>requirements.txt</code>).
<code>npm run build (frontend)</code>	Compilar producción estática (<code>dist/</code>).

11.3.9. Despliegue en servidor de producción

Esta subsubsección describe el despliegue en un servidor Linux (Ubuntu 22.04 LTS o Debian 12) que atienda las peticiones institucionales. El manual asume que ya se cumplen los prerrequisitos (Subsubsección 11.3.4 inclusive: base limpia, superuser creado, catálogos cargados).

Topología recomendada

Internet (HTTPS 443)

|

```

      v
+-----+
|      Nginx      | Sirve / -> frontend/dist  (SPA estático)
| (proxy inverso + | /api/      -> gunicorn (socket UNIX)
|  archivos estát.) | /static/ -> backend/staticfiles
+-----+
|
|      v (unix:/run/cyad/cyad.sock)
+-----+
|  gunicorn (3 wk) | Django / DRF
|  config.wsgi:app  |
+-----+
|
|      v (TCP 5432)
+-----+
|  PostgreSQL 17   | Usuario dedicado cyad_prod
+-----+

```

El Django *admin* (/admin/) no se expone públicamente: el SPA usa esa misma ruta (/admin/*) para su propio panel de administrador (ver `frontend/src/App.jsx`), así que Nginx no debe redirigirla hacia gunicorn. Si se necesita el admin de Django en producción para depuración puntual, accédase por túnel SSH a `localhost:8000` en el servidor, no vía Nginx.

Todos los servicios corren en la misma máquina en el caso más simple; se pueden separar la BD y el *app server* sin cambios en el código, ajustando `DB_HOST` en el `.env`.

Variables de entorno de producción

El archivo `backend/.env` para producción difiere del de desarrollo en varios campos clave:

```

1 DEBUG=False
2 SECRET_KEY=<clave-larga-generada-abajo>
3 ALLOWED_HOSTS=cyad.uam.mx
4 CORS_ALLOWED_ORIGINS=https://cyad.uam.mx
5 DB_NAME=cyad_prod
6 DB_USER=cyad_prod
7 DB_PASSWORD=<password-fuerte>
8 DB_HOST=localhost
9 DB_PORT=5432
10 JWT_ACCESS_MINUTES=15
11 JWT_REFRESH_DAYS=1

```

Generación de `SECRET_KEY`:

```

python -c "from django.core.management.utils import get_random_secret_key;
↪ print(get_random_secret_key())"

```

La ventana de `JWT_ACCESS_MINUTES=15` y `JWT_REFRESH_DAYS=1` reduce el impacto de un token filtrado; el frontend refresca el *access token* automáticamente en segundo plano mientras el *refresh* siga vigente.

El archivo `.env` debe tener permisos restrictivos:

```
chmod 600 backend/.env
chown www-data:www-data backend/.env
```

Base de datos en producción

Crear un rol *no-superuser* dedicado y una base propiedad de ese rol:

```
sudo -u postgres psql
CREATE USER cyad_prod WITH PASSWORD '<password-fuerte>';
CREATE DATABASE cyad_prod OWNER cyad_prod;
\q
```

Endurecer `pg_hba.conf` para permitir solo conexiones locales autenticadas por contraseña al usuario `cyad_prod` (no permita accesos remotos al usuario `postgres`).

Respaldo diario recomendado vía *cron*. Cree primero el directorio de respaldos (`postgres` debe poder escribir en él) y luego la tarea:

```
sudo install -d -o postgres -g postgres /var/backups/cyad
```

```
1 # /etc/cron.d/cyad-backup
2 0 3 * * * postgres pg_dump -Fc cyad_prod > /var/backups/cyad/cyad_$(date
   ↪ +\ %Y\ %m\ %d) .dump
```

Rotar respaldos con `logrotate` o un simple `find /var/backups/cyad -mtime +30 -delete`.

Backend con gunicorn + systemd

Instalar gunicorn en el mismo entorno virtual:

```
pip install gunicorn
```

Crear la unidad `/etc/systemd/system/cyad-backend.service`:

```
1 [Unit]
2 Description=CyAD Django backend (gunicorn)
3 After=network.target postgresql.service
4 Requires=postgresql.service
5
6 [Service]
7 Type=notify
8 User=www-data
9 Group=www-data
10 RuntimeDirectory=cyad
11 WorkingDirectory=/srv/cyad/proyecto_terminal_cyad/backend
12 Environment="PATH=/srv/cyad/proyecto_terminal_cyad/backend/.venv/bin"
13 ExecStart=/srv/cyad/proyecto_terminal_cyad/backend/.venv/bin/gunicorn \
14     config.wsgi:application \
15     --workers 3 \
16     --bind unix:/run/cyad/cyad.sock \
17     --access-logfile /var/log/cyad/access.log \
18     --error-logfile /var/log/cyad/error.log
19 Restart=on-failure
20
```

```
21 [Install]
22 WantedBy=multi-user.target
```

RuntimeDirectory=cyad hace que systemd cree /run/cyad/ (propiedad de www-data) antes de arrancar el servicio, para que gunicorn pueda crear ahí el *socket*; sin esta línea, www-data normalmente no tiene permiso de escritura directa en /run/.

Habilitar y arrancar:

```
sudo mkdir -p /var/log/cyad
sudo chown www-data:www-data /var/log/cyad
sudo systemctl daemon-reload
sudo systemctl enable --now cyad-backend
sudo systemctl status cyad-backend
```

Archivos estáticos del backend

El Django admin y drf-spectacular sirven CSS/JS estáticos que Nginx debe entregar directamente. Recolectarlos en backend/staticfiles/ (ruta ya definida como STATIC_ROOT en config/settings.py):

```
python manage.py collectstatic --noinput
```

Debe re-ejecutarse en cada actualización que añada estáticos nuevos.

Frontend en producción

Compilar los estáticos del SPA:

```
cd frontend
npm ci
npm run build
```

npm ci respeta package-lock.json exactamente y es más rápido que npm install para despliegues reproducibles. El resultado queda en frontend/dist/, que Nginx servirá como archivos estáticos.

Antes de compilar, crear frontend/.env.production con la URL pública de la API:

```
VITE_API_URL=https://cyad.uam.mx/api/v1
```

Nginx como reverse proxy

Crear /etc/nginx/sites-available/cyad.conf:

```
1 server {
2     listen 80;
3     server_name cyad.uam.mx;
4
5     # SPA: sirve archivos estáticos y hace fallback a index.html
6     root /srv/cyad/proyecto_terminal_cyad/frontend/dist;
7     index index.html;
8
9     location / {
```

```
10     try_files $uri $uri/ /index.html;
11 }
12
13 # Backend (API) via socket UNIX. NO se proxía /admin/: esa ruta
14 # la sirve el SPA (su propio panel de administrador); ver la nota
15 # sobre el Django admin al inicio de esta subsubsección.
16 location /api/ {
17     proxy_pass http://unix:/run/cyad/cyad.sock;
18     proxy_set_header Host $host;
19     proxy_set_header X-Real-IP $remote_addr;
20     proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
21     proxy_set_header X-Forwarded-Proto $scheme;
22 }
23
24 # Estáticos del backend (drf-spectacular)
25 location /static/ {
26     alias /srv/cyad/proyecto_terminal_cyad/backend/staticfiles/;
27     expires 30d;
28 }
29
30 client_max_body_size 10M;
31 }
```

Habilitar y recargar:

```
sudo ln -s /etc/nginx/sites-available/cyad.conf /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

HTTPS con Let's Encrypt

Con *certbot* instalado (`sudo apt install certbot python3-certbot-nginx`):

```
sudo certbot --nginx -d cyad.uam.mx
```

Certbot modifica el bloque de Nginx anterior para añadir escucha en :443 con el certificado emitido y una redirección 80 → 443. La renovación se instala automáticamente vía *systemd timer*; verificar con `sudo systemctl status certbot.timer`.

Actualizaciones (redeploy)

Cada nueva versión del código se despliega con:

```
cd /srv/cyad/proyecto_terminal_cyad
git pull
cd backend
source .venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py collectstatic --noinput
cd ../frontend
npm ci
npm run build
sudo systemctl restart cyad-backend
sudo systemctl reload nginx
```

Es recomendable hacer `pg_dump` antes de cada `migrate` en producción.

Nota para despliegues en Windows Server

Si el hosting institucional obliga a usar Windows Server, `gunicorn` no está disponible. La alternativa recomendada es `waitress`:

```
pip install waitress
waitress-serve --port=8000 config.wsgi:application
```

Como *reverse proxy* se usa IIS con el módulo *URL Rewrite + Application Request Routing*, o bien *Nginx for Windows* con una configuración equivalente a la mostrada arriba. El servicio se registra con `sc.exe create` o mediante *NSSM*. El resto de la configuración (`.env`, base de datos, `collectstatic`, `npm run build`) es idéntica.

11.3.10. Solución de problemas

- **Error FATAL: password authentication failed:** verifique `DB_USER` y `DB_PASSWORD` en `.env` y que el usuario tenga permisos sobre la base de datos.
- **Error CORS al iniciar sesión desde el frontend:** verifique que `CORS_ALLOWED_ORIGINS` incluya `http://localhost:5173`.
- **429 Too Many Requests al iniciar sesión:** se activó el *rate limiting* (5 intentos por minuto). Espere 60 segundos y reintente.
- **Puerto 5173 ocupado:** Vite usará automáticamente el siguiente puerto libre; verifique la URL impresa al inicio.
- **cargar_catalogos_csv reporta áreas no encontradas:** significa que el CSV trae un par (`area_nombre`, `area_descripcion`) que no está precargado en `areas.json`. Agregue el área faltante (ya sea editando `areas.json` y volviendo a hacer `loaddata`, o desde `/admin/catalogos/areas` en la UI) y vuelva a ejecutar el comando; las UEs afectadas se actualizarán con el área correcta.
- **psql: command not found en Windows:** agregue `C:\Program Files\PostgreSQL\<versión>\bin` al `PATH`, o llame a `psql` por ruta completa. Alternativamente, use *pgAdmin* o el script `scripts/create_db.py`, que utiliza el driver Python y no requiere `psql` en el `PATH`.
- **scripts/create_db.py responde 'utf-8' codec can't decode byte...**: es un error de decodificación, no el fallo real. El mensaje de *libpq* llega en la codificación regional de Windows (*cp1252*) y Python falla al intentar leerlo como *utf-8*. La causa típica detrás es autenticación fallida contra PostgreSQL: revise `DB_USER` y `DB_PASSWORD` en `backend/.env` y confirme que ese rol pueda conectarse a la base postgres (`psql -U <usuario> -d postgres`). El mismo síntoma aparece si el servicio de PostgreSQL no está corriendo.

11.4. Manual de usuario

11.4.1. Introducción

Este manual está dirigido a los tres tipos de usuario del sistema: el **visitante público** (cualquier persona con acceso a la URL institucional), el **profesor** y el **administrador de la División de CyAD**. Se recorre cada rol siguiendo el flujo natural de uso, con capturas de todas las pantallas involucradas.

Las capturas se tomaron con el sistema ejecutándose contra la base de datos de demostración (`python scripts/seed_demo.py`); los correos, nombres y contenidos mostrados son datos de ejemplo. En la interfaz pública, los requisitos de recuperación se presentan con el nombre “Evaluación de Recuperación”; ambos términos designan el mismo documento. La URL institucional final la comunicará la División de CyAD; en los ejemplos de este manual se utiliza el marcador `https://cyad.uam.mx`.

11.4.2. Uso — Vista pública (sin iniciar sesión)

La vista pública es la primera experiencia al abrir la URL del sistema y no requiere credenciales. Está pensada para que estudiantes, personal y visitantes puedan consultar cartas temáticas y requisitos de recuperación publicados por los profesores.

Página principal

Al ingresar a la URL raíz se muestra la portada con las dos opciones de consulta (Figura 22): *Carta Temática* y *Evaluación de Recuperación*. Un enlace en la esquina superior derecha (“Iniciar sesión”) permite pasar al formulario de acceso reservado a profesores y administradores.

Paso 1 — Elegir el tipo de documento y el periodo

Al hacer clic en “**Consultar**” sobre cualquiera de las dos tarjetas, el sistema abre la vista *Explorar* correspondiente y muestra el primer paso de la cascada: seleccionar el trimestre (Figura 23). Solo aparecen periodos que tienen al menos un documento publicado del tipo elegido, para no ofrecer opciones vacías.

Paso 2 — Elegir un programa académico

Al seleccionar el periodo aparecen los pasos 2 y 3: seleccionar una licenciatura o un posgrado, y luego expandir la UEA para ver los documentos publicados en ella (Figura 24). El árbol de UEA se agrupa por trimestre curricular; solo se listan las UEA con al menos un documento publicado del tipo elegido en ese periodo. Cada tarjeta interior corresponde a un grupo específico e incluye profesor, horario, modalidad y fecha de publicación.

El mismo flujo aplica a Evaluación de Recuperación (Figura 25); cambia el tipo de tarjeta y los textos, pero la cascada periodo → programa → UEA → documento es idéntica.

Detalle imprimible del documento

Al hacer clic en “*Ver documento*” se abre la vista pública del documento (Figura 26), con encabezado institucional, datos del grupo y campos de contenido. Un botón “**Imprimir**”

lanza el diálogo de impresión del navegador; el estilo de impresión oculta la barra superior y el pie para producir un PDF limpio.

Cada carta y cada requisito publicados tienen una URL propia y estable, apta para citarse en un plan de estudios, un correo o un temario:

- Cartas temáticas: <https://cyad.uam.mx/publico/cartas/<id>>
- Requisitos de recuperación: <https://cyad.uam.mx/publico/requisitos/<id>>

11.4.3. Acceso al sistema

Para iniciar sesión se ingresa a la URL del sistema y se utiliza el enlace “Iniciar sesión” de la esquina superior derecha; también es posible entrar directamente a la ruta `/login`. La Figura 27 muestra la pantalla de acceso.

Se ingresa el correo institucional (`usuario@uam.mx` o similar) y la contraseña proporcionada por el administrador. Tras iniciar sesión, el sistema redirige automáticamente al *dashboard* correspondiente según el rol de la cuenta (Administrador o Profesor).

Notas de seguridad:

- Se permiten hasta 5 intentos de inicio de sesión por minuto por dirección IP. Si se excede el límite, es necesario esperar un minuto.
- La sesión expira automáticamente por inactividad; el sistema renueva el *token* de acceso en segundo plano mientras haya actividad del usuario.
- Para restablecer una contraseña olvidada, es necesario contactar al administrador (véase Subsubsección 11.4.4).

Primer acceso. Cuando el administrador crea una cuenta de profesor, entrega personalmente al docente su correo institucional (que también funciona como *nombre de usuario*) y una contraseña inicial que el propio administrador define al crear la cuenta. En el primer inicio de sesión esa contraseña inicial se utiliza como definitiva. Esta versión del sistema no ofrece una pantalla propia de “cambiar mi contraseña” para el profesor: si desea que su contraseña personal sustituya a la temporal, deberá solicitar al administrador que la restablezca por una nueva de su elección.

11.4.4. Uso — rol Administrador

El rol Administrador tiene acceso al menú lateral izquierdo con *Dashboard*, *Profesores*, la sección *Catálogos* (Departamentos, Licenciaturas, Posgrados, Áreas, UEA, Periodos), la sección *Documentos* (Cartas Temáticas, Requisitos de Recuperación), la sección *Autoevaluación* y *Reportes*.

Dashboard

Tras iniciar sesión el sistema muestra el *dashboard* del administrador (Figura 28), con indicadores globales del sistema: total de profesores activos, número de cartas y requisitos publicados en el periodo activo, formularios de autoevaluación publicados, y tarjetas de cumplimiento por departamento para cada uno de los tres recursos.

Gestión de profesores

Desde el menú “**Profesores**” se accede a la lista de docentes registrados (Figura 29). La tabla muestra nombre, número económico, departamento, correo de acceso y estado (activo/inactivo). Un buscador filtra por nombre, correo o número económico. Cada fila tiene tres acciones: *Editar*, *Contraseña* y *Eliminar*.

Crear un profesor. El botón “+ **Nuevo Profesor**” abre una ventana modal con dos secciones: *Datos del profesor* (nombre, número económico y departamento) y *Cuenta de acceso* (correo institucional y contraseña temporal), como se muestra en la Figura 30. La contraseña se muestra en claro para que el administrador la copie y la entregue al profesor por el canal seguro que la División utilice.

Editar un profesor. El botón *Editar* en cada fila abre la ventana modal de edición (Figura 31) con los mismos campos que la creación más una casilla *Cuenta activa*, que permite deshabilitar temporalmente el acceso sin borrar la cuenta ni los documentos asociados.

Restablecer contraseña. El botón *Contraseña* abre la ventana modal de restablecimiento (Figura 32): el administrador escribe la nueva contraseña, la valida y se la entrega al docente. El campo se muestra en claro para que se pueda verificar la captura antes de guardar.

Catálogos institucionales

La sección **Catálogos** agrupa las entidades estables del sistema: departamentos, licenciaturas, posgrados, áreas, UEA y periodos. Cada catálogo sigue el mismo patrón: tabla + botón “+ Nuevo”. Al pulsar el botón se abre una ventana modal con los campos correspondientes.

UEA — carga individual y carga masiva. La sección de UEA es la que administra más registros (511 en el catálogo oficial cargado), por lo que ofrece dos flujos: alta individual y carga masiva por CSV. La Figura 37 muestra la ventana modal de alta individual (programa, clave, nombre, tipo, trimestre, créditos, área y liga), y la Figura 38 muestra la ventana modal de importación por CSV, con la descripción del formato aceptado y el selector de archivo.

Periodos y banderas por recurso

Desde “**Catálogos** → **Periodos**” se crean los trimestres del calendario académico (Figura 39) indicando clave, fecha de inicio, fecha de fin y las banderas de activación por recurso.

Cada periodo tiene tres banderas independientes, representadas como *chips* clicables en la tabla:

- **Cartas activas** — habilita a los profesores a capturar y publicar cartas temáticas para ese trimestre.
- **Requisitos activos** — habilita la captura de requisitos de recuperación para ese trimestre.
- **Autoevaluación activa** — habilita el formulario de autoevaluación docente para ese trimestre.

Un *chip* verde con círculo relleno (●) indica bandera activa; gris con círculo vacío (○), inactiva. Al hacer clic sobre un *chip* se conmuta la bandera, y el sistema garantiza

automáticamente que **solo un periodo por recurso** pueda tener cada bandera activa a la vez: al prender la bandera “Cartas” en un periodo, se apaga automáticamente en el periodo que la tenía antes. Esta regla evita que los profesores puedan capturar contra dos trimestres simultáneamente.

Un periodo que sigue activo para algún recurso no puede eliminarse. Para los demás, el diálogo de eliminación muestra una *vista previa* de los documentos y respuestas que se borrarían en cascada, y solo procede tras confirmarlo.

Constructor de formularios de autoevaluación

La sección **Autoevaluación** lista los formularios existentes con su estado (*Borrador* o *Publicado*), periodo, número de preguntas y número de respuestas recibidas (Figura 40). Un formulario en Borrador puede editarse libremente; uno Publicado se conserva versionado y sus respuestas quedan asociadas a esa versión concreta.

El botón “+ **Nuevo Formulario**” abre la ventana modal de creación (Figura 41), donde se define el título, una descripción opcional y el periodo destino. Un periodo solo aparece en el selector cuando aún no tiene un formulario asignado, para evitar conflictos de versionado.

Una vez creado el formulario, se accede al *constructor* — una vista dividida en tres pestañas: *Preguntas*, *Niveles de Desempeño* y *Estadísticas*.

Preguntas. La pestaña *Preguntas* (Figura 42) permite añadir secciones (con un peso porcentual que debe sumar 100 % entre todas) y, dentro de cada sección, preguntas. Se soportan ocho tipos de pregunta: texto corto, texto largo, opción única (radios), casillas (múltiple), sí/no, escala lineal, lista desplegable y cuadrícula. Las opciones de cada pregunta tienen puntajes individuales, que el sistema suma para calcular el resultado de la sección.

Regla de validación: la suma de los pesos de las secciones debe ser exactamente 100 %. El sistema no permite publicar un formulario que no cumpla esta regla.

Niveles de desempeño. La pestaña *Niveles de Desempeño* (Figura 43) define los rangos de puntaje que traducen el porcentaje obtenido en una etiqueta cualitativa (por ejemplo, *En desarrollo*, *Satisfactorio* y *Sobresaliente*). Cada nivel define un porcentaje mínimo y uno máximo, una observación y un color; si un porcentaje cae en el límite de dos rangos, se asigna el nivel de menor orden.

Estadísticas. La pestaña *Estadísticas* (Figura 44) resume las respuestas recibidas mientras el formulario está publicado: número de respuestas, promedio general, promedio por sección, distribución por nivel de desempeño y porcentajes por pregunta. Es una *fotografía* calculada al abrir la vista; para tener datos frescos hay que recargar.

Un formulario solo aparece ante los profesores cuando (a) se **publica** desde el constructor y (b) el periodo correspondiente tiene la bandera “Autoevaluación activa” encendida (véase Subsubsección 11.4.4).

Reportes

Desde el menú **Reportes** (Figura 45) el administrador puede descargar hojas de cálculo Excel con el estado del cumplimiento por tipo de documento y por periodo. El formulario permite elegir tipo de documento (Cartas, Requisitos, Autoevaluación), periodo (o *Todos*

los periodos) y opcionalmente el departamento del profesor.

Si se elige “Todos los periodos”, el archivo generado contiene una hoja por cada periodo con datos, para poder comparar trimestres desde el mismo documento. El nombre del archivo incluye el tipo de reporte y la fecha de exportación (por ejemplo, `cumplimiento_cartas_2026-08-23.xlsx`).

Momento de cálculo: los reportes se generan al momento de la descarga contra la base de datos actual, no a partir de un *job* diferido. Cualquier autoevaluación enviada, carta publicada o requisito capturado en los segundos previos se refleja en el archivo. Un XLSX descargado es una *fotografía* en ese instante; para tener datos frescos hay que volver a descargarlo.

11.4.5. Uso — rol Profesor

Al iniciar sesión como profesor se muestra el *dashboard* personal (Figura 46), con: tarjetas de resumen del periodo activo para cada uno de los tres recursos (Cartas Temáticas, Requisitos de Recuperación y Formularios de Autoevaluación pendientes), listas recientes agrupadas por periodo, y un menú lateral con acceso directo a los tres módulos.

Crear una Carta Temática

Desde “Cartas Temáticas → Nueva” se abre el formulario de creación (Figura 47). Los pasos son:

1. Buscar la UEA por clave o nombre y seleccionar una opción del autocompletado.
2. El sistema asigna automáticamente el periodo cuya bandera “Cartas” esté activa; el profesor no lo elige.
3. Completar los datos del grupo (nombre, ID, horario, modalidad).
4. Completar el contenido: descripción, objetivos, contenidos, evaluación, bibliografía. Todos los campos son de texto libre; los que se dejen vacíos no aparecerán en la vista pública.
5. Presionar **Guardar** para conservar como *borrador* o **Publicar** para que sea visible en la vista pública.

Una carta publicada puede ser *despublicada* y editada nuevamente; la URL pública se mantiene tras cada nueva publicación.

Crear una Evaluación de Recuperación

El flujo es análogo al de cartas temáticas. Desde “Requisitos de Rec. → Nuevo” se abre el formulario correspondiente (Figura 48), con búsqueda de UEA, asignación automática de periodo y campos específicos del documento (requisitos, lugar, fecha y recursos para la evaluación).

Contestar la autoevaluación docente

Cuando el administrador abre un periodo de autoevaluación, el formulario correspondiente aparece en la sección **Autoevaluación** del profesor. Al hacer clic en *Responder* se abre el formulario dinámico (Figura 49), renderizado a partir de las secciones, preguntas y opciones definidas por el administrador.

Se pueden guardar respuestas parciales (*en progreso*) y regresar más tarde. Al terminar se presiona **Enviar**; el sistema calcula automáticamente el puntaje ponderado y muestra el nivel de desempeño alcanzado. Una vez enviada, la autoevaluación no puede modificarse desde la interfaz.

11.4.6. Preguntas frecuentes

¿Qué pasa si publico una carta con errores? Puede despublicarse, corregirla y volver a publicarla; la URL pública se mantiene.

¿Puedo tener varias cartas para la misma UEA en el mismo trimestre? Sí, si son grupos distintos (identificados por *Nombre del grupo + ID del grupo*).

¿Se puede reabrir una autoevaluación ya enviada? No desde la interfaz: una vez enviada, la respuesta queda registrada. Si fuera necesario corregirla, el caso debe plantearse a los responsables técnicos del sistema.

¿Cómo se calcula el puntaje? Para cada sección se suman los puntos de las opciones elegidas y se divide entre el máximo posible, lo que da un porcentaje por sección; el porcentaje global es la suma de esos porcentajes multiplicados por el peso de cada sección (que en conjunto suman 100 %). El nivel de desempeño (por ejemplo, *En desarrollo*, *Satisfactorio* o *Sobresaliente*) se obtiene de rangos configurables por el administrador.

¿La vista pública muestra los datos personales del profesor? No. Muestra el nombre público (que el administrador puede editar), la UEA y el contenido académico. Datos como el correo o el número económico quedan privados.

¿Cómo exporto los reportes a Excel? Desde “Reportes”, se selecciona tipo, periodo y opcionalmente departamento, y se pulsa *Descargar Excel*. El archivo es compatible con Microsoft Excel y con LibreOffice Calc, y su nombre incluye la fecha de exportación.

¿Cómo activo la autoevaluación para un trimestre? Desde “Catálogos → Periodos”, se localiza el periodo en la tabla y se hace clic sobre el *chip* “Autoevaluación” hasta ponerlo en verde (●). Se apagará automáticamente en el periodo que la tuviera antes, garantizando que solo un trimestre esté activo a la vez.

¿Puedo cambiar mi propia contraseña como profesor? En esta versión del sistema no. Es necesario solicitar al administrador que la restablezca por una de su elección desde “Profesores → Restablecer contraseña”.

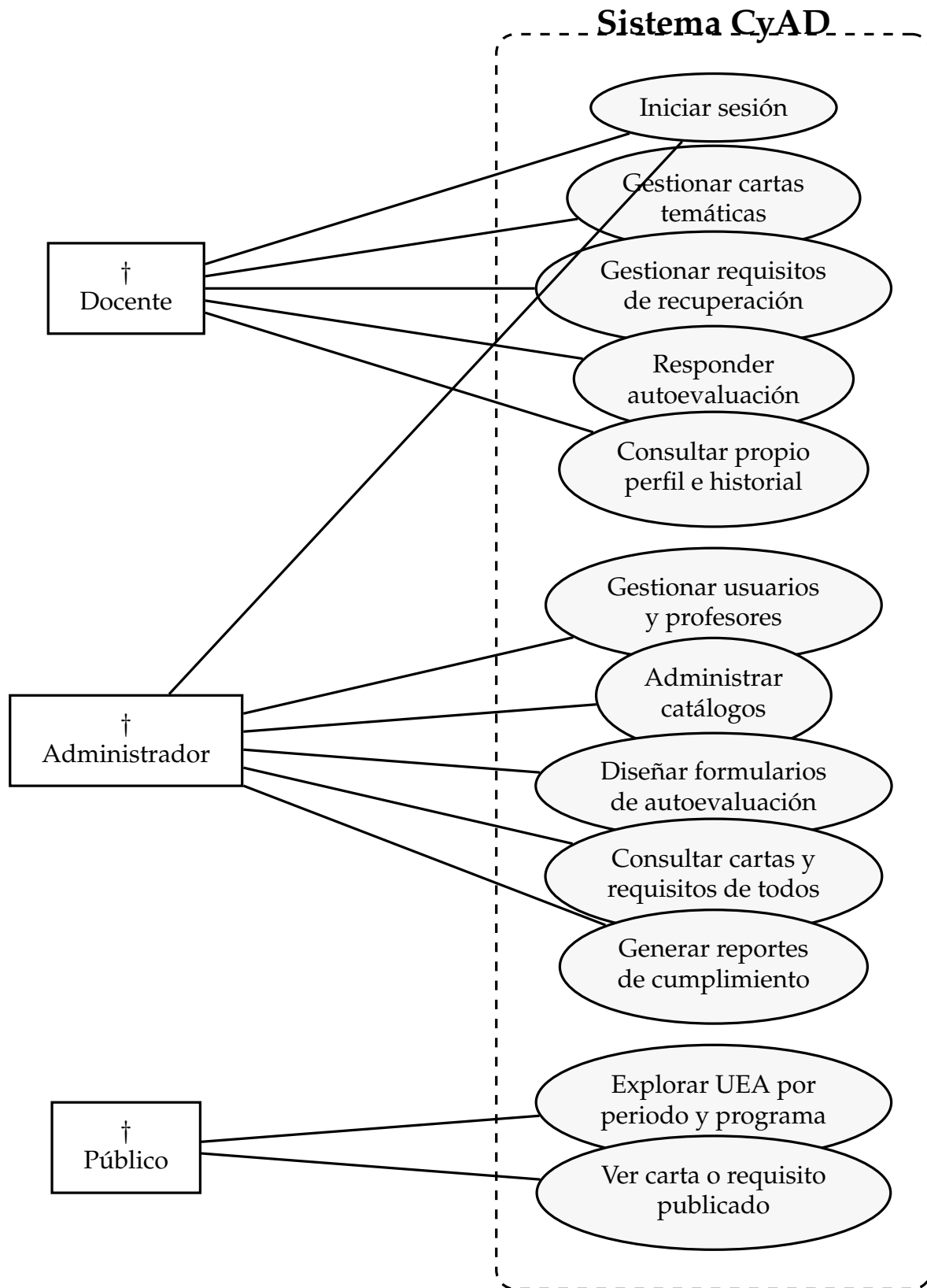


Figura 2. Diagrama de casos de uso del sistema.

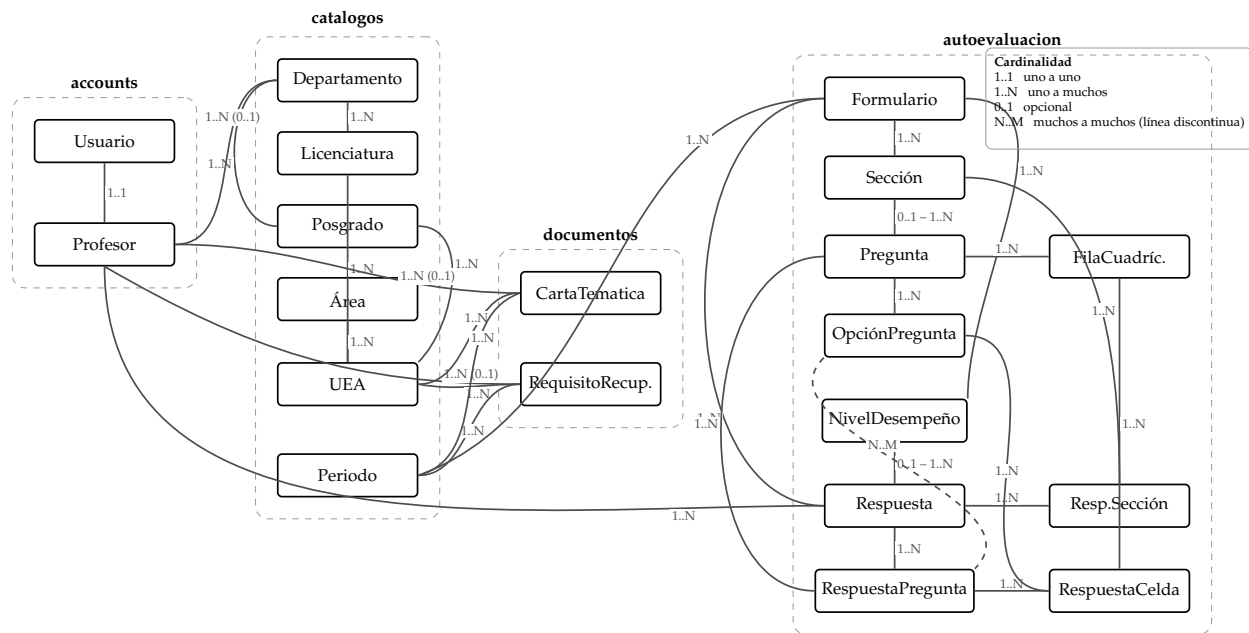


Figura 3. Diagrama Entidad-Relación del sistema (agrupado por app Django).

El modal "Nuevo Profesor" permite registrar un nuevo profesor. Los campos y valores mostrados son:

- DATOS DEL PROFESOR:**
 - Nombre completo ***: Dra. Ana María Ramírez Solís
 - Número económico**: 45678
 - Departamento**: Investigación y Conocimiento
- CUENTA DE ACCESO:**
 - Correo institucional ***: aramirez@azc.uam.mx
 - Contraseña inicial ***: TempClave2026!

Sección de botones: Cancelar, Crear Profesor.

Figura 4. Modal "Nuevo Profesor" desde /admin/profesores.

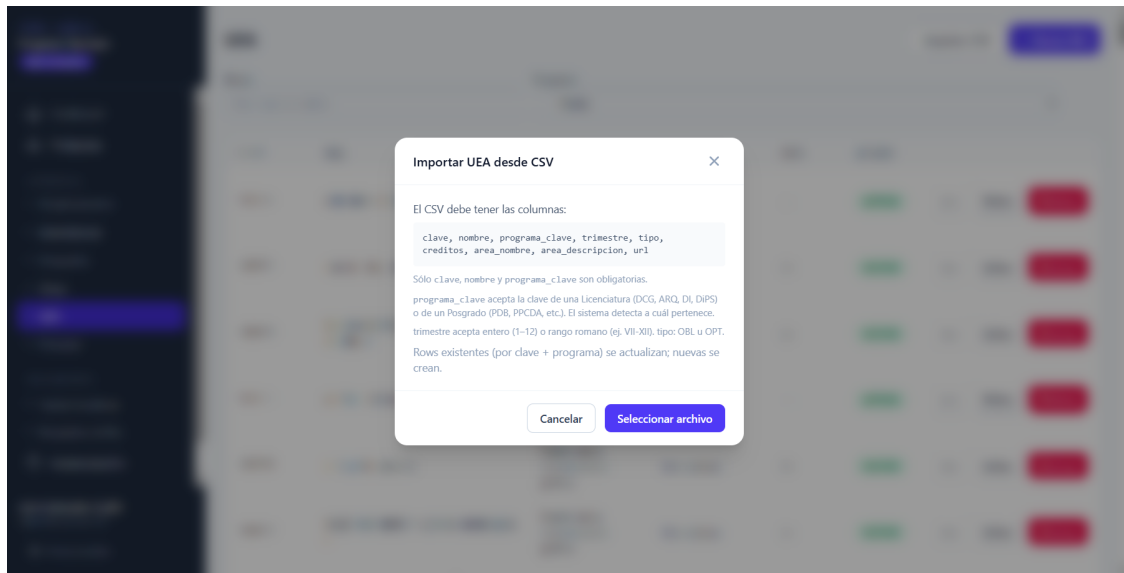


Figura 5. Importación masiva de UEAs desde CSV en /admin/catalogos/uea.

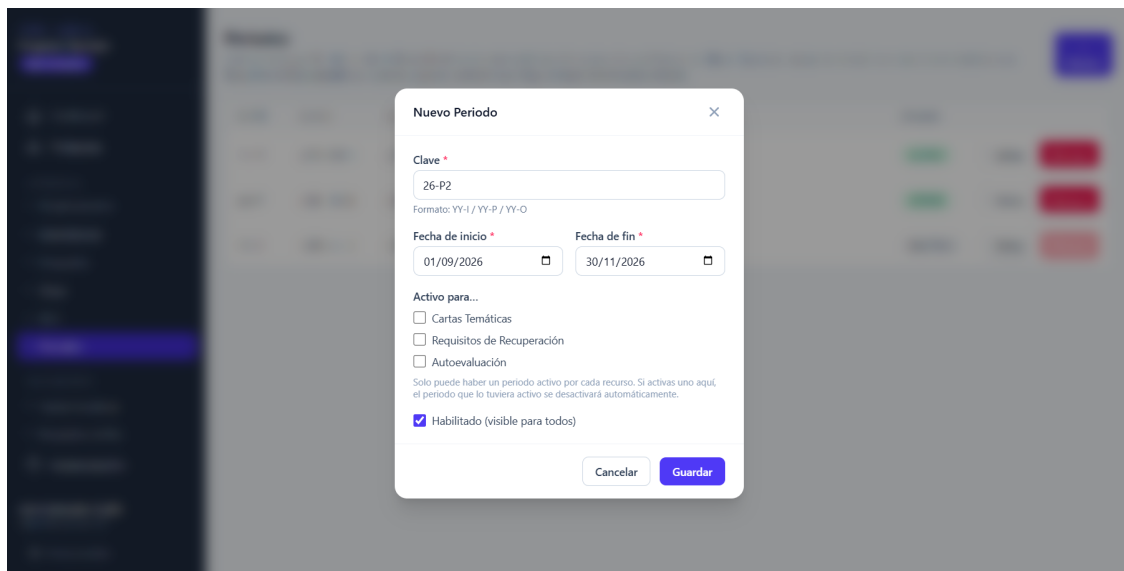
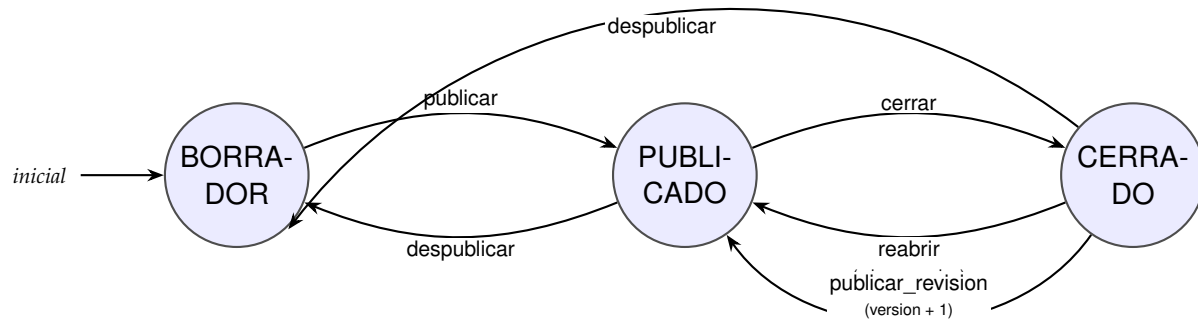


Figura 6. /admin/catalogos/periodos: tabla con *chips* de banderas por recurso y modal "Nuevo periodo".



Acción transversal duplicar: clona la estructura completa a otro periodo (sin arrastrar respuestas) desde cualquier estado.

Figura 7. Máquina de estados del Formulario de autoevaluación. Las transiciones que preservan la versión se dibujan a la izquierda entre PUBLICADO y CERRADO; la revisión con incremento de versión (publicar_revisión) se traza como arista externa.



Figura 8. Panel “Niveles de Desempeño” del constructor de autoevaluaciones.

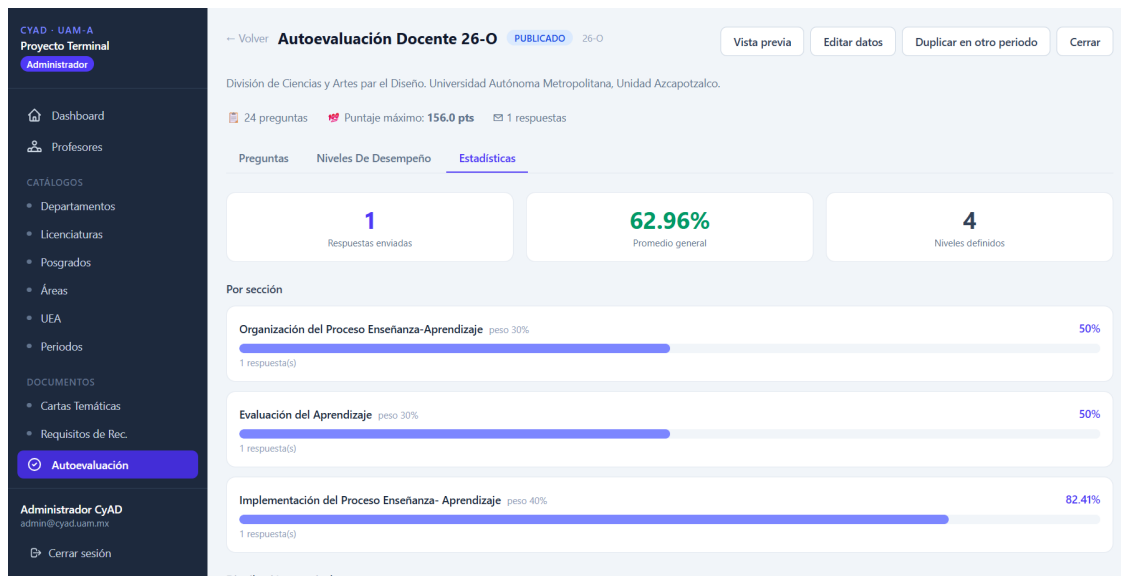


Figura 9. Pestaña “Estadísticas” del constructor con las respuestas ya enviadas: promedio general, porcentaje por sección, distribución por nivel de desempeño y por pregunta.

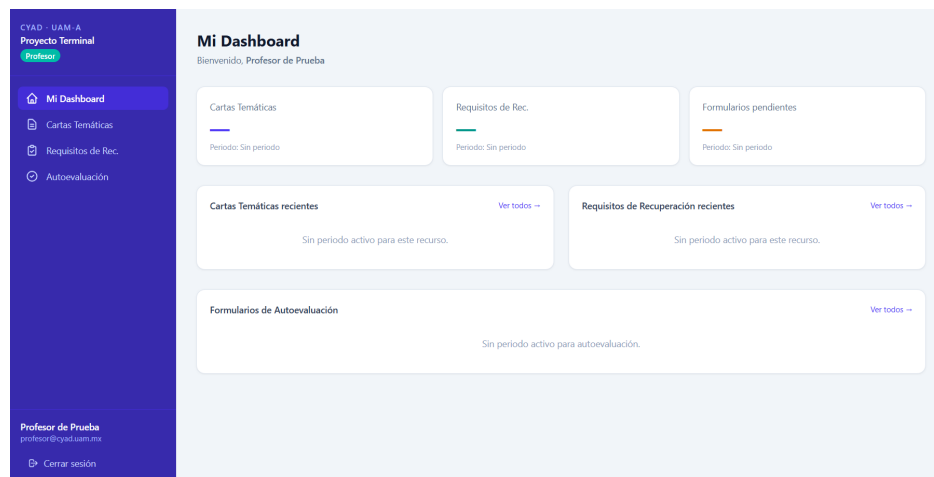


Figura 10. Pantalla de inicio de sesión.

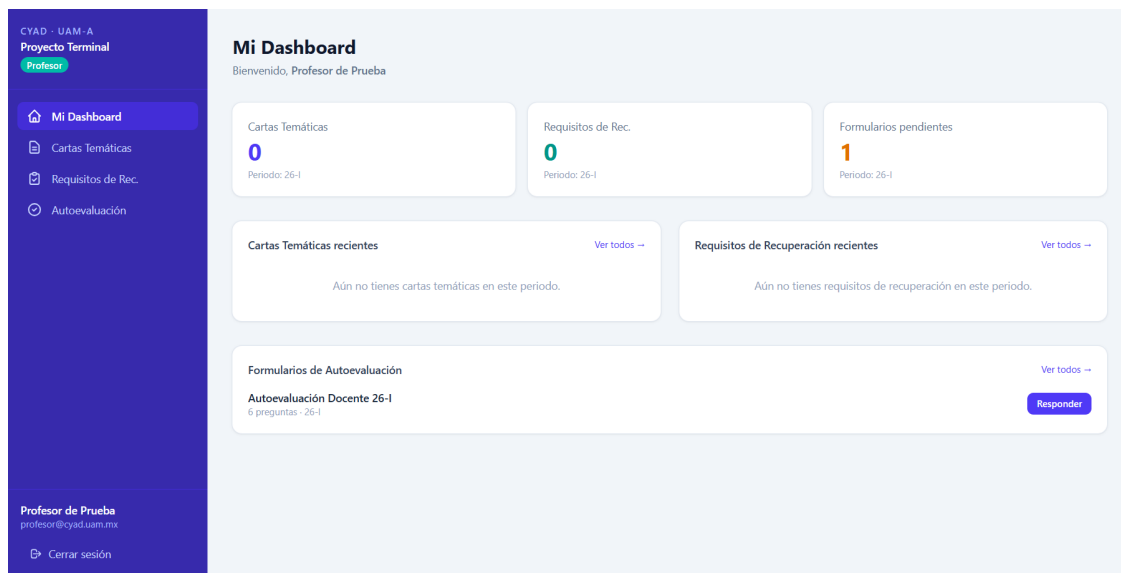


Figura 11. *Dashboard* del profesor: tarjetas de resumen y listas agrupadas por periodo.

The screenshot shows the 'Nueva Carta Temática' form. The form is divided into several sections. At the top, there's a header with 'CYAD - UAM-A Proyecto Terminal' and 'Profesor'. Below this is a sidebar with navigation links: 'Mi Dashboard', 'Cartas Temáticas', 'Requisitos de Rec.', and 'Autoevaluación'. The main content area shows the 'Nueva Carta Temática' form. The form includes a search bar for 'Selección de UEA' with a placeholder 'ej. 1100023 o Diseño Industrial'. Below this is a section for 'Información del grupo' with fields for 'Periodo' (assigned automatically), 'Modalidad' (Optional), 'Nombre del grupo', 'ID del grupo', and 'Horario'. At the bottom is a 'Contenido' section with a placeholder 'Todos los campos aceptan texto libre. Los que dejes vacíos no se mostrarán en la vista pública.'

Figura 12. Formulario de creación/edición de carta temática.

The screenshot shows the 'Nuevo Requisito de Recuperación' (New Recovery Requirement) form. The left sidebar contains the user's profile (CYAD - UAM-A, Proyecto Terminal, Profesor) and navigation links (Mi Dashboard, Cartas Temáticas, Requisitos de Rec., Autoevaluación). The main form area is divided into sections: 'Selección de UEA' with a search bar and instructions; 'Información del grupo' with fields for Periodo (26-I), Modalidad de la UEA (Opcional), Nombre del grupo, ID del grupo, and Horario; and 'Detalles de la evaluación' with a 'Lugar' field.

Figura 13. Formulario de requisitos de recuperación.

The screenshot shows the 'Autoevaluación Docente 26-I' (Teacher Self-Evaluation 26-I) form. The left sidebar is identical to Figure 13. The main form area has a title 'Autoevaluación Docente 26-I' and a subtitle 'Formulario de autoevaluación del periodo 26-I para profesores CyAD.' Below this is the 'Planeación' section with the instruction 'Reflexión sobre la planeación del curso.' The form contains two numbered questions: 1. '¿Con qué frecuencia entregaste la planeación del curso en tiempo?' with radio button options: Siempre, Casi siempre, A veces, and Nunca. 2. '¿Qué actividades de planeación realizaste? (marca todas las que apliquen)' with checkbox options: Carta temática actualizada, Calendarización de evaluaciones, Bibliografía revisada, and Coordinación con academia.

Figura 14. Formulario de autoevaluación con renderizado dinámico de preguntas.

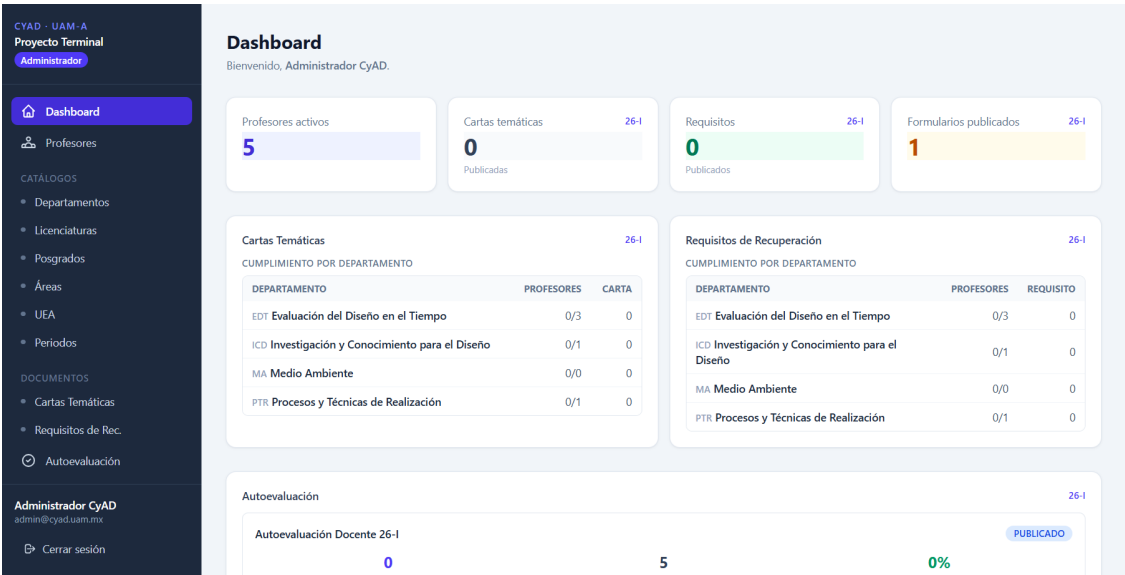


Figura 15. *Dashboard* del administrador: KPIs globales y cumplimiento por departamento.

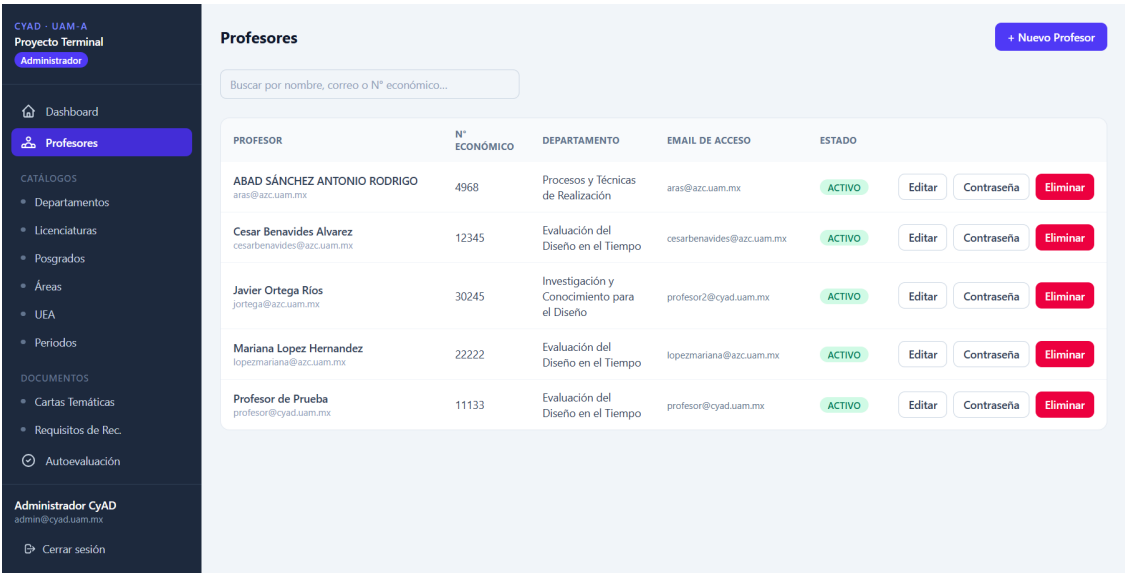


Figura 16. Gestión de profesores: crear, editar, restablecer contraseña.

CYAD · UAM-A
Proyecto Terminal
Administrador

Dashboard
Profesores
CATÁLOGOS
• Departamentos
• Licenciaturas
• Posgrados
• Áreas
• UEA
• Periodos
DOCUMENTOS
• Cartas Temáticas
• Requisitos de Rec.
Autoevaluación
Administrador CyAD
admin@cyad.uam.mx
Cerrar sesión

← Volver **Autoevaluación Docente 26-P completo** BORRADOR 26-P Vista previa Editar datos Duplicar en otro periodo Publicar

División de Ciencias y Artes par el Diseño, Universidad Autónoma Metropolitana, Unidad Azcapotzalco.

41 preguntas Puntaje máximo: 204.0 pts 0 respuestas

Preguntas Niveles De Desempeño Estadísticas

✓ 1 pesos = 100.0% + Añadir sección

Organización del Proceso Enseñanza-Aprendizaje
Los siguientes datos son relativos a la relación de la persona docente/alumnado/UEA

Peso: 15.00 % Editar

1 Opción única *obligatoria
Cubrí con los contenidos que indica el Programa de estudios para las UEA en:
- Menos del 50% (0.00 pts)
- 50% a 70% (1.00 pts)
- 70% a 80% (2.00 pts)
- 80% a 90% (3.00 pts)
- 90% a 100% (4.00 pts)

Organización del Proceso Enseñanza-Aprendizaje Editar

2 Opción única *obligatoria
Cubrí las actividades programadas en tiempo y forma:
- Menos del 50% (0.00 pts)
- 50%-60% (1.00 pts)
- 70% a 80% (2.00 pts)
- 80% a 90% (3.00 pts)
- 90 % a 100% (4.00 pts)

Organización del Proceso Enseñanza-Aprendizaje Editar

Figura 17. Constructor de formularios de autoevaluación (*form-builder*) con secciones y preguntas ponderadas.

CYAD · UAM-A
Proyecto Terminal
Administrador

Dashboard
Profesores
CATÁLOGOS
• Departamentos
• Licenciaturas
• Posgrados
• Áreas
• UEA
• Periodos
DOCUMENTOS
• Cartas Temáticas
• Requisitos de Rec.
Autoevaluación
Administrador CyAD
admin@cyad.uam.mx
Cerrar sesión

Reportes
Descarga de documentos.

Descargar Excel
Genera un Excel con los documentos publicados del tipo seleccionado. Si eliges "Todos los periodos", el archivo tendrá una hoja por cada periodo con datos. Filtra opcionalmente por departamento del profesor.

Tipo de documento
Cartas Temáticas

Periodo
Todos los periodos

Departamento
— Todos —

Descargar Excel

Figura 18. Panel de reportes con descarga en XLSX.

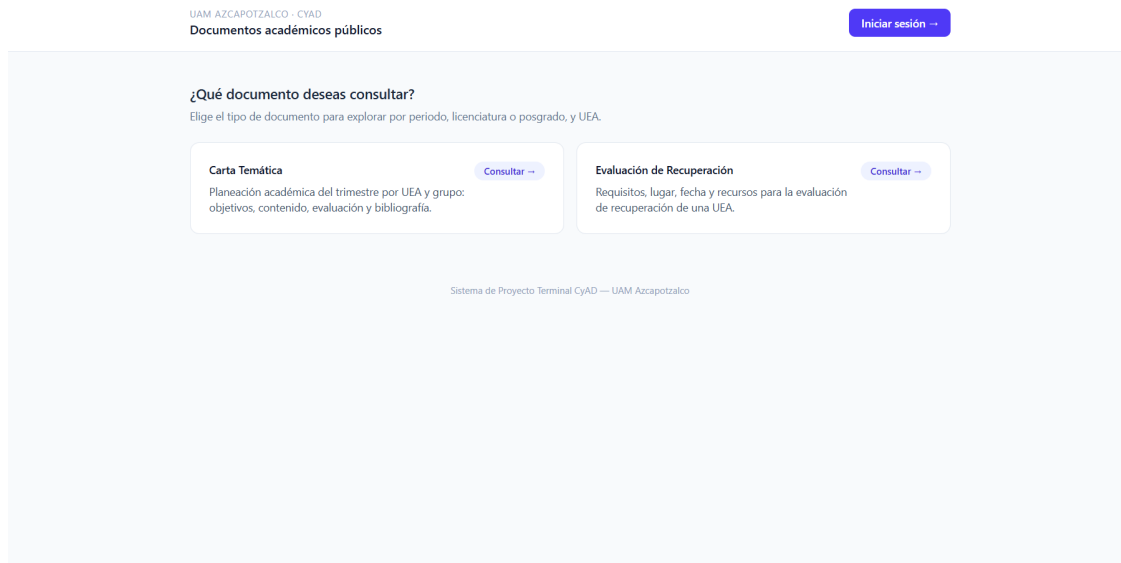


Figura 19. Portada pública: elección del tipo de documento a consultar.

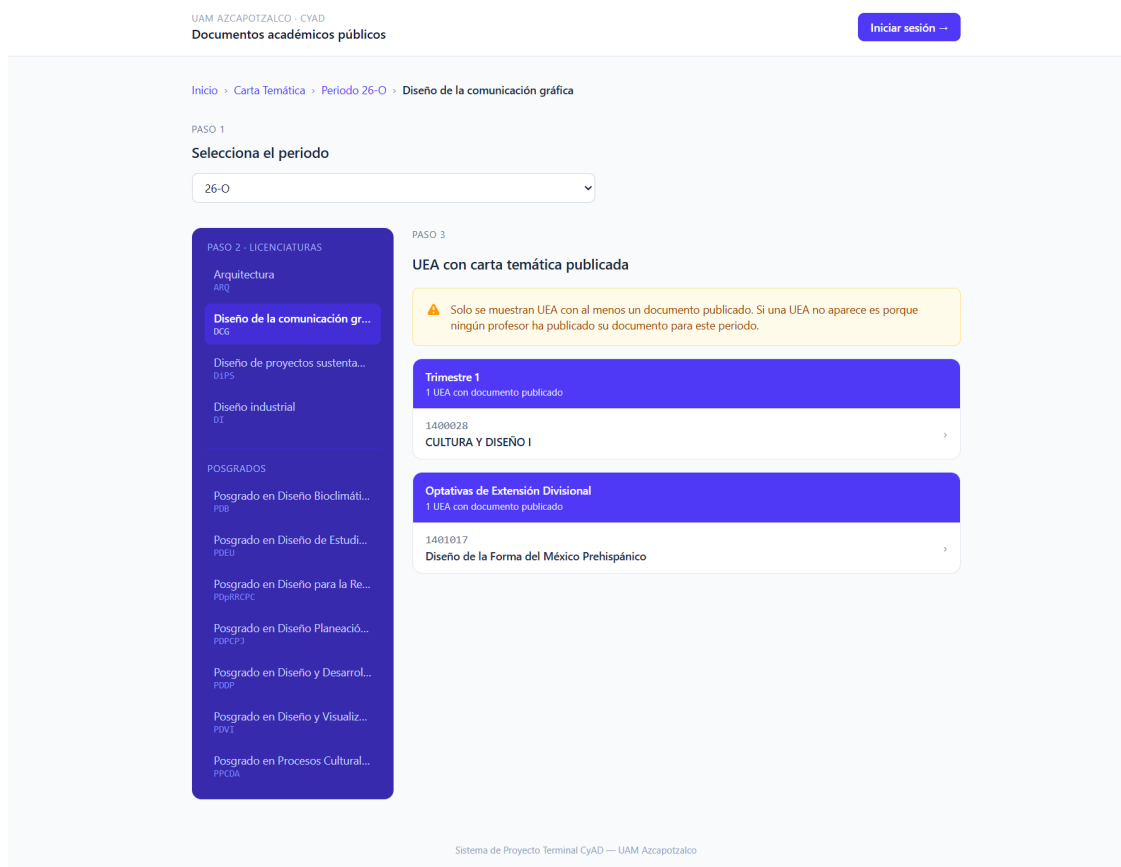


Figura 20. Vista pública *Explorar*: cascada periodo→programa→UEA con acordeón de documentos.

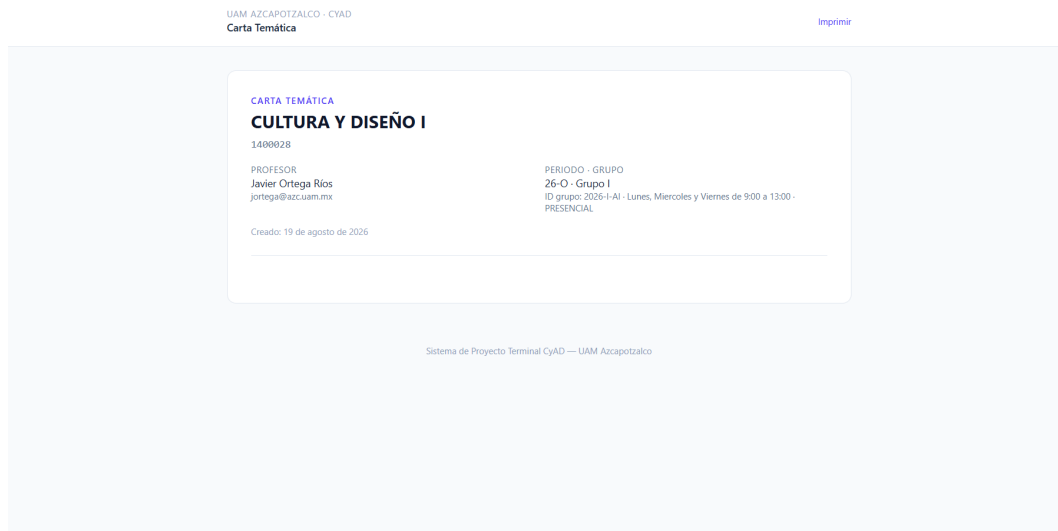


Figura 21. Vista pública de una carta temática publicada, con formato imprimible.

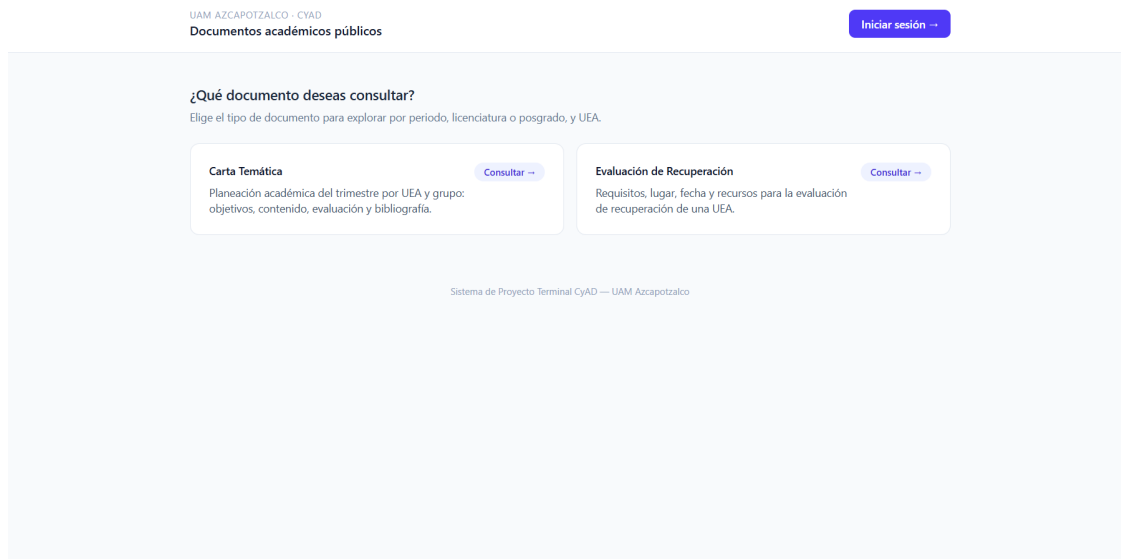


Figura 22. Portada pública: elección del tipo de documento a consultar.

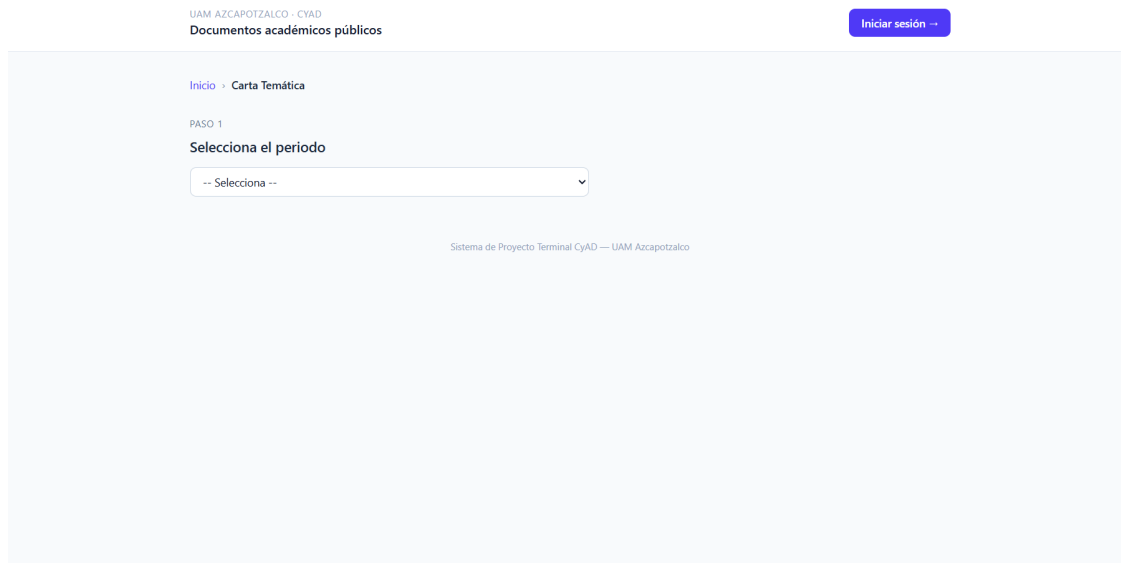


Figura 23. Vista pública “Explorar Carta Temática”, paso 1: selección de periodo.

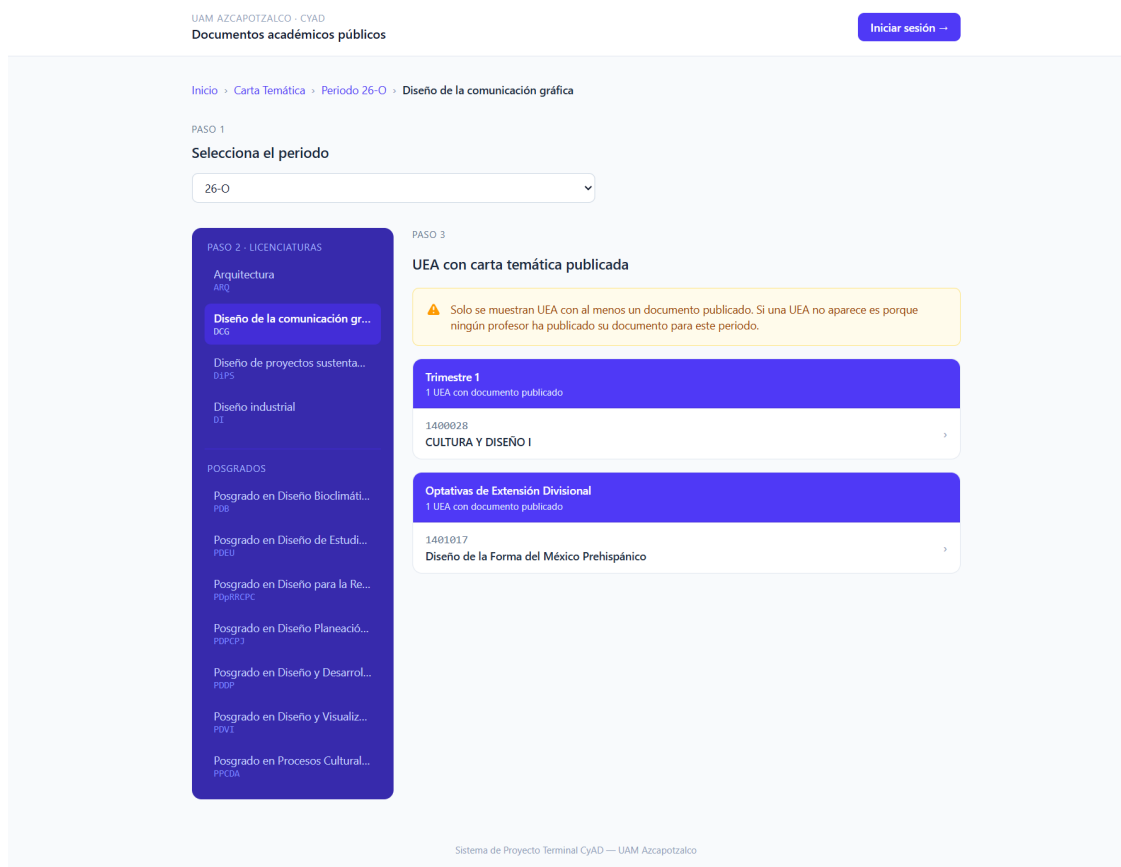


Figura 24. Consulta pública de Cartas Temáticas: periodo y licenciatura seleccionados, UEA con documento expandido.

UAM AZCAPOTZALCO - CYAD
Documentos académicos públicos

Inicio > Evaluación de Recuperación > Periodo 26-P > Diseño de proyectos sustentables > UEA 1400028

PASO 1
Selecciona el periodo

26-P

PASO 2 : LICENCIATURAS

- Arquitectura ARQ
- Diseño de la comunicación gr... DCG
- Diseño de proyectos sustentables... DIPS**
- Diseño industrial DI

POSGRADOS

- Posgrado en Diseño Bioclimático... PDB
- Posgrado en Diseño de Estudi... PDEU
- Posgrado en Diseño para la Re... PDPBPC
- Posgrado en Diseño Planeación... PDPFPI
- Posgrado en Diseño y Desarrol... PDDP
- Posgrado en Diseño y Visualiz... PDVI
- Posgrado en Procesos Cultural... PPDCA

PASO 3
UEA con evaluación de recuperación publicada

⚠ Solo se muestran UEA con al menos un documento publicado. Si una UEA no aparece es porque ningún profesor ha publicado su documento para este periodo.

Trimestre 1
1 UEA con documento publicado

1400028
CULTURA Y DISEÑO I

GRUPO	GRUPO A1 - ID 2026-I-A1	Ver documento →	GRUPO A3 - ID 2026-I-A3	Ver documento →
Javier Ortega Ríos	Lunes, Miércoles y Viernes de...	Publicado el 20 de julio de 2026	Profesor de Prueba	Lunes, Miércoles y Viernes de...
				Publicado el 19 de julio de 2026

Sistema de Proyecto Terminal CyAD — UAM Azcapotzalco

Figura 25. Consulta pública de Evaluaciones de Recuperación con UEA expandida.

UAM AZCAPOTZALCO - CYAD
Carta Temática

Imprimir

CARTA TEMÁTICA
CULTURA Y DISEÑO I
1400028

PROFESOR
Javier Ortega Ríos
jortega@azc.uam.mx

PERIODO : GRUPO
26-O · Grupo I
ID grupo: 2026-I-AI · Lunes, Miércoles y Viernes de 9:00 a 13:00 · PRESENCIAL

Creado: 19 de agosto de 2026

Sistema de Proyecto Terminal CyAD — UAM Azcapotzalco

Figura 26. Vista pública imprimible de una carta temática publicada.

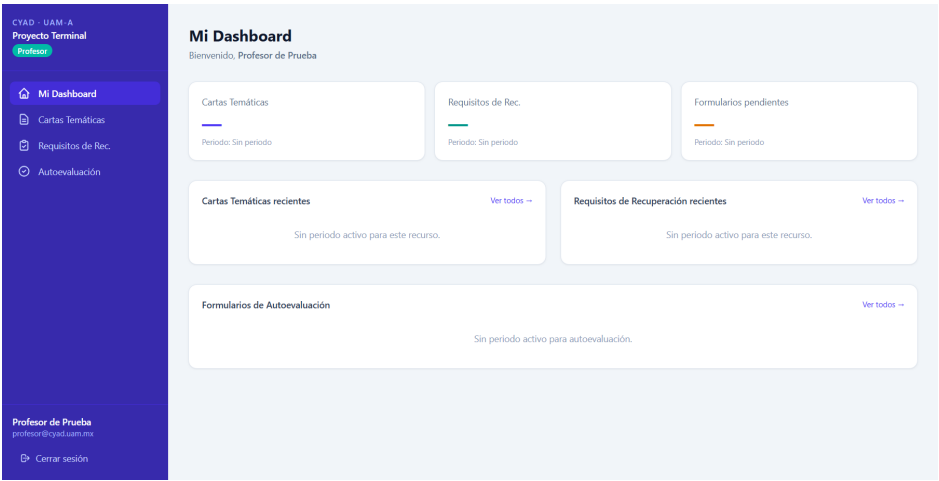


Figura 27. Pantalla de inicio de sesión.

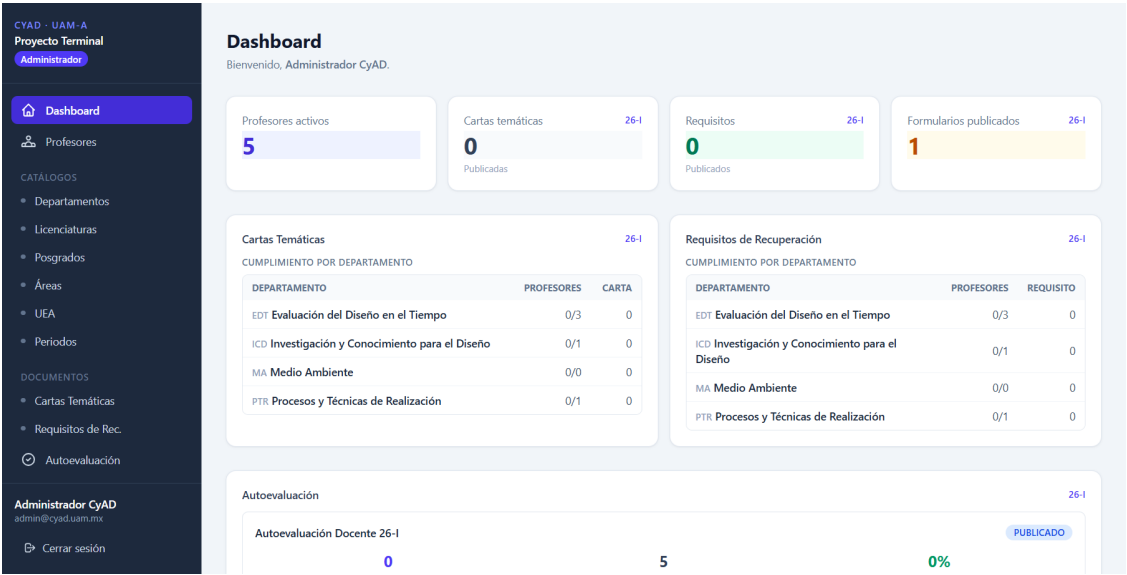
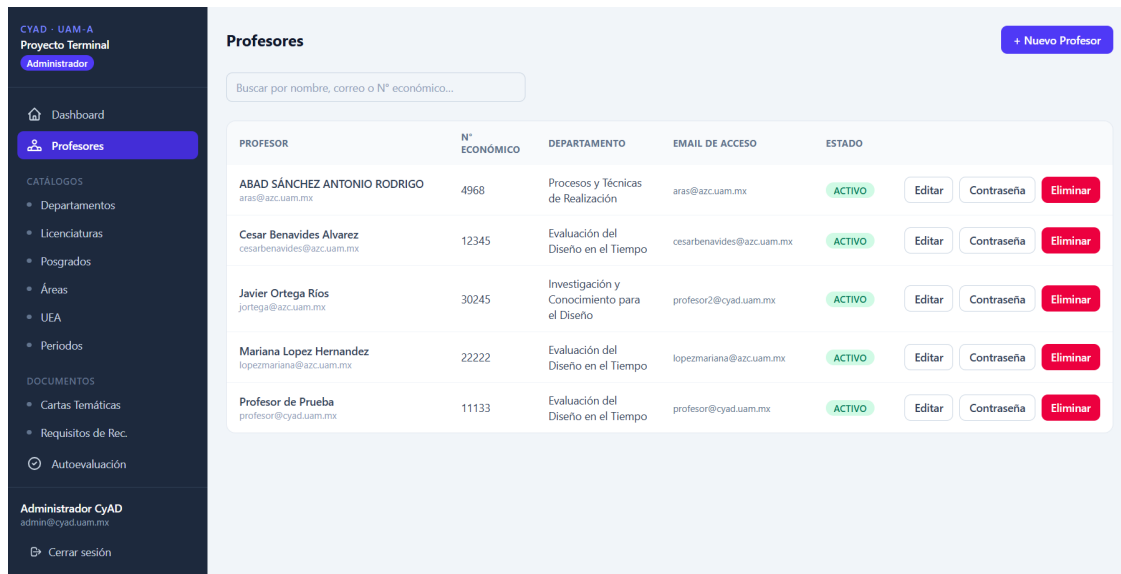
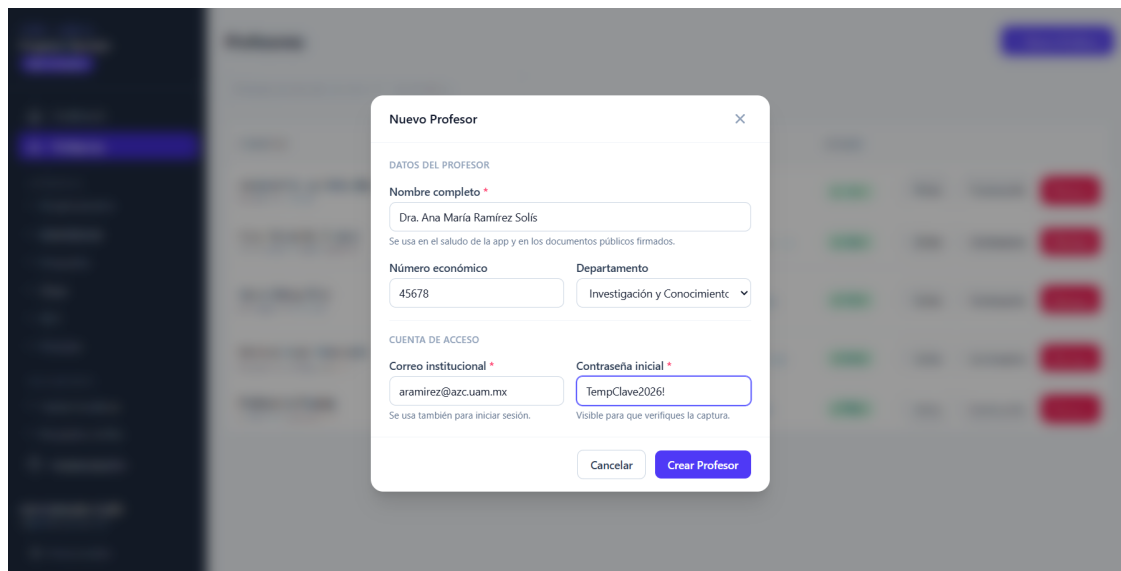


Figura 28. Dashboard del administrador con KPIs globales y cumplimiento por departamento.



PROFESOR	Nº ECONÓMICO	DEPARTAMENTO	EMAIL DE ACCESO	ESTADO	
ABAD SÁNCHEZ ANTONIO RODRIGO aras@azc.uam.mx	4968	Procesos y Técnicas de Realización	aras@azc.uam.mx	ACTIVO	<button>Editar</button> <button>Contraseña</button> <button>Eliminar</button>
Cesar Benavides Alvarez cesarbenavides@azc.uam.mx	12345	Evaluación del Diseño en el Tiempo	cesarbenavides@azc.uam.mx	ACTIVO	<button>Editar</button> <button>Contraseña</button> <button>Eliminar</button>
Javier Ortega Rios jortega@azc.uam.mx	30245	Investigación y Conocimiento para el Diseño	profesor2@cyad.uam.mx	ACTIVO	<button>Editar</button> <button>Contraseña</button> <button>Eliminar</button>
Mariana Lopez Hernandez lopezmariana@azc.uam.mx	22222	Evaluación del Diseño en el Tiempo	lopezmariana@azc.uam.mx	ACTIVO	<button>Editar</button> <button>Contraseña</button> <button>Eliminar</button>
Profesor de Prueba profesor@cyad.uam.mx	11133	Evaluación del Diseño en el Tiempo	profesor@cyad.uam.mx	ACTIVO	<button>Editar</button> <button>Contraseña</button> <button>Eliminar</button>

Figura 29. Lista de profesores registrados.



Nuevo Profesor

DATOS DEL PROFESOR

Nombre completo *
Dra. Ana María Ramírez Solís

Se usa en el saludo de la app y en los documentos públicos firmados.

Número económico
45678

Departamento
Investigación y Conocimiento

CUENTA DE ACCESO

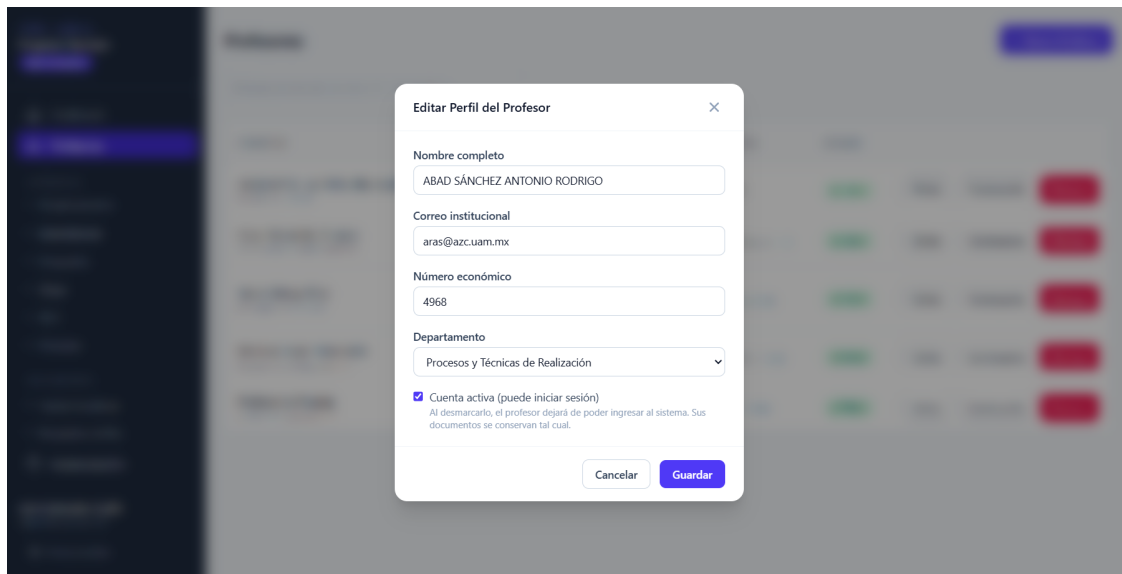
Correo institucional *
aramirez@azc.uam.mx

Contraseña inicial *
TempClave2026!

Se usa también para iniciar sesión. Visible para que verifiques la captura.

Cancelar Crear Profesor

Figura 30. Modal “Nuevo Profesor” con datos de ejemplo listos para guardar.



Editar Perfil del Profesor

Nombre completo
ABAD SÁNCHEZ ANTONIO RODRIGO

Correo institucional
aras@azc.uam.mx

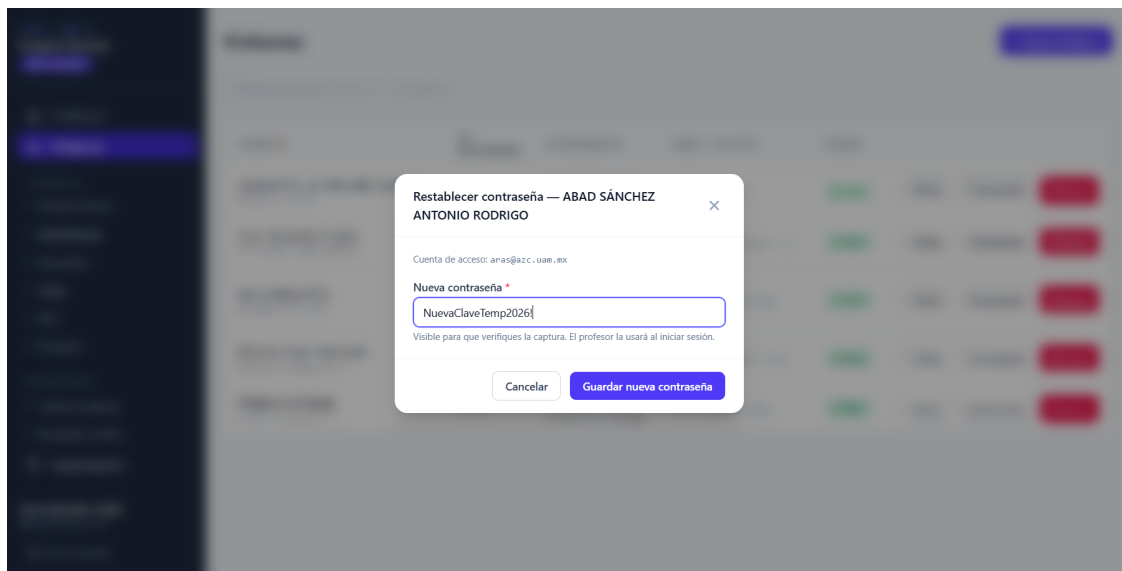
Número económico
4968

Departamento
Procesos y Técnicas de Realización

☒ Cuenta activa (puede iniciar sesión)
Al desmarcarla, el profesor dejará de poder ingresar al sistema. Sus documentos se conservan tal cual.

Cancelar Guardar

Figura 31. Modal “Editar Perfil del Profesor”.



Restablecer contraseña — ABAD SÁNCHEZ ANTONIO RODRIGO

Cuenta de acceso: aras@azc.uam.mx

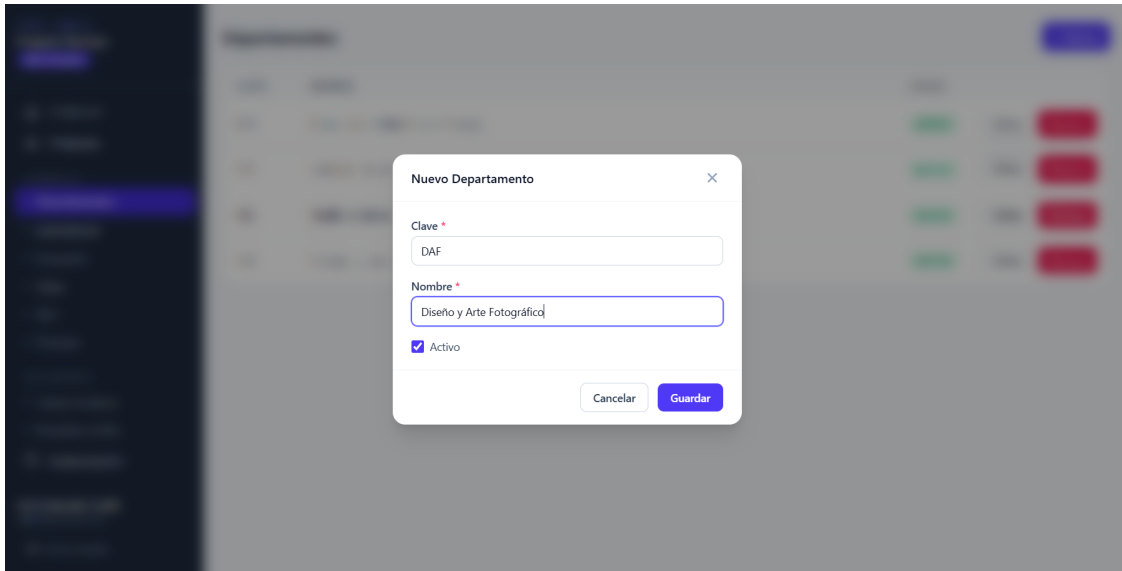
Nueva contraseña *

NuevaClaveTemp2020{

Visible para que verifiques la captura. El profesor la usará al iniciar sesión.

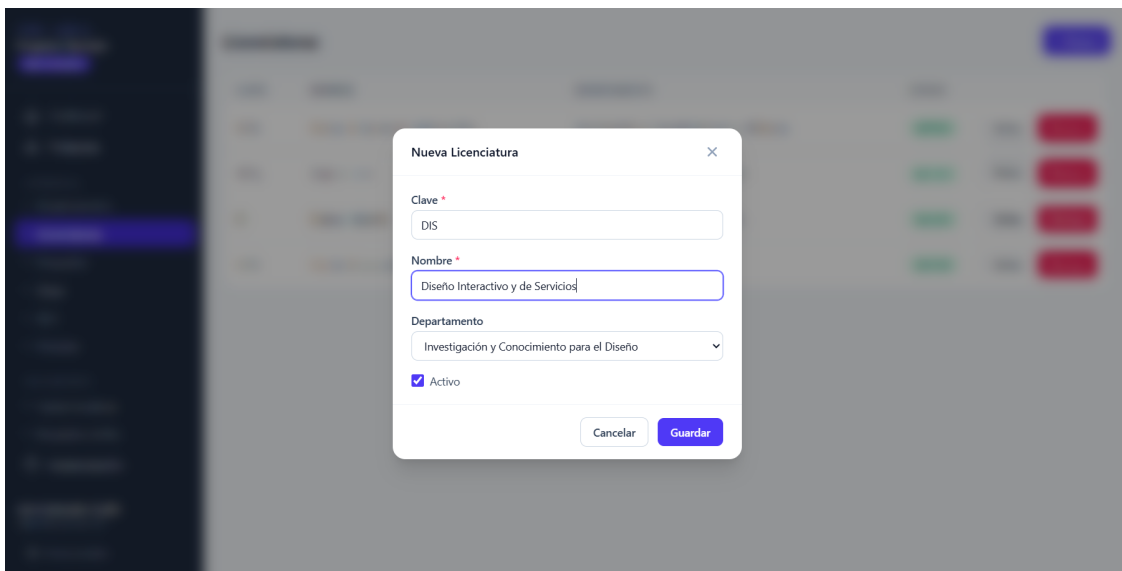
Cancelar Guardar nueva contraseña

Figura 32. Modal “Restablecer contraseña” de un profesor.



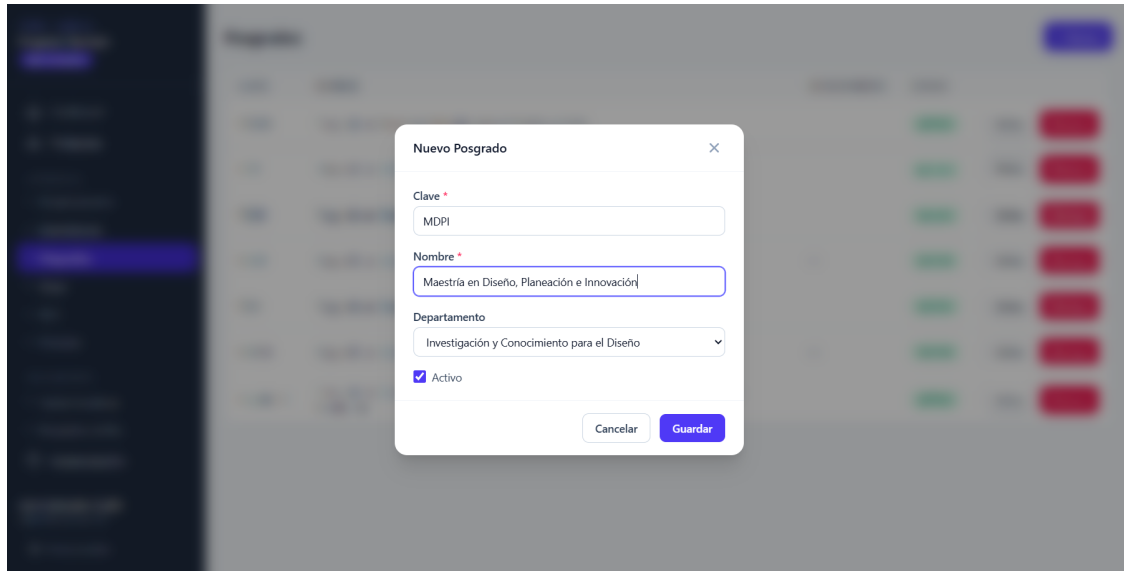
A screenshot of a web application showing a modal form titled "Nuevo Departamento". The form has a close button (X) in the top right corner. It contains two text input fields: "Clave *" with the value "DAF" and "Nombre *" with the value "Diseño y Arte Fotográfico". Below these fields is a checkbox labeled "Activo" which is checked. At the bottom of the form are two buttons: "Cancelar" and "Guardar". The background is a blurred view of a department list.

Figura 33. Nuevo departamento: clave y nombre.



A screenshot of a web application showing a modal form titled "Nueva Licenciatura". The form has a close button (X) in the top right corner. It contains three fields: a text input for "Clave *" with the value "DIS", a text input for "Nombre *" with the value "Diseño Interactivo y de Servicios", and a dropdown menu for "Departamento" with the selected value "Investigación y Conocimiento para el Diseño". Below these fields is a checkbox labeled "Activo" which is checked. At the bottom of the form are two buttons: "Cancelar" and "Guardar". The background is a blurred view of a department list.

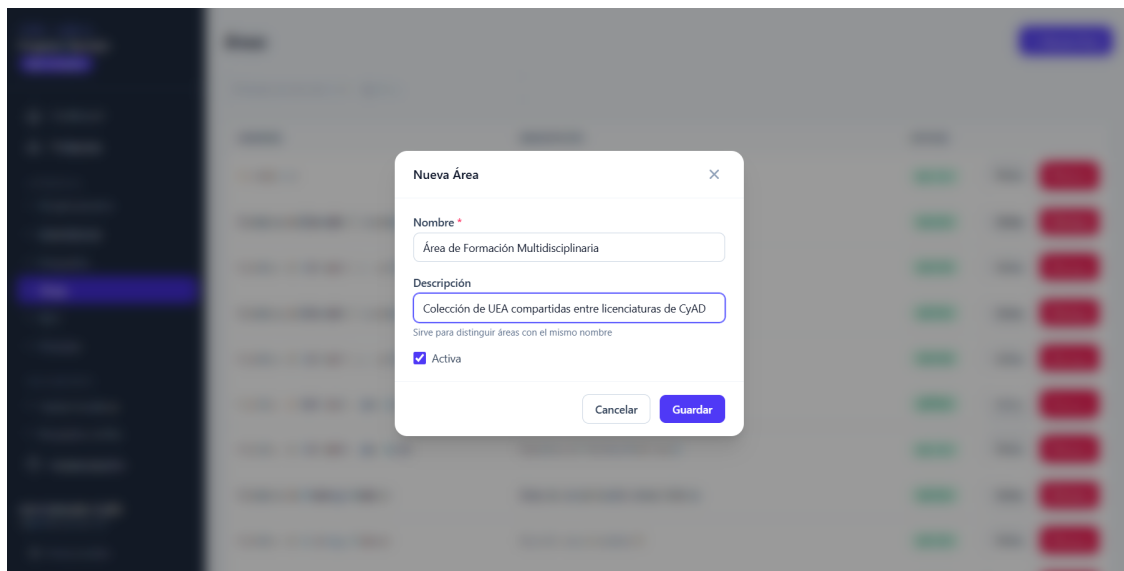
Figura 34. Nueva licenciatura: clave, nombre y departamento.



A screenshot of a web application showing a modal form titled "Nuevo Posgrado". The form has a close button (X) in the top right corner. It contains the following fields and controls:

- Clave ***: A text input field containing "MDPI".
- Nombre ***: A text input field containing "Maestría en Diseño, Planeación e Innovación".
- Departamento**: A dropdown menu with the selected option "Investigación y Conocimiento para el Diseño".
- Activo**: A checked checkbox.
- Buttons**: "Cancelar" and "Guardar" buttons at the bottom right.

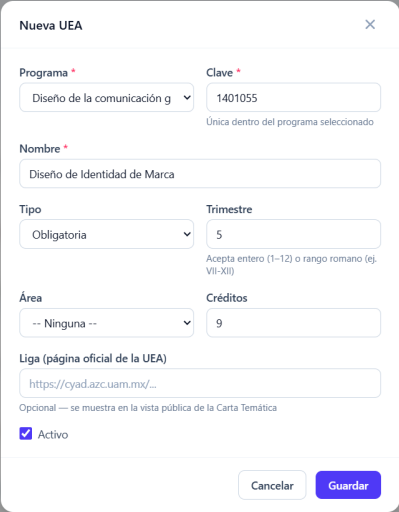
Figura 35. Nuevo posgrado: clave, nombre y departamento.



A screenshot of a web application showing a modal form titled "Nueva Área". The form has a close button (X) in the top right corner. It contains the following fields and controls:

- Nombre ***: A text input field containing "Área de Formación Multidisciplinaria".
- Descripción**: A text input field containing "Colección de UEA compartidas entre licenciaturas de CyAD". Below this field is a small note: "Sirve para distinguir áreas con el mismo nombre".
- Activa**: A checked checkbox.
- Buttons**: "Cancelar" and "Guardar" buttons at the bottom right.

Figura 36. Nueva área: nombre y descripción (permite distinguir áreas homónimas).



Formulario "Nueva UEA" para alta individual de una UEA. El formulario contiene los siguientes campos:

- Programa ***: Menú desplegable con la opción "Diseño de la comunicación g".
- Clave ***: Campo de texto con el valor "1401055". Debajo se indica: "Única dentro del programa seleccionado".
- Nombre ***: Campo de texto con el valor "Diseño de Identidad de Marca".
- Tipo**: Menú desplegable con la opción "Obligatoria".
- Trimestre**: Campo de texto con el valor "5". Debajo se indica: "Acepta entero (1-12) o rango romano (ej. VII-XII)".
- Área**: Menú desplegable con la opción "-- Ninguna --".
- Créditos**: Campo de texto con el valor "9".
- Liga (página oficial de la UEA)**: Campo de texto con el valor "https://cyad.azc.uam.mx/...". Debajo se indica: "Opcional — se muestra en la vista pública de la Carta Temática".
- Activo**: Casilla de verificación marcada con un checkmark.

En la parte inferior del formulario hay dos botones: "Cancelar" y "Guardar".

Figura 37. Alta individual de una UEA.



Formulario "Importar UEA desde CSV" para importación masiva de UEA desde un archivo CSV. El formulario contiene la siguiente información:

El CSV debe tener las columnas:

```
clave, nombre, programa_clave, trimestre, tipo,
creditos, area_nombre, area_descripcion, url
```

Sólo clave, nombre y programa_clave son obligatorias.

programa_clave acepta la clave de una Licenciatura (DCG, ARQ, DI, DIPS) o de un Posgrado (POB, PPCDA, etc.). El sistema detecta a cuál pertenece.

trimestre acepta entero (1-12) o rango romano (ej. VII-XII); tipo: OBL u OPT.

Rows existentes (por clave + programa) se actualizarán; nuevas se crean.

En la parte inferior del formulario hay dos botones: "Cancelar" y "Seleccionar archivo".

Figura 38. Importación masiva de UEA desde un archivo CSV.

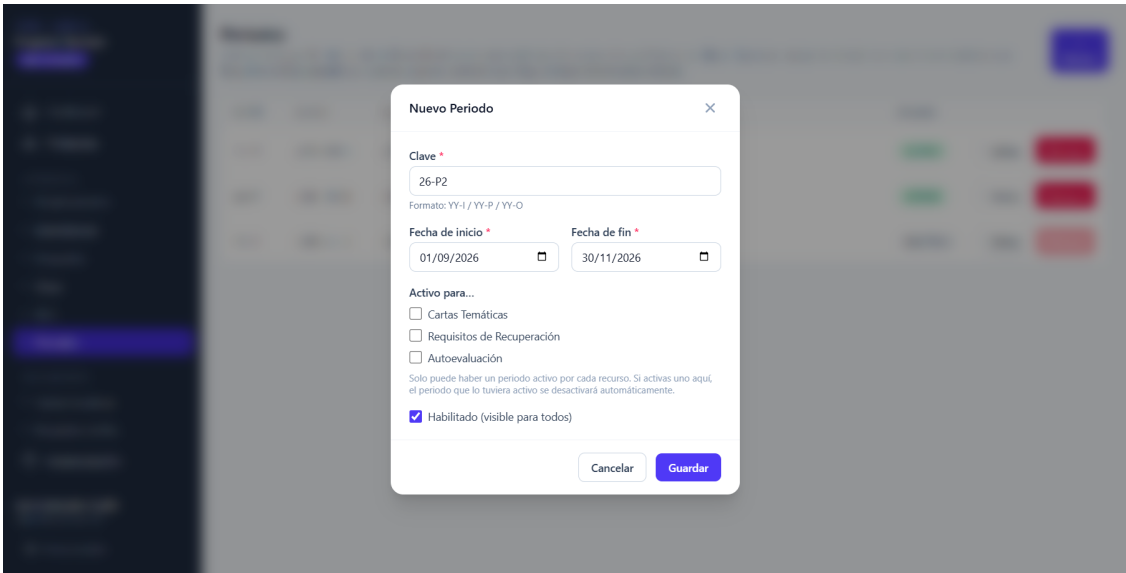


Figura 39. Gestión de periodos: tabla con los *chips* de banderas por recurso y modal de alta.

CYAD - UAM-A

Proyecto Terminal

Administrador

Dashboard

Profesores

CATALOGOS

- Departamentos
- Licenciaturas
- Posgrados
- Áreas
- UEA
- Periodos

DOCUMENTOS

- Cartas Temáticas
- Requisitos de Rec.

Autoevaluación

Administrador CyAD
admin@cyad.uam.mx

Cerrar sesión

Autoevaluación

Formularios de evaluación docente.

FORMULARIO

ESTADO

PREGUNTAS

RESPUESTAS

Autoevaluación Docente 26-I
26-I

PUBLICADO

6

0

Ver

Editar

Duplicar

Eliminar

Autoevaluación Docente 26-P completo
26-P

BORRADOR

41

0

Ver

Editar

Duplicar

Eliminar

Autoevaluación Docente 26-O
26-O

PUBLICADO

24

1

Ver

Editar

Duplicar

Eliminar

Autoevaluación Docente 26-P v2
26-P

PUBLICADO

24

0

Ver

Editar

Duplicar

Eliminar

Figura 40. Lista de formularios de autoevaluación con estado, periodo, preguntas y res-
puestas.

Nuevo Formulario de Autoevaluación

ⓘ No hay periodos habilitados para autoevaluación. Activa la autoevaluación en un periodo desde Catálogos antes de crear un formulario.

Título *
Autoevaluación Demo Manual (captura)

Descripción
Formulario de ejemplo generado durante la sesión de captura del manual de usuario.

Periodo *
-- Selecciona --

Solo se listan los periodos habilitados para autoevaluación.
Después de crearlo entrarás al editor de preguntas. Cuando esté listo, lo podrás publicar.

Cancelar Crear borrador

Figura 41. Modal “Nuevo Formulario de Autoevaluación”.

CYAD - UAM-A
Proyecto Terminal
Administrador

Dashboard
Profesores
CATALOGOS
• Departamentos
• Licenciaturas
• Posgrados
• Áreas
• UEA
• Periodos
DOCUMENTOS
• Cartas Temáticas
• Requisitos de Rec.
Autoevaluación
Administrador CyAD
admin@cyad.uam.mx
Cerrar sesión

Volver **Autoevaluación Docente 26-P completo** BORRADOR 26-P Vista previa Editar datos Duplicar en otro periodo Publicar

División de Ciencias y Artes par el Diseño. Universidad Autónoma Metropolitana, Unidad Azcapotzalco.

41 preguntas Puntaje máximo: 204.0 pts 0 respuestas

Preguntas Niveles De Desempeño Estadísticas

✓ 1 pesos = 100.0% + Añadir sección

Organización del Proceso Enseñanza-Aprendizaje Peso: 15.00 % Editar ✕
Los siguientes datos son relativos a la relación de la persona docente/alumnado/UEA

1 **Opción única** *obligatoria Organización del Proceso Enseñanza-Aprendizaje Editar ✕
Cubrí con los contenidos que indica el Programa de estudios para las UEA en:
- Menos del 50% (0.00 pts)
- 50% a 70% (1.00 pts)
- 70% a 80% (2.00 pts)
- 80% a 90% (3.00 pts)
- 90% a 100% (4.00 pts)

2 **Opción única** *obligatoria Organización del Proceso Enseñanza-Aprendizaje Editar ✕
Cubrí las actividades programadas en tiempo y forma:
- Menos del 50% (0.00 pts)
- 50%-60% (1.00 pts)
- 70% a 80% (2.00 pts)
- 80% a 90% (3.00 pts)
- 90 % a 100% (4.00 pts)

Figura 42. Constructor de formularios: pestaña “Preguntas” con secciones plegables y pesos.



Figura 43. Constructor de formularios: pestaña “Niveles de Desempeño”.

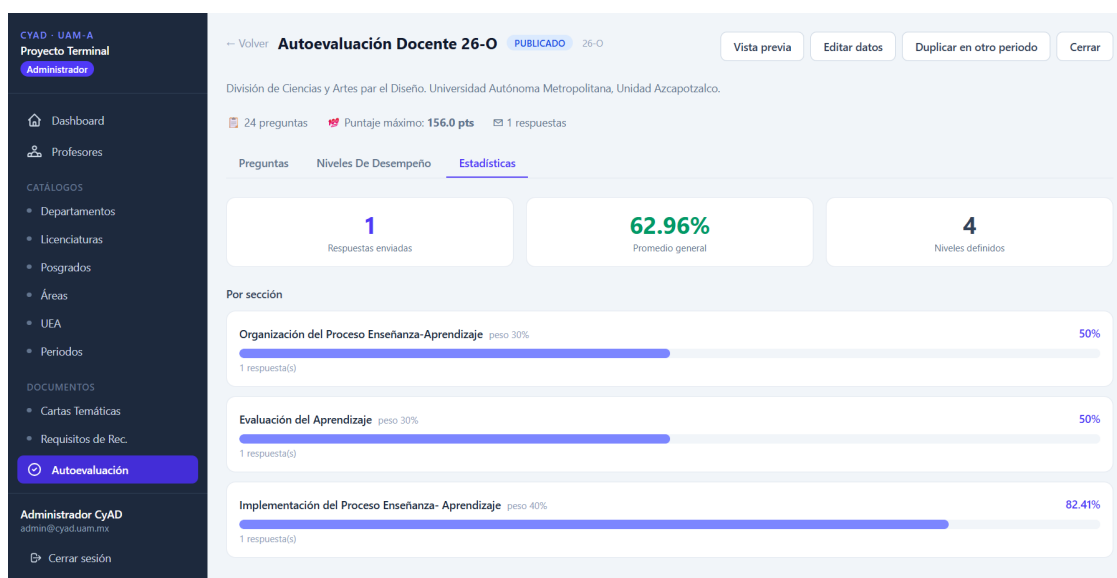


Figura 44. Constructor de formularios: pestaña “Estadísticas” con las respuestas ya enviadas.

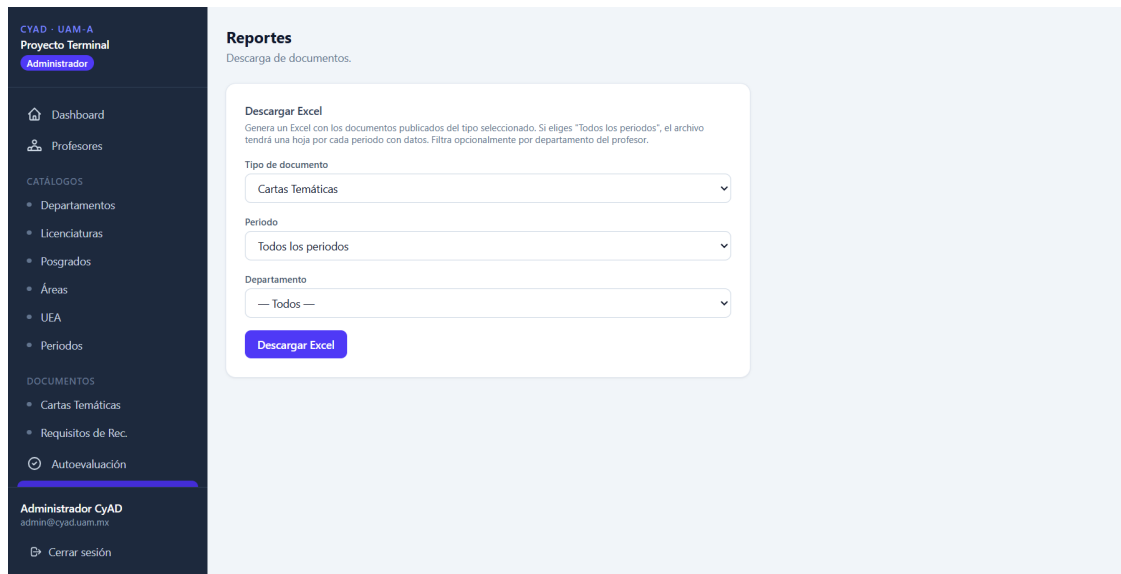


Figura 45. Panel de reportes con selección de tipo, periodo y departamento.

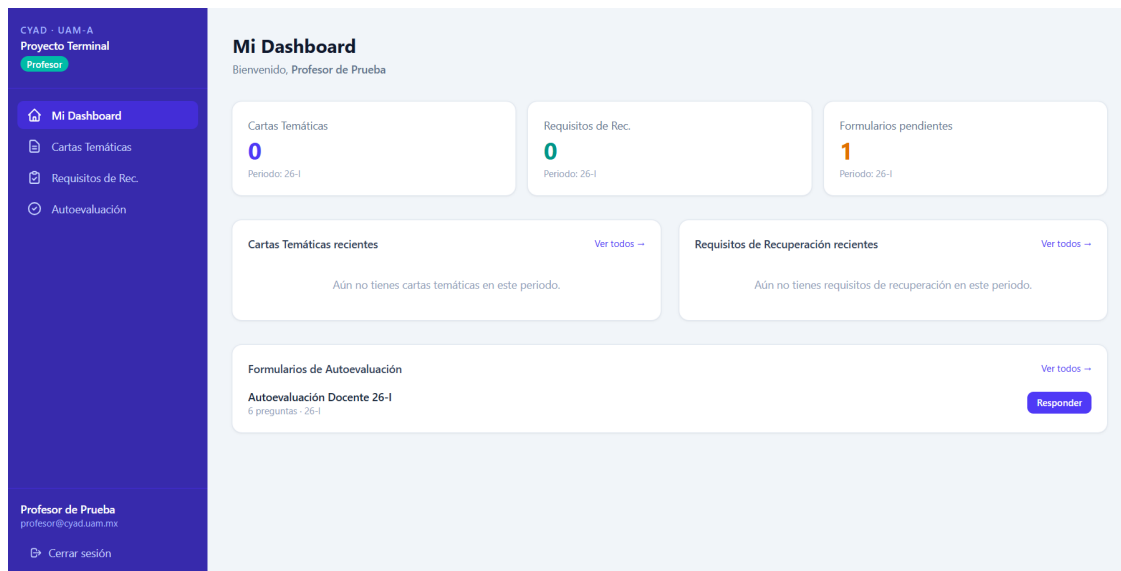


Figura 46. Dashboard del profesor.

The screenshot shows the 'Nueva Carta Temática' (New Thematic Card) form. On the left is a dark blue sidebar with the user's name 'CYAD - UAM-A Proyecto Terminal Profesor' and navigation links: 'Mi Dashboard', 'Cartas Temáticas' (highlighted), 'Requisitos de Rec.', and 'Autoevaluación'. At the bottom of the sidebar, it says 'Profesor de Prueba profesor@cyad.uam.mx' and 'Cerrar sesión'. The main content area has a title 'Nueva Carta Temática' and a 'Volver' link. It contains several sections: 'Selección de UEA' with a search bar and instructions; 'Información del grupo' with fields for 'Periodo' (26-I), 'Modalidad' (Opcional), 'Nombre del grupo' (Grupo A), 'ID del grupo' (2026-I-A1), and 'Horario' (Lun-Mié 9:00-11:00); and 'Contenido' with a note about text input.

Figura 47. Formulario de creación de carta temática.

The screenshot shows the 'Nuevo Requisito de Recuperación' (New Recovery Requirement) form. The sidebar is identical to the previous form. The main content area has a title 'Nuevo Requisito de Recuperación' and a 'Volver' link. It contains sections: 'Selección de UEA' with a search bar and instructions; 'Información del grupo' with fields for 'Periodo' (26-I), 'Modalidad de la UEA' (Opcional), 'Nombre del grupo' (Grupo A), 'ID del grupo' (2026-I-A1), and 'Horario' (Lun-Mié 9:00-11:00); and 'Detalles de la evaluación' with a 'Lugar' field.

Figura 48. Formulario de creación de evaluación de recuperación.

CYAD · UAM-A
Proyecto Terminal
Profesor

← Volver **Autoevaluación Docente 26-I**
Formulario de autoevaluación del periodo 26-I para profesores CyAD.

Planeación
Reflexión sobre la planeación del curso.

1 ¿Con qué frecuencia entregaste la planeación del curso en tiempo? *

☐ Siempre

☐ Casi siempre

☐ A veces

☐ Nunca

2 ¿Qué actividades de planeación realizaste? (marca todas las que apliquen) *

☐ Carta temática actualizada

☐ Calendarización de evaluaciones

☐ Bibliografía revisada

☐ Coordinación con academia

Profesor de Prueba
profesor@cyad.uam.mx

Cerrar sesión

Figura 49. Formulario de autoevaluación renderizado dinámicamente para el profesor.